Documentacion del Proyecto

mercadopago-carrito-backEnd

Volcado completo de codigo fuente

Generado el 08/11/2026

Total de archivos: 79

Indice de archivos

1. .gitignore
2. .windsurfignore
3. backEnd/package.json
4. backEnd/src/app.js
5. backEnd/src/config/environment.js
6. backEnd/src/controllers/chat.js
7. backEnd/src/controllers/pagos.js
8. backEnd/src/routes/chat.js
9. backEnd/src/routes/pagos.js
10. backEnd/src/services/ia.service.js
11. backEnd/src/services/mercadopago.service.js
12. eslint.config.js
13. firebaseIndicaciones.txt
14. index.html
15. INSTRUCCIONES_TARJETAS.txt
16. package.json
17. public/data/productos.json
18. public/favicon.svg
19. public/icons.svg
20. README.md
21. src/App.css
22. src/App.jsx
23. src/assets/react.svg
24. src/comentarios.txt
25. src/components/Administrar/admin.jsx
26. src/components/Administrar/FormularioProducto.jsx
27. src/components/Administrar/FormularioProducto.module.css
28. src/components/Administrar/GestionProductos.jsx
29. src/components/Administrar/GestionProductos.module.css
30. src/components/Administrar/PromoverAdmin.jsx
31. src/components/Administrar/PromoverAdmin.module.css
32. src/components/Administrar/TablaOrdenes.jsx
33. src/components/Administrar/TablaOrdenes.module.css
34. src/components/Auth/Login.jsx
35. src/components/Auth/Login.module.css
36. src/components/Auth/Registro.jsx
37. src/components/Carrito/Carrito.jsx
38. src/components/Carrito/Carrito.module.css
39. src/components/Carrito/CarritoItem.jsx
40. src/components/Carrito/CarritoItem.module.css
41. src/components/CatalogoProductos/CatalogoProductos.jsx
42. src/components/CatalogoProductos/CatalogoProductos.module
43. src/components/CatalogoProductos/constans.jsx
44. src/components/ChatIA/ChatIA.jsx
45. src/components/ChatIA/ChatIA.module.css
46. src/components/Item/Item.jsx
47. src/components/Item/Item.module.css
48. src/components/Layout/Footer.jsx
49. src/components/Layout/Footer.module.css
50. src/components/Layout/Header.jsx
51. src/components/Layout/Header.module.css
52. src/components/Layout/Layout.jsx
53. src/components/Layout/Layout.module.css
54. src/components/Layout/Navbar.jsx
55. src/components/Layout/Navbar.module.css
56. src/components/Pago/PagoExitoso.jsx
57. src/components/Pago/PagoExitoso.module.css
58. src/components/Pago/PagoFallido.jsx
59. src/components/Pago/PagoFallido.module.css
60. src/components/Pago/PagoPendiente.jsx
61. src/components/Pago/PagoPendiente.module.css
62. src/components/ProductoDetalle/ProductoDetalle.jsx
63. src/components/ProductoDetalle/ProductoDetalle.module.css
64. src/components/RutaPrivadaElementales.jsx
65. src/context/auth/AuthContext.jsx
66. src/context/auth/AuthProvider.jsx
67. src/context/cart/CartContext.jsx
68. src/context/cart/CartProvider.jsx
69. src/firebase/config.js
70. src/main.jsx
71. src/services/cargaProductos.jsx
72. src/services/migrarDatosIniciales.js
73. src/services/ordenesService.js
74. src/services/pruebaLectura.js
75. src/styles/globals.css
76. src/styles/index.css
77. test.js
78. vercel.json
79. vite.config.js

.gitignore · 27 lineas

```
# Logs
logs
*.log
npm-debug.log*
yarn-debug.log*
yarn-error.log*
pnpm-debug.log*
lerna-debug.log*

node_modules
dist
dist-ssr
*.local
.env
.env.*

# Editor directories and files
.vscode/*
!.vscode/extensions.json
.idea
.DS_Store
*.suo
*.ntvs*
*.njsproj
*.sln
*.sw?
```

.windsurfignore · 8 lineas

```
node_modules/
dist/
build/

.git/
coverage/
.vscode/
*.log
```

backEnd/package.json JSON · 18 lineas

```
{
  "name": "mercadopago-carrito-backEnd",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "start": "node src/app.js",
    "dev": "node --watch src/app.js"
  },
  "dependencies": {
    "@google/generative-ai": "^0.24.1",
    "cors": "^2.8.6",
    "dotenv": "^17.4.2",
    "express": "^5.2.1",
    "mercadopago": "^3.2.0"
  }
}
```

backEnd/src/app.js JS · 21 lineas

```
import express from 'express';
import cors from 'cors';
import { pagosRouter } from './routes/pagos.js';
import { chatRouter } from './routes/chat.js';

const app = express();

app.use(cors());
app.use(express.json());

// Registro de endpoints utilizando las importaciones nombradas corregidas
app.use("/pagos", pagosRouter);
app.use("/chat", chatRouter);

console.log("Puerto donde se levanta el servidor en render :", process.env.PORT);
const PORT = process.env.PORT || 3001;

app.listen(PORT, () => {
  console.log(`Servidor corriendo en el puerto ${PORT}`);
});
```

backEnd/src/config/environment.js JS · 22 lineas

```
import { MercadoPagoConfig } from 'mercadopago';
import dotenv from 'dotenv';

// Asegura leer las variables del .env
dotenv.config();

export const CLIENT_URL = process.env.CLIENT_URL || "http://localhost:5173";
export const GROQ_KEY = process.env.GROQ_API_KEY;
export const OPENROUTER_KEY = process.env.OPENROUTER_API_KEY;

export const GROQ_MODEL = "llama-3.1-8b-instant";
export const OPENROUTER_MODEL = "meta-llama/llama-3.1-8b-instruct";

export const client = new MercadoPagoConfig({
  accessToken: process.env.MERCADOPAGO_ACCESS_TOKEN
});

console.log("¿El Token existe?:", !!process.env.MERCADOPAGO_ACCESS_TOKEN);
if (process.env.MERCADOPAGO_ACCESS_TOKEN) {
  console.log("Inicio del token detectado:", process.env.MERCADOPAGO_ACCESS_TOKEN.substring(0, 10) + "...");
}
```

backEnd/src/controllers/chat.js JS · 17 lineas

```
import { procesarChat } from '../services/ia.service.js';

export const enviarMensaje = async (req, res) => {
  const { mensaje } = req.body;

  if (!mensaje) {
    return res.status(400).json({ texto: "Falta el mensaje del usuario." });
  }

  try {
    const resultado = await procesarChat(mensaje);
    return res.json(resultado);
  } catch (error) {
    return res.status(500).json({ texto: "El servicio de IA no está disponible." });
  }
};
```

backEnd/src/controllers/pagos.js JS · 36 lineas

```
import { crearPreferencia } from '../services/mercadopago.service.js';

export const crearPago = async (req, res) => {
  try {
    const { items, totalFinal, orderId } = req.body;
    console.log("orderId recibido:", orderId);
    console.log("totalFinal", totalFinal);

    if (!totalFinal || isNaN(totalFinal)) {
      return res.status(400).json({ error: "El precio total no es válido o está vacío." });
    }

    if (!items || items.length === 0) {
      return res.status(400).json({ error: "El carrito está vacío" });
    }

    const result = await crearPreferencia(totalFinal, orderId);

    console.log(JSON.stringify(result, null, 2));

    res.json({
      id: result.id,
      init_point: result.init_point
    });
    console.log("Preference ID:", result.id);
    console.log("Collector ID:", result.collector_id);
    console.log("Init Point:", result.init_point);

  } catch (error) {
    console.error("Error creando preferencia:", error?.message || error);
    if (error?.cause) console.error("Causa:", error.cause);
    if (error?.status) console.error("HTTP status MP:", error.status);
    res.status(500).json({ error: "Error al generar el pago", detalle: error?.message });
  }
};
```

backEnd/src/routes/chat.js JS · 10 lineas

```
import express from 'express';
import { enviarMensaje } from '../controllers/chat.js';

const router = express.Router();

router.post("/", enviarMensaje);

export const chatRouter = router;
```

backEnd/src/routes/pagos.js JS · 9 lineas

```
import express from 'express';
import { crearPago } from '../controllers/pagos.js';

const router = express.Router();

router.post("/", crearPago);

export const pagosRouter = router;
```

backEnd/src/services/ia.service.js JS · 60 lineas

```
import { GROQ_KEY, OPENROUTER_KEY, CLIENT_URL, GROQ_MODEL, OPENROUTER_MODEL } from '../config/environment.js';

function buildBody(mensaje, model) {
  return JSON.stringify({
    model,
    messages: [
      {
        role: "system",
        content: "Eres un vendedor experto de laptops. Responde claro, concreto y recomienda modelos según uso.",
      },
      {
        role: "user",
        content: mensaje,
      },
    ],
  });
}

export const procesarChat = async (mensaje) => {
  try {
    const response = await fetch("https://api.groq.com/openai/v1/chat/completions", {
      method: "POST",
      headers: {
        Authorization: `Bearer ${GROQ_KEY}`,
        "Content-Type": "application/json",
      },
      body: buildBody(mensaje, GROQ_MODEL),
    });

    const data = await response.json();

    if (data.choices?.[0]?.message?.content) {
      return { texto: data.choices[0].message.content };
    }
  } catch (error) {
    console.warn("Groq no disponible:", error.message);
  }

  try {
    const response = await fetch("https://openrouter.ai/api/v1/chat/completions", {
      method: "POST",
      headers: {
        Authorization: `Bearer ${OPENROUTER_KEY}`,
        "Content-Type": "application/json",
        "HTTP-Referer": CLIENT_URL,
        "X-Title": "Chat Laptop Store",
      },
      body: buildBody(mensaje, OPENROUTER_MODEL),
    });

    const data = await response.json();
    const texto = data.choices?.[0]?.message?.content || data.error?.message || "Sin respuesta";
    return { texto };

  } catch (error) {
    console.error("OpenRouter también falló:", error.message);
    throw error;
  }
};
```

backEnd/src/services/mercadopago.service.js JS · 31 lineas

```
import { Preference } from 'mercadopago';
import { client, CLIENT_URL } from '../config/environment.js';

export const crearPreferencia = async (totalFinal, orderId) => {
  const itemsMercadoPago = [
    {
      title: "Compra tienda online",
      quantity: 1,
      unit_price: Math.round(Number(totalFinal) * 100) / 100,
      currency_id: "ARS",
    },
  ];

  const preference = new Preference(client);

  console.log(JSON.stringify(itemsMercadoPago, null, 2));

  const result = await preference.create({
    body: {
      items: itemsMercadoPago,
      back_urls: {
        success: `${CLIENT_URL}/pago-exitoso?orderId=${orderId}`,
        failure: `${CLIENT_URL}/pago-fallido?orderId=${orderId}`,
        pending: `${CLIENT_URL}/pago-pendiente?orderId=${orderId}`,
      },
    }
  });

  return result;
};
```

eslint.config.js JS · 30 lineas

```
import js from '@eslint/js'
import globals from 'globals'
import reactHooks from 'eslint-plugin-react-hooks'
import reactRefresh from 'eslint-plugin-react-refresh'
import { defineConfig, globalIgnores } from 'eslint/config'

export default defineConfig([
  globalIgnores(['dist']),
  {
    files: ['**/*.{js,jsx}'],
    extends: [
      js.configs.recommended,
      reactHooks.configs.flat.recommended,
      reactRefresh.configs.vite,
    ],
    languageOptions: {
      ecmaVersion: 2020,
      globals: globals.browser,
      parserOptions: {
        ecmaVersion: 'latest',
        ecmaFeatures: { jsx: true },
        sourceType: 'module',
      },
    },
    rules: {
      'no-unused-vars': ['error', { varsIgnorePattern: '^[A-Z_]' }],
    },
  },
])
```

firebaseIndicaciones.txt TXT · 138 lineas

```
En esta serie vamos a desarrollar paso a paso una aplicación
de e-commerce con React, Firebase y Mercado Pago.
No solo veremos cómo escribir el código, sino también
por qué funciona cada parte y cuándo conviene utilizarla.
 Si este contenido te resulta útil, te invito a darle 'Me gusta',
 suscribirte al canal y activar las notificaciones para no perderte
 los próximos capítulos. Ahora sí, comencemos."



TPprimerEntrega/
│
├── frontend/
│
│ ├── public/
│ │ ├── carrito1.png
│ │ ├── carrito2.png
│ │ └── ...
│ │
│ ├── src/
│ │
│ │ ├── components/
│ │ │ ├── Item/
│ │ │ ├── Carrito/
│ │ │ ├── CatalogoProductos/
│ │ │ ├── ChatIA/
│ │ │ ├── Login/
│ │ │ └── ProtectedRoute/
│ │ │
│ │ ├── context/
│ │ │ ├── CartContext.jsx
│ │ │ └── AuthContext.jsx
│ │ │
│ │ ├── firebase/
│ │ │ └── config.js
│ │ │
│ │ ├── services/
│ │ │ ├── productService.js
│ │ │ ├── authService.js
│ │ │ └── chatApi.js
│ │ │
│ │ ├── routes/
│ │ │ └── AppRouter.jsx
│ │ │
│ │ ├── pages/
│ │ │ ├── Home.jsx
│ │ │ ├── Login.jsx
│ │ │ ├── Admin.jsx
│ │ │ └── Checkout.jsx
│ │ │
│ │ ├── App.jsx
│ │ ├── main.jsx
│ │ └── ...
│ │
│ ├── package.json
│ ├── vite.config.js
│ ├── eslint.config.js
│ └── .gitignore
│
│
├── backend/
│
│ ├── routes/
│ │ └── chat.routes.js
│ │
│ ├── controllers/
│ │ └── chat.controller.js
│ │
│ ├── server.js
│ ├── package.json
│ ├── .env
│ ├── .gitignore
│ └── node_modules/
│
│
```

```
├── README.md
└── .gitignore
frontend/firebase/config.js

conecta Firebase

initializeApp()
getFirestore()
getAuth()
getStorage()
services/productService.js

maneja:

getProducts()
addProduct()
updateProduct()
deleteProduct()
uploadImage()
services/authService.js

maneja:

login()
register()
logout()
ProtectedRoute

protege rutas como:

/admin
/checkout
/profile
Imágenes de productos
mejor práctica

NO:

public/

para productos dinámicos.

SÍ:

Firebase Storage

porque:

admin sube imagen
Firebase genera URL
Firestore guarda URL
React la muestra


Productos → Firebase
IA → Render backend
Frontend → Vercel
Imágenes → Storage
Auth → Firebase

Eso está muy bien armado.

Mucho mejor que MockAPI.
```

index.html HTML · 18 lineas

```
<!doctype html>
<html lang="es">

<head>
  <meta charset="UTF-8" />
  <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>mi-proyecto-react</title>
</head>

<body>
  <!-- React monta toda la aplicación aquí -->
  <div id="root"></div>
  <!-- Punto de entrada principal -->
  <script type="module" src="/src/main.jsx"></script>
</body>

</html>
```

INSTRUCCIONES_TARJETAS.txt TXT · 121 lineas

```
============================================================
  INSTRUCCIONES PARA PROBAR EL PAGO CON MERCADO PAGO
============================================================

El sistema de pagos está integrado con Mercado Pago en modo SANDBOX (pruebas).
Esto significa que los pagos son simulados y no se mueve dinero real.

Para probar el flujo de pago se deben usar las tarjetas de prueba que
provee Mercado Pago. NO usar tarjetas reales en modo sandbox.

------------------------------------------------------------
**Importante:** Mercado Pago a veces tarda en responder luego
de clickear "Finalizar compra". Por favor, esperar
antes de considerar que el pago no funcionó.
-------------------------------------------------------------------------------------
IMPORTANTE:
Buyer test   USAR este correo :
 test_user_3168841862494326733@testuser.com
 ---------------------------------------------------------------------------
IMPORTANTE:
Las imágenes Mercado1.png, Mercado2.png y Mercado3.png,
ubicadas en la carpeta "imagenes" del proyecto,
muestran la secuencia correcta para completar el formulario
de Mercado Pago en modo Sandbox.

------------------------------------------------------------
TARJETAS DE PRUEBA DISPONIBLES
------------------------------------------------------------

Tarjeta              Número               CVV    Vencimiento
------------------------------------------------------------
Mastercard           5031 7557 3453 0604  123    11/30
Visa                 4509 9535 6623 3704  123    11/30
American Express     3711 803032 57522    1234   11/30
Mastercard Débito    5287 3383 1025 3304  123    11/30
Visa Débito          4002 7686 9439 5619  123    11/30


------------------------------------------------------------
ESTADOS DE PAGO SEGÚN EL NOMBRE DEL TITULAR
------------------------------------------------------------
IMPORTANTE:  Buyer test  usar su correo  test_user_3168841862494326733@testuser.com
En public ver Mercado1.png --Mercado2.png ----Mercado3.png
con la secuencia de datos completados correctamente

El resultado del pago se controla escribiendo uno de estos
valores en el campo "Nombre del titular" de la tarjeta:

Nombre titular  Estado          Descripción                          DNI requerido
-----------------------------------------------------------------------------------
APRO            Aprobado        Pago aprobado correctamente          12345678
OTHE            Rechazado       Rechazado por error general          12345678
CONT            Pendiente       Pendiente de acreditación            -
CALL            Rechazado       Requiere validación para autorizar   -
FUND            Rechazado       Importe insuficiente                 -
SECU            Rechazado       Código de seguridad inválido         -
EXPI            Rechazado       Problema con fecha de vencimiento    -
FORM            Rechazado       Error de formulario                  -

Ejemplo: para simular un pago aprobado usar cualquier tarjeta
de la lista con nombre del titular "APRO" y DNI "12345678".

IMPORTANTE:  Buyer test  usar su correo  test_user_3168841862494326733@testuser.com
En public ver Mercado1.png --Mercado2.png ----Mercado3.png
con la secuencia de datos completados correctamente
------------------------------------------------------------
FLUJO COMPLETO DE PRUEBA
------------------------------------------------------------

1. Agregar un producto al carrito desde el catálogo
2. Ir al carrito y hacer click en "Finalizar compra"
3. Si no está logueado, iniciar sesión con:
   Email:      castromanmarita@gmail.com  clave marita01  (cuenta cliente de prueba)
   o registrar una cuenta nueva
4. El sistema crea la orden en Firestore con estado "pendiente"
```

```
5. Se redirige al checkout de Mercado Pago
6. Ingresar los datos de una tarjeta de prueba
7. Según el nombre del titular ingresado:
   - APRO → redirige a /pago-exitoso → orden pasa a "pagada"
   - Rechazo → redirige a /pago-fallido → orden queda como "fallida"
   usar nombre de titular OTHE para simular un rechazo por ejemplo ver tabla
  o con nombre titular  - CONT  → redirige a /pago-pendiente → orden queda "pendiente"
  PORQUE SI HAY OTROS ERRORE SOLO MUESTRA LA PANTALA DE MERCADO PAGO fatal y no va a poder Ir
  al carrito mediante link en las pruebas es la forma en que mercado pago actua
  imposible hardcodearlo


------------------------------------------------------------
CUENTA ADMINISTRADOR
------------------------------------------------------------

Las credenciales de administrador fueron enviadas al docente por correo.

Desde el área de administración (/privada) se puede:
- Ver todas las órdenes generadas
- Gestionar productos (CRUD)
- Promover usuarios a administrador


------------------------------------------------------------
POR QUE APARECE LA PANTALLA "ALGO SALIO MAL" DE MERCADO PAGO
------------------------------------------------------------

Si al intentar pagar aparece la pantalla de error de Mercado Pago
con el mensaje "Una de las partes con la que intentas hacer el
pago es de prueba", es porque se está usando una tarjeta real
(o una cuenta real de Mercado Pago) en modo sandbox.

Esa pantalla pertenece al dominio de Mercado Pago
(mercadopago.com.ar) y NO puede ser modificada ni personalizada
por el vendedor. Mercado Pago la llama internamente "fatal error"
y no ejecuta ninguna redirección hacia el sitio del vendedor
cuando ocurre — por eso no aparece el botón de volver al carrito.

SOLUCION: usar únicamente las tarjetas de prueba de la tabla
de arriba. Con esas tarjetas el pago se procesa correctamente
y las redirecciones funcionan.

En producción con tarjetas reales este error no ocurre.
============================================================
```

package.json JSON · 1 lineas


```
{"name": "mercadopago-carrito-backEnd", "private": true, "version": "0.0.0", "type": "module", "scripts": {"dev": "vite",
"build": "vite build", "lint": "eslint .", "preview": "vite preview"}, "dependencies": {"firebase": "^12.13.0", "react":
"^19.2.4", "react-dom": "^19.2.4", "react-icons": "^5.6.0", "react-router-dom": "^7.15.0"}, "devDependencies":
{"@eslint/js": "^9.39.4", "@types/react": "^19.2.14", "@types/react-dom": "^19.2.3", "@vitejs/plugin-react": "^6.0.1",
"eslint": "^9.39.4", "eslint-plugin-react-hooks": "^7.0.1", "eslint-plugin-react-refresh": "^0.5.2", "globals": "^17.0.0",
"vite": "^8.0.4"}}
```

public/data/productos.json JSON · 374 lineas


```
[
  {
    "id": 1,
    "nombre": "Laptop",
    "stock": 10,
    "precio": 1200,
    "categoria": "computacion"
  },
  {
    "id": 2,
    "nombre": "Mouse",
    "stock": 5,
    "precio": 45,
    "categoria": "accesorios"
  },
  {
    "id": 3,
    "nombre": "Monitor",
    "stock": 2,
    "precio": 350,
    "categoria": "computacion"
  },
  {
    "id": 4,
    "nombre": "Teclado",
    "stock": 8,
    "precio": 89,
    "categoria": "accesorios"
  },
  {
    "id": 5,
    "nombre": "Webcam HD",
    "stock": 15,
    "precio": 75,
    "categoria": "accesorios"
  },
  {
    "id": 6,
    "nombre": "Auriculares Bluetooth",
    "stock": 12,
    "precio": 120,
    "categoria": "audio"
  },
  {
    "id": 7,
    "nombre": "Micrófono USB",
    "stock": 6,
    "precio": 95,
    "categoria": "audio"
  },
  {
    "id": 8,
    "nombre": "Tablet 10 pulgadas",
    "stock": 4,
    "precio": 450,
    "categoria": "moviles"
  },
  {
    "id": 9,
    "nombre": "Smartphone Android",
    "stock": 9,
    "precio": 680,
    "categoria": "moviles"
  },
  {
    "id": 10,
    "nombre": "Disco Duro Externo 1TB",
    "stock": 7,
    "precio": 85,
    "categoria": "almacenamiento"
  },
  {
    "id": 11,
    "nombre": "SSD 500GB",
    "stock": 11,
```

```
  "precio": 110,
  "categoria": "almacenamiento"
},
{
  "id": 12,
  "nombre": "Memoria RAM 16GB",
  "stock": 14,
  "precio": 130,
  "categoria": "componentes"
},
{
  "id": 13,
  "nombre": "Tarjeta Gráfica RTX 3060",
  "stock": 3,
  "precio": 450,
  "categoria": "componentes"
},
{
  "id": 14,
  "nombre": "Fuente de Poder 650W",
  "stock": 8,
  "precio": 95,
  "categoria": "componentes"
},
{
  "id": 15,
  "nombre": "Gabinete Gaming",
  "stock": 5,
  "precio": 120,
  "categoria": "componentes"
},
{
  "id": 16,
  "nombre": "Router WiFi 6",
  "stock": 9,
  "precio": 180,
  "categoria": "redes"
},
{
  "id": 17,
  "nombre": "Switch 8 Puertos",
  "stock": 6,
  "precio": 65,
  "categoria": "redes"
},
{
  "id": 18,
  "nombre": "Cable Ethernet 10m",
  "stock": 20,
  "precio": 15,
  "categoria": "accesorios"
},
{
  "id": 19,
  "nombre": "Hub USB 4 Puertos",
  "stock": 13,
  "precio": 35,
  "categoria": "accesorios"
},
{
  "id": 20,
  "nombre": "Batería Externa 20000mAh",
  "stock": 10,
  "precio": 55,
  "categoria": "accesorios"
},
{
  "id": 21,
  "nombre": "Cargador USB-C 65W",
  "stock": 8,
  "precio": 45,
  "categoria": "accesorios"
},
{
  "id": 22,
  "nombre": "Soporte Monitor Adjustable",
  "stock": 7,
```

```
  "precio": 85,
  "categoria": "accesorios"
},
{
  "id": 23,
  "nombre": "Silla Gaming Ergonómica",
  "stock": 4,
  "precio": 320,
  "categoria": "muebles"
},
{
  "id": 24,
  "nombre": "Escritorio Gaming 120cm",
  "stock": 3,
  "precio": 280,
  "categoria": "muebles"
},
{
  "id": 25,
  "nombre": "Lámpara LED RGB",
  "stock": 12,
  "precio": 45,
  "categoria": "accesorios"
},
{
  "id": 26,
  "nombre": "Mousepad XL Gaming",
  "stock": 15,
  "precio": 35,
  "categoria": "accesorios"
},
{
  "id": 27,
  "nombre": "Control Xbox Series X",
  "stock": 6,
  "precio": 65,
  "categoria": "gaming"
},
{
  "id": 28,
  "nombre": "Joystick PS5",
  "stock": 8,
  "precio": 75,
  "categoria": "gaming"
},
{
  "id": 29,
  "nombre": "Steam Deck 64GB",
  "stock": 2,
  "precio": 520,
  "categoria": "gaming"
},
{
  "id": 30,
  "nombre": "Nintendo Switch OLED",
  "stock": 5,
  "precio": 380,
  "categoria": "gaming"
},
{
  "id": 31,
  "nombre": "PlayStation 5",
  "stock": 1,
  "precio": 550,
  "categoria": "gaming"
},
{
  "id": 32,
  "nombre": "Xbox Series X",
  "stock": 3,
  "precio": 520,
  "categoria": "gaming"
},
{
  "id": 33,
  "nombre": "VR Meta Quest 2",
  "stock": 4,
```

```
  "precio": 420,
  "categoria": "gaming"
},
{
  "id": 34,
  "nombre": "Tarjeta de Captura 4K",
  "stock": 2,
  "precio": 280,
  "categoria": "streaming"
},
{
  "id": 35,
  "nombre": "Green Screen 2x2m",
  "stock": 6,
  "precio": 85,
  "categoria": "streaming"
},
{
  "id": 36,
  "nombre": "Ring Light LED",
  "stock": 9,
  "precio": 65,
  "categoria": "streaming"
},
{
  "id": 37,
  "nombre": "Tripie para Cámara",
  "stock": 5,
  "precio": 120,
  "categoria": "fotografia"
},
{
  "id": 38,
  "nombre": "Cámara DSLR 24MP",
  "stock": 2,
  "precio": 680,
  "categoria": "fotografia"
},
{
  "id": 39,
  "nombre": "Lente 50mm f/1.8",
  "stock": 4,
  "precio": 220,
  "categoria": "fotografia"
},
{
  "id": 40,
  "nombre": "Flash Externo",
  "stock": 7,
  "precio": 180,
  "categoria": "fotografia"
},
{
  "id": 41,
  "nombre": "Drone Mini 4K",
  "stock": 3,
  "precio": 450,
  "categoria": "drones"
},
{
  "id": 42,
  "nombre": "Baterías Drone (Pack 3)",
  "stock": 8,
  "precio": 95,
  "categoria": "drones"
},
{
  "id": 43,
  "nombre": "Cargador Drone",
  "stock": 6,
  "precio": 65,
  "categoria": "drones"
},
{
  "id": 44,
  "nombre": "Smartwatch Fitness",
  "stock": 10,
```

```
    "precio": 180,
    "categoria": "wearables"
  },
  {
    "id": 45,
    "nombre": "Bandas Fitness Tracker",
    "stock": 12,
    "precio": 65,
    "categoria": "wearables"
  },
  {
    "id": 46,
    "nombre": "Auriculares Deportivos",
    "stock": 15,
    "precio": 85,
    "categoria": "audio"
  },
  {
    "id": 47,
    "nombre": "Altavoces Bluetooth Portátiles",
    "stock": 9,
    "precio": 120,
    "categoria": "audio"
  },
  {
    "id": 48,
    "nombre": "Barra Sonido 2.1",
    "stock": 4,
    "precio": 220,
    "categoria": "audio"
  },
  {
    "id": 49,
    "nombre": "Proyector LED 1080p",
    "stock": 3,
    "precio": 380,
    "categoria": "video"
  },
  {
    "id": 50,
    "nombre": "Pantalla Táctil 15 pulgadas",
    "stock": 5,
    "precio": 280,
    "categoria": "accesorios"
  },
  {
    "id": 51,
    "nombre": "Cooling Pad Laptop",
    "stock": 8,
    "precio": 45,
    "categoria": "accesorios"
  },
  {
    "id": 52,
    "nombre": "Candado Portátil USB",
    "stock": 12,
    "precio": 35,
    "categoria": "seguridad"
  },
  {
    "id": 53,
    "nombre": "Cámara de Seguridad WiFi",
    "stock": 6,
    "precio": 120,
    "categoria": "seguridad"
  }
]
```

public/favicon.svg SVG · 1 lineas


```
<svg xmlns="http://www.w3.org/2000/svg" width="48" height="46" fill="none" viewBox="0 0 48 46"><path fill="#863bff"
d="M25.946 44.938c-.664.845-2.021.375-2.021-.698V33.937a2.26 2.26 0 0 0-2.262-2.262H10.287c-.92 0-1.456-1.04-.92-
1.788l7.48-10.471c1.07-1.497 0-3.578-1.842-3.578H1.237c-.92 0-1.456-1.04-.92-
1.788L10.013.474c.214-.297.556-.474.92-.474h28.894c.92 0 1.456 1.04.92 1.788l-7.48 10.471c-1.07 1.498 0 3.579 1.842
3.579h11.377c.943 0 1.473 1.088.89 1.83L25.947 44.94z" style="fill:#863bff;fill:color(display-p3 .5252 .23 1);fill-
opacity:1"/><mask id="a" width="48" height="46" x="0" y="0" maskUnits="userSpaceOnUse" style="mask-type:alpha"><path
fill="#000" d="M25.842 44.938c-.664.844-2.021.375-2.021-.698V33.937a2.26 2.26 0 0 0-2.262-2.262H10.183c-.92 0-1.456-
1.04-.92-1.788l7.48-10.471c1.07-1.498 0-3.579-1.842-3.579H1.133c-.92 0-1.456-1.04-.92-
1.787L9.91.473c.214-.297.556-.474.92-.474h28.894c.92 0 1.456 1.04.92 1.788l-7.48 10.471c-1.07 1.498 0 3.578 1.842
3.578h11.377c.943 0 1.473 1.088.89 1.832L25.843 44.94z" style="fill:#000;fill-opacity:1"/></mask><g mask="url(#a)"><g
filter="url(#b)"><ellipse cx="5.508" cy="14.704" fill="#ede6ff" rx="5.508" ry="14.704"
style="fill:#ede6ff;fill:color(display-p3 .9275 .9033 1);fill-opacity:1" transform="matrix(.00324 1 1 -.00324 -4.47
31.516)"/></g><g filter="url(#c)"><ellipse cx="10.399" cy="29.851" fill="#ede6ff" rx="10.399" ry="29.851"
style="fill:#ede6ff;fill:color(display-p3 .9275 .9033 1);fill-opacity:1" transform="matrix(.00324 1 1 -.00324 -39.328
7.883)"/></g><g filter="url(#d)"><ellipse cx="5.508" cy="30.487" fill="#7e14ff" rx="5.508" ry="30.487"
style="fill:#7e14ff;fill:color(display-p3 .4922 .0767 1);fill-opacity:1" transform="rotate(89.814 -25.913 -14.639)scale(1
-1)"/></g><g filter="url(#e)"><ellipse cx="5.508" cy="30.599" fill="#7e14ff" rx="5.508" ry="30.599"
style="fill:#7e14ff;fill:color(display-p3 .4922 .0767 1);fill-opacity:1" transform="rotate(89.814 -32.644 -3.334)scale(1
-1)"/></g><g filter="url(#f)"><ellipse cx="5.508" cy="30.599" fill="#7e14ff" rx="5.508" ry="30.599"
style="fill:#7e14ff;fill:color(display-p3 .4922 .0767 1);fill-opacity:1" transform="matrix(.00324 1 1 -.00324 -34.34
30.47)"/></g><g filter="url(#g)"><ellipse cx="14.072" cy="22.078" fill="#ede6ff" rx="14.072" ry="22.078"
style="fill:#ede6ff;fill:color(display-p3 .9275 .9033 1);fill-opacity:1" transform="rotate(93.35 24.506 48.493)scale(-1
1)"/></g><g filter="url(#h)"><ellipse cx="3.47" cy="21.501" fill="#7e14ff" rx="3.47" ry="21.501"
style="fill:#7e14ff;fill:color(display-p3 .4922 .0767 1);fill-opacity:1" transform="rotate(89.009 28.708 47.59)scale(-1
1)"/></g><g filter="url(#i)"><ellipse cx="3.47" cy="21.501" fill="#7e14ff" rx="3.47" ry="21.501"
style="fill:#7e14ff;fill:color(display-p3 .4922 .0767 1);fill-opacity:1" transform="rotate(89.009 28.708 47.59)scale(-1
1)"/></g><g filter="url(#j)"><ellipse cx=".387" cy="8.972" fill="#7e14ff" rx="4.407" ry="29.108"
style="fill:#7e14ff;fill:color(display-p3 .4922 .0767 1);fill-opacity:1" transform="rotate(39.51 .387 8.972)"/></g><g
filter="url(#k)"><ellipse cx="47.523" cy="-6.092" fill="#7e14ff" rx="4.407" ry="29.108"
style="fill:#7e14ff;fill:color(display-p3 .4922 .0767 1);fill-opacity:1" transform="rotate(37.892 47.523 -6.092)"/></g><g
filter="url(#l)"><ellipse cx="41.412" cy="6.333" fill="#47bfff" rx="5.971" ry="9.665"
style="fill:#47bfff;fill:color(display-p3 .2799 .748 1);fill-opacity:1" transform="rotate(37.892 41.412 6.333)"/></g><g
filter="url(#m)"><ellipse cx="-1.879" cy="38.332" fill="#7e14ff" rx="4.407" ry="29.108"
style="fill:#7e14ff;fill:color(display-p3 .4922 .0767 1);fill-opacity:1" transform="rotate(37.892 -1.88 38.332)"/></g><g
filter="url(#n)"><ellipse cx="-1.879" cy="38.332" fill="#7e14ff" rx="4.407" ry="29.108"
style="fill:#7e14ff;fill:color(display-p3 .4922 .0767 1);fill-opacity:1" transform="rotate(37.892 -1.88 38.332)"/></g><g
filter="url(#o)"><ellipse cx="35.651" cy="29.907" fill="#7e14ff" rx="4.407" ry="29.108"
style="fill:#7e14ff;fill:color(display-p3 .4922 .0767 1);fill-opacity:1" transform="rotate(37.892 35.651 29.907)"/></g><g
filter="url(#p)"><ellipse cx="38.418" cy="32.4" fill="#47bfff" rx="5.971" ry="15.297"
style="fill:#47bfff;fill:color(display-p3 .2799 .748 1);fill-opacity:1" transform="rotate(37.892 38.418 32.4)"/></g></g>
<defs><filter id="b" width="60.045" height="41.654" x="-19.77" y="16.149" color-interpolation-filters="sRGB"
filterUnits="userSpaceOnUse"><feFlood flood-opacity="0" result="BackgroundImageFix"/><feBlend in="SourceGraphic"
in2="BackgroundImageFix" result="shape"/><feGaussianBlur result="effect1_foregroundBlur_2002_17158" stdDeviation="7.659"/>
</filter><filter id="c" width="90.34" height="51.437" x="-54.613" y="-7.533" color-interpolation-filters="sRGB"
filterUnits="userSpaceOnUse"><feFlood flood-opacity="0" result="BackgroundImageFix"/><feBlend in="SourceGraphic"
in2="BackgroundImageFix" result="shape"/><feGaussianBlur result="effect1_foregroundBlur_2002_17158" stdDeviation="7.659"/>
</filter><filter id="d" width="79.355" height="29.4" x="-49.64" y="2.03" color-interpolation-filters="sRGB"
filterUnits="userSpaceOnUse"><feFlood flood-opacity="0" result="BackgroundImageFix"/><feBlend in="SourceGraphic"
in2="BackgroundImageFix" result="shape"/><feGaussianBlur result="effect1_foregroundBlur_2002_17158" stdDeviation="4.596"/>
</filter><filter id="e" width="79.579" height="29.4" x="-45.045" y="20.029" color-interpolation-filters="sRGB"
filterUnits="userSpaceOnUse"><feFlood flood-opacity="0" result="BackgroundImageFix"/><feBlend in="SourceGraphic"
in2="BackgroundImageFix" result="shape"/><feGaussianBlur result="effect1_foregroundBlur_2002_17158" stdDeviation="4.596"/>
</filter><filter id="f" width="79.579" height="29.4" x="-43.513" y="21.178" color-interpolation-filters="sRGB"
filterUnits="userSpaceOnUse"><feFlood flood-opacity="0" result="BackgroundImageFix"/><feBlend in="SourceGraphic"
in2="BackgroundImageFix" result="shape"/><feGaussianBlur result="effect1_foregroundBlur_2002_17158" stdDeviation="4.596"/>
</filter><filter id="g" width="74.749" height="58.852" x="15.756" y="-17.901" color-interpolation-filters="sRGB"
filterUnits="userSpaceOnUse"><feFlood flood-opacity="0" result="BackgroundImageFix"/><feBlend in="SourceGraphic"
in2="BackgroundImageFix" result="shape"/><feGaussianBlur result="effect1_foregroundBlur_2002_17158" stdDeviation="7.659"/>
</filter><filter id="h" width="61.377" height="25.362" x="23.548" y="2.284" color-interpolation-filters="sRGB"
filterUnits="userSpaceOnUse"><feFlood flood-opacity="0" result="BackgroundImageFix"/><feBlend in="SourceGraphic"
in2="BackgroundImageFix" result="shape"/><feGaussianBlur result="effect1_foregroundBlur_2002_17158" stdDeviation="4.596"/>
</filter><filter id="i" width="61.377" height="25.362" x="23.548" y="2.284" color-interpolation-filters="sRGB"
filterUnits="userSpaceOnUse"><feFlood flood-opacity="0" result="BackgroundImageFix"/><feBlend in="SourceGraphic"
in2="BackgroundImageFix" result="shape"/><feGaussianBlur result="effect1_foregroundBlur_2002_17158" stdDeviation="4.596"/>
</filter><filter id="j" width="56.045" height="63.649" x="-27.636" y="-22.853" color-interpolation-filters="sRGB"
filterUnits="userSpaceOnUse"><feFlood flood-opacity="0" result="BackgroundImageFix"/><feBlend in="SourceGraphic"
in2="BackgroundImageFix" result="shape"/><feGaussianBlur result="effect1_foregroundBlur_2002_17158" stdDeviation="4.596"/>
</filter><filter id="k" width="54.814" height="64.646" x="20.116" y="-38.415" color-interpolation-filters="sRGB"
filterUnits="userSpaceOnUse"><feFlood flood-opacity="0" result="BackgroundImageFix"/><feBlend in="SourceGraphic"
in2="BackgroundImageFix" result="shape"/><feGaussianBlur result="effect1_foregroundBlur_2002_17158" stdDeviation="4.596"/>
</filter><filter id="l" width="33.541" height="35.313" x="24.641" y="-11.323" color-interpolation-filters="sRGB"
filterUnits="userSpaceOnUse"><feFlood flood-opacity="0" result="BackgroundImageFix"/><feBlend in="SourceGraphic"
in2="BackgroundImageFix" result="shape"/><feGaussianBlur result="effect1_foregroundBlur_2002_17158" stdDeviation="4.596"/>
</filter><filter id="m" width="54.814" height="64.646" x="-29.286" y="6.009" color-interpolation-filters="sRGB"
filterUnits="userSpaceOnUse"><feFlood flood-opacity="0" result="BackgroundImageFix"/><feBlend in="SourceGraphic"
```

in2="BackgroundImageFix" result="shape"/><feGaussianBlur result="effect1_foregroundBlur_2002_17158" stdDeviation="4.596"/>
</filter><filter id="n" width="54.814" height="64.646" x="-29.286" y="6.009" color-interpolation-filters="sRGB"
filterUnits="userSpaceOnUse"><feFlood flood-opacity="0" result="BackgroundImageFix"/><feBlend in="SourceGraphic"
in2="BackgroundImageFix" result="shape"/><feGaussianBlur result="effect1_foregroundBlur_2002_17158" stdDeviation="4.596"/>
</filter><filter id="o" width="54.814" height="64.646" x="8.244" y="-2.416" color-interpolation-filters="sRGB"
filterUnits="userSpaceOnUse"><feFlood flood-opacity="0" result="BackgroundImageFix"/><feBlend in="SourceGraphic"
in2="BackgroundImageFix" result="shape"/><feGaussianBlur result="effect1_foregroundBlur_2002_17158" stdDeviation="4.596"/>
</filter><filter id="p" width="39.409" height="43.623" x="18.713" y="10.588" color-interpolation-filters="sRGB"
filterUnits="userSpaceOnUse"><feFlood flood-opacity="0" result="BackgroundImageFix"/><feBlend in="SourceGraphic"
in2="BackgroundImageFix" result="shape"/><feGaussianBlur result="effect1_foregroundBlur_2002_17158" stdDeviation="4.596"/>
</filter></defs></svg>

public/icons.svg SVG · 25 lineas

```
<svg xmlns="http://www.w3.org/2000/svg">
  <symbol id="bluesky-icon" viewBox="0 0 16 17">
    <g clip-path="url(#bluesky-clip)"><path fill="#08060d" d="M7.75 7.735c-.693-1.348-2.58-3.86-4.334-5.097-1.68-1.187-
2.32-.981-2.74-.79C.188 2.065.1 2.812.1 3.251s.241 3.602.398 4.13c.52 1.744 2.367 2.333 4.07 2.145-2.495.37-4.71 1.278-
1.805 4.512 3.196 3.309 4.38-.71 4.987-2.746.608 2.036 1.307 5.91 4.93 2.746 2.72-2.746.747-4.143-1.747-4.512 1.702.189
3.55-.4 4.07-2.145.156-.528.397-3.691.397-4.13s-.088-1.186-.575-1.406c-.42-.19-1.06-.395-2.741.79-1.755 1.24-3.64 3.752-
4.334 5.099"/></g>
    <defs><clipPath id="bluesky-clip"><path fill="#fff" d="M.1.85h15.3v15.3H.1z"/></clipPath></defs>
  </symbol>
  <symbol id="discord-icon" viewBox="0 0 20 19">
    <path fill="#08060d" d="M16.224 3.768a14.5 14.5 0 0 0-3.67-1.153c-.158.286-.343.67-.47.976a13.5 13.5 0 0 0-4.067
0c-.128-.306-.317-.69-.476-.976A14.4 14.4 0 0 0 3.868 3.77C1.546 7.28.916 10.703 1.231 14.077a14.7 14.7 0 0 0 4.5
2.306q.545-.748.965-1.587a9.5 9.5 0 0 1-1.518-.74q.191-.14.372-.293c2.927 1.369 6.107 1.369 8.999
0q.183.152.372.294-.723.437-1.52.74.418.838.963 1.588a14.6 14.6 0 0 0 4.504-2.308c.37-3.911-.63-7.302-2.644-10.309m-9.13
8.234c-.878 0-1.599-.82-1.599-1.82 0-.998.705-1.82 1.6-1.82.894 0 1.614.82 1.599 1.82.001 1-.705 1.82-1.6 1.82m5.91 0c-.878
0-1.599-.82-1.599-1.82 0-.998.705-1.82 1.6-1.82.893 0 1.614.82 1.599 1.82 0 1-.706 1.82-1.6 1.82"/>
  </symbol>
  <symbol id="documentation-icon" viewBox="0 0 21 20">
    <path fill="none" stroke="#aa3bff" stroke-linecap="round" stroke-linejoin="round" stroke-width="1.35" d="m15.5 13.333
1.533 1.322c.645.555.967.833.967 1.178s-.322.623-.967 1.179L15.5 18.333m-3.333-5-1.534 1.322c-.644.555-.966.833-.966
1.178s.322.623.966 1.179l1.534 1.321"/>
    <path fill="none" stroke="#aa3bff" stroke-linecap="round" stroke-linejoin="round" stroke-width="1.35" d="M17.167
10.836v-4.32c0-1.41 0-2.117-.224-2.68-.359-.906-1.118-1.621-2.08-1.96-.599-.21-1.349-.21-2.848-.21-2.623 0-3.935 0-
4.983.369-1.684.591-3.013 1.842-3.641 3.428C3 6.449 3 7.684 3 10.154v2.122c0 2.558 0 3.838.706
4.726q.306.383.713.671c.76.536 1.79.64 3.581.66"/>
    <path fill="none" stroke="#aa3bff" stroke-linecap="round" stroke-linejoin="round" stroke-width="1.35" d="M3 10a2.78
2.78 0 0 1 2.778-2.778c.555 0 1.209.097 1.748-.047.48-.129.854-.503.982-.982.145-.54.048-1.194.048-1.749a2.78 2.78 0 0 1
2.777-2.777"/>
  </symbol>
  <symbol id="github-icon" viewBox="0 0 19 19">
    <path fill="#08060d" fill-rule="evenodd" d="M9.356 1.85C5.05 1.85 1.57 5.356 1.57 9.694a7.84 7.84 0 0 0 5.324
7.44c.387.079.528-.168.528-.376 0-.182-.013-.805-.013-1.454-2.165.467-2.616-.935-2.616-.935-.349-.91-.864-1.143-.864-
1.143-.71-.48.051-.48.051-.48.787.051 1.2.805 1.2.805.695 1.194 1.817.857 2.268.649.064-.507.27-.857.49-1.052-1.728-.182-
3.545-.857-3.545-3.87 0-.857.31-1.558.8-2.104-.078-.195-.349-1 .077-2.078 0 0 .657-.208 2.14.805a7.5 7.5 0 0 1
1.946-.26c.657 0 1.328.092 1.946.26 1.483-1.013 2.14-.805 2.14-.805.426 1.078.155 1.883.078 2.078.502.546.799 1.247.799
2.104 0 3.013-1.818 3.675-3.558 3.87.284.247.528.714.528 1.454 0 1.052-.012 1.896-.012 2.156 0 .208.142.455.528.377a7.84
7.84 0 0 0 5.324-7.441c.013-4.338-3.48-7.844-7.773-7.844" clip-rule="evenodd"/>
  </symbol>
  <symbol id="social-icon" viewBox="0 0 20 20">
    <path fill="none" stroke="#aa3bff" stroke-linecap="round" stroke-linejoin="round" stroke-width="1.35" d="M12.5
6.667a4.167 4.167 0 1 0-8.334 0 4.167 4.167 0 0 0 8.334 0"/>
    <path fill="none" stroke="#aa3bff" stroke-linecap="round" stroke-linejoin="round" stroke-width="1.35" d="M2.5
16.667a5.833 5.833 0 0 1 8.75-5.053m3.837.474.513
1.035c.07.144.257.282.414.309l.93.155c.596.1.736.536.307.965l-.723.73a.64.64 0 0
0-.152.531l.207.903c.164.715-.213.991-.84.618l-.872-.52a.63.63 0 0 0-.577 0l-.872.52c-.624.373-
1.003.094-.84-.618l.207-.903a.64.64 0 0 0-.152-.532l-.723-.729c-.426-.43-.289-.864.306-.964l.93-.156a.64.64 0 0 0
.412-.31l.513-1.034c.28-.562.735-.562 1.012 0"/>
  </symbol>
  <symbol id="x-icon" viewBox="0 0 19 19">
    <path fill="#08060d" fill-rule="evenodd" d="M1.893 1.98c.052.072 1.245 1.769 2.653 3.77l2.892
4.114c.183.261.333.48.333.486s-.068.089-.152.183l-.522.593-.765.867-3.597 4.087c-.375.426-.734.834-.798.905a1 1 0 0
0-.118.148c0 .01.236.017.664.017h.663l.729-.83c.4-.457.796-.906.879-.999a692 692 0 0 0 1.794-
2.038c.034-.037.301-.34.594-.675l.551-.624.345-.392a7 7 0 0 1 .34-.374c.006 0 .93 1.306 2.052 2.903l2.084
2.965.045.063h2.275c1.87 0 2.273-.003 2.266-.021-.008-.02-1.098-1.572-3.894-5.547-2.013-2.862-2.28-3.246-2.273-
3.266.008-.019.282-.332 2.085-2.38l2-2.274 1.567-1.782c.022-.028-.016-.03-.65-.03h-.674l-.3.342a871 871 0 0 1-1.782
2.025c-.067.075-.405.458-.75.852a100 100 0 0 1-.803.91c-.148.172-.299.344-.99
1.127-.304.343-.32.358-.345.327-.015-.019-.904-1.282-1.976-2.808L6.365 1.85H1.8zm1.782.91 8.078 11.294c.772 1.08 1.413
1.973 1.425 1.984.016.017.241.02 1.05.017l1.03-.004-2.694-3.766L7.796 5.75 5.722 2.852l-1.039-.004-1.039-.004z" clip-
rule="evenodd"/>
  </symbol>
</svg>
```

README.md MD · 290 lineas

Mi Tienda React - TP Entrega Final

Esta entrega final se encuentra en un repositorio nuevo porque incorpora funcionalidades adicionales (autenticación, CRUD, carrito e integración con Mercado Pago), por lo que se presenta como un proyecto independiente de la primera entrega.

> ## ⚠️ IMPORTANTE — Instrucciones para probar el pago
>
> El sistema de pagos usa **Mercado Pago en modo sandbox** (pruebas).
> Para probar el checkout es obligatorio usar **tarjetas de prueba** — no tarjetas reales.
>
> 📄 **Ver archivo: `INSTRUCCIONES_TARJETAS.txt`** en la raíz del proyecto.
> Contiene las tarjetas de prueba, los estados simulados por nombre de titular y el flujo completo de prueba paso a paso.

Demo en vivo

Recurso	URL
Repositorio	https://github.com/nestdanchia/mercadopago-carrito-backEnd
Frontend (Vercel)	https://mercadopago-carrito-back-end.vercel.app/
Backend (Render)	https://mercadopago-carrito-backend.onrender.com

Credenciales de administrador: fueron enviadas al docente.

Valores de prueba requeridos por Mercado Pago

Importante: Las siguientes capturas muestran los valores de prueba proporcionados por Mercado Pago para el entorno Sandbox. Estos datos deben ingresarse exactamente como se indican. Si se utilizan valores distintos (por ejemplo, una tarjeta o un nombre de titular no válidos para pruebas), Mercado Pago puede mostrar una página de error (/fatal) y el flujo de pago no regresará a la aplicación.

<h3>1. Completar los datos de la tarjeta de prueba</h3>

<h3>2. Confirmar el pago</h3>

<h3>3. Resultado de la operación</h3>

Pruebas con Mercado Pago

Las instrucciones completas para realizar las pruebas del checkout se encuentran en:

📄 **[INSTRUCCIONES_TARJETAS.txt](./INSTRUCCIONES_TARJETAS.txt)**

Ver archivo: `INSTRUCCIONES_TARJETAS.txt` en la raíz del proyecto.

>**Importante:** Mercado Pago a veces tarda en responder luego de clickear "Finalizar compra". Por favor, esperar unos segundos antes de considerar que el pago no funcionó.

> El backend está en Render con plan gratuito. Si no recibe peticiones por 15 minutos se duerme. La primera solicitud al chat IA o al pago puede tardar hasta 60 segundos mientras se reactiva.

Funcionalidades

- Catálogo de productos — listado con imágenes, precios y stock
- Ver detalle — página individual de cada producto
- Carrito de compras — agregar productos, ver resumen, aplicar bono de descuento
- Bono de descuento del 30% — se activa automáticamente cuando el total supera $1.000.000
- Validación de stock al finalizar la compra — evita overselling

- Autenticación — registro, login y logout con Firebase Authentication
- Recupero de contraseña — por email vía Firebase
- CRUD de productos — alta, edición y eliminación (solo administradores)
- Panel de administración — gestión de productos, visualización y eliminación de órdenes
- Promover usuarios a administrador — desde el área privada
- Chat con IA — asistente integrado para consultas sobre laptops
- Integración con Mercado Pago — sandbox con preferencias, back_urls y páginas de resultado
- Órdenes en Firestore — se crean como `"pendiente"` y se actualizan según el resultado del pago

---

Tecnologías utilizadas

Frontend:
- React 19
- Vite
- React Router DOM v7
- React Icons
- Context API (carrito y autenticación)

Backend:
- Node.js
- Express v5
- SDK oficial de Mercado Pago v3
- dotenv / cors

Base de datos y autenticación:
- Firebase Authentication
- Firebase Firestore

IA:
- Groq API (proveedor principal — modelo llama-3.1-8b-instant)
- OpenRouter API (fallback — modelo meta-llama/llama-3.1-8b-instruct)

Pagos:
- Mercado Pago Checkout Pro (SDK Node.js)

Despliegue:
- Vercel (frontend React)
- Render (backend Node + Express)

Servicios externos

Servicio	Función
Firebase Auth	Autenticación de usuarios
Firestore	Base de datos de productos, órdenes y usuarios
Mercado Pago	Procesamiento de pagos
Groq	IA principal del chat
OpenRouter	IA fallback del chat
Render	Hosting del backend
Vercel	Hosting del frontend

Estructura del proyecto

```
mercadopago-carrito-backEnd/
│
├── backEnd/                          ← Backend separado (antes era server.js monolítico)
│   ├── src/
│   │   ├── app.js                    ← Entry point: Express + middleware + rutas
│   │   ├── config/
│   │   │   └── environment.js        ← Variables de entorno, tokens y keys
│   │   ├── controllers/
│   │   │   ├── chat.js               ← Manejo de req/res del endpoint /chat
│   │   │   └── pagos.js              ← Manejo de req/res del endpoint /pagos
│   │   ├── routes/
│   │   │   ├── chat.js               ← Definición de la ruta /chat
│   │   │   └── pagos.js              ← Definición de la ruta /pagos
│   │   └── services/
│   │       ├── ia.service.js         ← Lógica de IA (Groq + fallback OpenRouter)
│   │       └── mercadopago.service.js ← Lógica de creación de preferencias MP
│   ├── .env                          ← Variables de entorno del backend (no se sube a Git)
```

```
│   └── package.json
│
├── public/
│   ├── data/
│   │   └── productos.json          ← fallback local si Firestore falla
│   └── *.jpeg / *.png              ← imágenes de productos
│
├── src/
│   ├── components/
│   │   ├── Administrar/            ← CRUD productos, tabla órdenes, promover admin
│   │   ├── Auth/                   ← Login y Registro
│   │   ├── Carrito/                ← Carrito y CarritoItem
│   │   ├── CatalogoProductos/      ← Grilla de productos
│   │   ├── ChatIA/                 ← Asistente IA
│   │   ├── Item/                   ← Tarjeta individual de producto
│   │   ├── Layout/                 ← Header, Navbar, Footer
│   │   ├── Pago/                   ← PagoExitoso, PagoFallido, PagoPendiente
│   │   └── ProductoDetalle/        ← Vista detalle de producto
│   │
│   ├── context/
│   │   ├── auth/                   ← AuthProvider, AuthContext
│   │   └── cart/                   ← CartProvider, CartContext
│   │
│   ├── firebase/
│   │   └── config.js               ← Configuración Firebase
│   │
│   ├── services/
│   │   ├── cargaProductos.jsx      ← CRUD Firestore + fallback JSON
│   │   ├── ordenesService.js       ← Crear, validar stock y actualizar órdenes
│   │   ├── migrarDatosIniciales.js ← Seed inicial (comentado, histórico)
│   │   └── pruebaLectura.js        ← Diagnóstico de conexión al arrancar
│   │
│   ├── comentarios.txt             ← Documentación técnica del proyecto
│   ├── App.jsx
│   └── main.jsx
│
├── vercel.json                     ← Configuración de rutas para Vercel (SPA)
├── package.json                    ← Dependencias del frontend
├── .env                            ← Variables de entorno del frontend (no se sube a Git)
├── INSTRUCCIONES_TARJETAS.txt      ← Guía para probar el pago con MP
└── README.md
```

Flujo de compra

1. El usuario navega el catálogo y agrega productos al carrito
2. Si el total supera $1.000.000 se genera un código de bono del 30%
3. Al finalizar la compra, si no está logueado se lo redirige al login
4. Se valida el stock disponible en Firestore antes de crear la orden
5. Se crea la orden en Firestore con estado `"pendiente"`
6. El backend genera una preferencia en Mercado Pago y devuelve el `init_point`
7. El usuario paga en el checkout de Mercado Pago con tarjetas de prueba
8. Al volver, `/pago-exitoso`, `/pago-fallido` o `/pago-pendiente` actualiza el estado de la orden

Sistema de fallback de productos

Si Firestore no responde, la aplicación carga automáticamente `public/data/productos.json` sin mostrar ningún error al usuario. En ningún caso la app queda sin productos.

Chat con IA

El asistente está disponible en `/chat`. Responde consultas sobre laptops. El backend intenta primero con Groq y si falla usa OpenRouter como respaldo automático.

Correr el proyecto localmente

El profesor no necesita hacer esto. La app ya está desplegada y funciona desde el navegador.

El proyecto se divide en dos partes independientes: **backend** (carpeta `backEnd/`) y **frontend** (raíz del proyecto).

Cada una tiene su propio `package.json` y su propio `.env`.

1. Instalar dependencias (una sola vez)

```bash
# Backend
cd backEnd
npm install

# Frontend (desde la raíz del proyecto)
npm install
```

2. Levantar el backend (puerto 3001)

Terminal 1:

```bash
cd backEnd
npm start
```

3. Levantar el frontend (puerto 5173)

Terminal 2 (desde la raíz del proyecto):

```bash
npm run dev
```

Primero se levanta el backend (puerto 3001) y después el frontend (puerto 5173). Ese orden es el correcto porque el frontend le pega al backend a través de `VITE_API_URL`.

Archivos `.env`

`backEnd/.env` (variables del backend):

```
GROQ_API_KEY=...
OPENROUTER_API_KEY=...
CLIENT_URL=http://localhost:5173
MERCADOPAGO_ACCESS_TOKEN=...
```

`.env` (raíz del proyecto — variables del frontend):

```
VITE_API_URL=http://localhost:3001
VITE_FIREBASE_API_KEY=...
VITE_FIREBASE_AUTH_DOMAIN=...
VITE_FIREBASE_PROJECT_ID=...
VITE_FIREBASE_STORAGE_BUCKET=...
VITE_FIREBASE_MESSAGING_SENDER_ID=...
VITE_FIREBASE_APP_ID=...
```

Integración con Mercado Pago

El token de acceso vive únicamente en el servidor (`.env`), nunca en el frontend.
Se usa el SDK oficial de Mercado Pago para Node.js (`mercadopago` v3).
El checkout está en modo sandbox con cuentas de prueba.
En producción se deben actualizar las `back_urls` con las URLs de Vercel y activar `auto_return: "approved"` en `backEnd/src/services/mercadopago.service.js`.

Aclaración sobre el repositorio

Esta es la entrega final del TP. La primera entrega está en un repositorio separado. Este proyecto la extiende con autenticación, roles, CRUD completo, validación de stock, órdenes y pago con Mercado Pago.

src/App.css CSS · 34 lineas

```css
.frame {
  width: 960px;
  max-width: 100%;
  min-height: 220px;
  margin: 24px auto;
  padding: 24px;
  border: 0;
  border-radius: 12px;
  background-color: transparent;

  display: grid;
  grid-template-columns: repeat(4, minmax(180px, 1fr));
  align-items: stretch;
  gap: 16px;
}

.item {
  min-width: 0;
}

/* Responsive — en tablet 2 columnas, en mobile 1 columna */
@media (max-width: 768px) {
  .frame {
    grid-template-columns: repeat(2, 1fr);
    padding: 16px;
  }
}

@media (max-width: 480px) {
  .frame {
    grid-template-columns: 1fr;
    padding: 12px;
  }
}
```

src/App.jsx JSX · 75 lineas

```jsx
import { Routes, Route } from "react-router-dom";
import Layout from "./components/Layout/Layout";
import { CatalogoProductos } from "./components/CatalogoProductos/CatalogoProductos";
import { ChatIA } from "./components/ChatIA/ChatIA";
import ProductoDetalle from "./components/ProductoDetalle/ProductoDetalle";
import { CartProvider } from "./context/cart/CartProvider";
import { AuthProvider } from "./context/auth/AuthProvider";
import { Carrito } from "./components/Carrito/Carrito";
import { RutaPrivadaElemental } from "./components/RutaPrivadaElementales";
import CargaInicial from "./services/migrarDatosIniciales";
import PruebaLectura from "./services/pruebaLectura";
import Login from "./components/Auth/Login";
import Registro from "./components/Auth/Registro";
import Admin from "./components/Administrar/admin";
import PromoverAdmin from "./components/Administrar/PromoverAdmin";
import TablaOrdenes from "./components/Administrar/TablaOrdenes";
import PagoExitoso from "./components/Pago/PagoExitoso";
import PagoFallido from "./components/Pago/PagoFallido";
import PagoPendiente from "./components/Pago/PagoPendiente";
// CartProvider comparte el estado global del carrito con todos los
// componentes hijos que consuman CartContext usando useContext().
// spa La página HTML principal es una sola, y React va cambiando qué componente
//  mostrar mediante re-renders.
function App() {
  //CargaInicial(); se incorpora como componente hijo de CartProvider no afecta
  // el renderizado de la app pues retorna null  fue usado para cargar los datos iniciales a Firestore

  return (
    <AuthProvider>
      <CartProvider>
        <CargaInicial />
        <PruebaLectura />
        <Routes>
          <Route element={<Layout />}>
            <Route path="/carrito" element={<Carrito />} />
            <Route path="/admin" element={<Admin />} />
            <Route
              path="/"
              element={
                <CatalogoProductos mensaje="Tienda Virtual - Productos Disponibles" />
              }
            />
            <Route path="/chat" element={<ChatIA />} />
            <Route
              path="/registro"
              element={<Registro />}
            />
            <Route path="/login" element={<Login />} />
            {/* Ruta protegida de ejemplo */}
            <Route
              path="/privada"
              element={
                <RutaPrivadaElemental>
                  <div>
                    <h2>Ordenes</h2>
                    <p>Ves este mensaje porque te autenticaste como administrador.</p>
                    <PromoverAdmin />
                    <TablaOrdenes />
                  </div>
                </RutaPrivadaElemental>
              }
            />
            <Route path="/producto/:id" element={<ProductoDetalle />} />
            <Route path="/pago-exitoso" element={<PagoExitoso />} />
            <Route path="/pago-fallido" element={<PagoFallido />} />
            <Route path="/pago-pendiente" element={<PagoPendiente />} />
          </Route>
        </Routes>
      </CartProvider>
    </AuthProvider>
  );
}

export default App;
```

src/assets/react.svg SVG · 1 lineas


```
<svg xmlns="http://www.w3.org/2000/svg" xmlns:xlink="http://www.w3.org/1999/xlink" aria-hidden="true" role="img"
class="iconify iconify--logos" width="35.93" height="32" preserveAspectRatio="xMidYMid meet" viewBox="0 0 256 228"><path
fill="#00D8FF" d="M210.483 73.824a171.49 171.49 0 0 0-8.24-2.597c.465-1.9.893-3.777 1.273-5.621c6.238-30.281 2.16-54.676-
11.769-62.708c-13.355-7.7-35.196.329-57.254 19.526a171.23 171.23 0 0 0-6.375 5.848a155.866 155.866 0 0 0-4.241-
3.917C100.759 3.829 77.587-4.822 63.673 3.233C50.33 10.957 46.379 33.89 51.995 62.588a170.974 170.974 0 0 0 1.892 8.48c-
3.28.932-6.445 1.924-9.474 2.98C17.309 83.498 0 98.307 0 113.668c0 15.865 18.582 31.778 46.812 41.427a145.52 145.52 0 0 0
6.921 2.165a167.467 167.467 0 0 0-2.01 9.138c-5.354 28.2-1.173 50.591 12.134 58.266c13.744 7.926 36.812-.22 59.273-
19.855a145.567 145.567 0 0 0 5.342-4.923a168.064 168.064 0 0 0 6.92 6.314c21.758 18.722 43.246 26.282 56.54 18.586c13.731-
7.949 18.194-32.003 12.4-61.268a145.016 145.016 0 0 0-1.535-6.842c1.62-.48 3.21-.974 4.76-1.488c29.348-9.723 48.443-25.443
48.443-41.52c0-15.417-17.868-30.326-45.517-39.844Zm-6.365 70.984c-1.4.463-2.836.91-4.3 1.345c-3.24-10.257-7.612-21.163-
12.963-32.432c5.106-11 9.31-21.767 12.459-31.957c2.619.758 5.16 1.557 7.61 2.4c23.69 8.156 38.14 20.213 38.14 29.504c0
9.896-15.606 22.743-40.946 31.14Zm-10.514 20.834c2.562 12.94 2.927 24.64 1.23 33.787c-1.524 8.219-4.59 13.698-8.382
15.893c-8.067 4.67-25.32-1.4-43.927-17.412a156.726 156.726 0 0 1-6.437-5.87c7.214-7.889 14.423-17.06 21.459-27.246c12.376-
1.098 24.068-2.894 34.671-5.345a134.17 134.17 0 0 1 1.386 6.193ZM87.276 214.515c-7.882 2.783-14.16 2.863-17.955.675c-8.075-
4.657-11.432-22.636-6.853-46.752a156.923 156.923 0 0 1 1.869-8.499c10.486 2.32 22.093 3.988 34.498 4.994c7.084 9.967 14.501
19.128 21.976 27.15a134.668 134.668 0 0 1-4.877 4.492c-9.933 8.682-19.886 14.842-28.658 17.94ZM50.35 144.747c-12.483-4.267-
22.792-9.812-29.858-15.863c-6.35-5.437-9.555-10.836-9.555-15.216c0-9.322 13.897-21.212 37.076-29.293c2.813-.98 5.757-1.905
8.812-2.773c3.204 10.42 7.406 21.315 12.477 32.332c-5.137 11.18-9.399 22.249-12.634 32.792a134.718 134.718 0 0 1-6.318-
1.979Zm12.378-84.26c-4.811-24.587-1.616-43.134 6.425-47.789c8.564-4.958 27.502 2.111 47.463 19.835a144.318 144.318 0 0 1
3.841 3.545c-7.438 7.987-14.787 17.08-21.808 26.988c-12.04 1.116-23.565 2.908-34.161 5.309a160.342 160.342 0 0 1-1.76-
7.887Zm110.427 27.268a347.8 347.8 0 0 0-7.785-12.803c8.168 1.033 15.994 2.404 23.343 4.08c-2.206 7.072-4.956 14.465-8.193
22.045a381.151 381.151 0 0 0-7.365-13.322Zm-45.032-43.861c5.044 5.465 10.096 11.566 15.065 18.186a322.04 322.04 0 0 0-
30.257-.006c4.974-6.559 10.069-12.652 15.192-18.18ZM82.802 87.83a323.167 323.167 0 0 0-7.227 13.238c-3.184-7.553-5.909-
14.98-8.134-22.152c7.304-1.634 15.093-2.97 23.209-3.984a321.524 321.524 0 0 0-7.848 12.897Zm8.081 65.352c-8.385-.936-
16.291-2.203-23.593-3.793c2.26-7.3 5.045-14.885 8.298-22.6a321.187 321.187 0 0 0 7.257 13.246c2.594 4.48 5.28 8.868 8.038
13.147Zm37.542 31.03c-5.184-5.592-10.354-11.779-15.403-18.433c4.902.192 9.899.29 14.978.29c5.218 0 10.376-.117
15.453-.343c-4.985 6.774-10.018 12.97-15.028 18.486Zm52.198-57.817c3.422 7.8 6.306 15.345 8.596 22.52c-7.422 1.694-15.436
3.058-23.88 4.071a382.417 382.417 0 0 0 7.859-13.026a347.403 347.403 0 0 0 7.425-13.565Zm-16.898 8.101a358.557 358.557 0 0
1-12.281 19.815a329.4 329.4 0 0 1-23.444.823c-7.967 0-15.716-.248-23.178-.732a310.202 310.202 0 0 1-12.513-
19.846h.001a307.41 307.41 0 0 1-10.923-20.627a310.278 310.278 0 0 1 10.89-20.637l-.001.001a307.318 307.318 0 0 1 12.413-
19.761c7.613-.576 15.42-.876 23.31-.876H128c7.926 0 15.743.303 23.354.883a329.357 329.357 0 0 1 12.335 19.695a358.489
358.489 0 0 1 11.036 20.54a329.472 329.472 0 0 1-11 20.722Zm22.56-122.124c8.572 4.944 11.906 24.881 6.52 51.026c-.344
1.668-.73 3.367-1.15 5.09c-10.622-2.452-22.155-4.275-34.23-5.408c-7.034-10.017-14.323-19.124-21.64-27.008a160.789 160.789 0
0 1 5.888-5.4c18.9-16.447 36.564-22.941 44.612-18.3ZM128 90.808c12.625 0 22.86 10.235 22.86 22.86s-10.235 22.86-22.86
22.86s-22.86-10.235-22.86-22.86s10.235-22.86 22.86-22.86Z"></path></svg>
```

src/comentarios.txt TXT · 380 lineas

```
============================================================
  DOCUMENTACION TECNICA - TIENDA REACT + FIREBASE + MERCADOPAGO
============================================================



------------------------------------------------------------
1. CICLO DE VIDA DEL USUARIO (flujo completo)
------------------------------------------------------------

PASO 1 - La app arranca
  App.jsx monta AuthProvider y CartProvider.
  AuthProvider instala un observador en Firebase (onAuthStateChanged).
  con !loading && children opta por mostar todo el aargol de la aplicacion dentro de las etiquetas
  de  AuthProvider que engloban al arbol
  Mientras Firebase no responde, loading = true y la app no muestra nada.
  Hasta tanto el usuario este autenticado .o sea se lo identifica
  Cuando Firebase confirma si hay sesion o no, loading pasa a false y la app se renderiza.

PASO 2 - El usuario navega por el catalogo
  CatalogoProductos se monta y ejecuta su useEffect de carga.
  Llama a obtenerProductos() del service cargaProductos.jsx.
  Si Firestore responde: muestra los productos.
  Si Firestore falla: carga el fallback desde /data/productos.json (public/).

  CONTEXTO Y CARTPROVIDER:
  CartProvider vive en App.jsx por encima de todas las rutas.
  Crea el CartContext con createContext() y lo expone via CartContext.Provider.
  Esto significa que el estado del carrito (carrito[], vaciarCarrito, agregarAlCarrito,
  totalUnidades) existe en un unico lugar en memoria y es accesible desde cualquier
  componente hijo sin pasar props manualmente.
  CatalogoProductos consume CartContext via useContext(CartContext) para llamar
  a agregarAlCarrito() cuando el usuario compra un producto.
  Carrito consume el mismo CartContext para leer carrito[] y mostrar los items.
  Ambos componentes estan en rutas distintas y nunca se comunican directamente.
  CartProvider es el intermediario: mantiene el estado vivo mientras el usuario
  navega entre paginas. Cuando CatalogoProductos escribe (agrega al carrito),
  Carrito lee el resultado aunque sean componentes completamente separados.
  Este patron se llama Context API y evita el "prop drilling" (pasar datos por
  multiples niveles de componentes que no los necesitan).

  El usuario puede agregar productos al carrito sin estar logueado.
  El carrito vive en CartContext (memoria, no persistido en base de datos).

PASO 3 - El usuario va al carrito
  Lee el estado del CartContext: items, cantidades, totales.
  Si el totalBruto supera $1.000.000 se genera un codigo de bono del 30%.
  El usuario puede ingresar el codigo para aplicar el descuento.
  El descuento se calcula sobre el totalBruto, no por articulo.

PASO 4 - El usuario presiona "Finalizar compra"
  Si no esta logueado: alert + redireccion a /login.
  Si esta logueado: continua el flujo de compra.

PASO 5 - Se procesa la compra
  Se arma el objeto nuevaOrden con los datos del usuario, items y totales.
  Se llama a crearOrden() del service ordenesService.js -> guarda en Firestore con estado "pendiente".
  Se llama al backend Express (localhost:3001/pagos) para crear la preferencia en MercadoPago.
  Si el backend falla: se muestra el error y la orden queda en "pendiente" sin redirigir.
  Si el backend responde con data.init_point: se vacia el carrito y se redirige al checkout de MercadoPago.

PASO 6 - El usuario paga en MercadoPago
  MercadoPago muestra el checkout con el monto total como un unico item ("Compra tienda online").
  Al terminar redirige segun el resultado (manual en localhost, automatico en produccion con auto_return):
    - Pago aprobado  -> /pago-exitoso?orderId=xxx  -> estado: "pagada"
    - Pago fallido   -> /pago-fallido?orderId=xxx  -> estado: "fallida"
    - Pago pendiente -> /pago-pendiente?orderId=xxx -> estado: "pendiente" (con payment_id)

PASO 7 - Actualizacion de la orden
  Cada pagina (PagoExitoso, PagoFallido, PagoPendiente) lee el orderId de la URL.
  Llama a actualizarEstadoOrden() del service ordenesService.js.
  Actualiza en Firestore solo los campos: estado, payment_id, payment_status.
  El resto de los campos de la orden (items, totales, usuario) quedan intactos.
```

---
2. FUNCIONES CALLBACK - DONDE Y POR QUE SE USAN

A) crearCallbackCompra(productoId) - CatalogoProductos.jsx
   Es una funcion que retorna otra funcion (patron closure/fabrica).
   Se llama una vez por cada producto al renderizar la grilla.
   Captura el productoId en su closure.
   Devuelve una funcion (cantidad) => { ... } que se pasa a Item como prop onCompra.
   Cuando el usuario hace clic en "Comprar" en Item, Item llama onCompra(cantidadSeleccionada).
   Esa llamada llega al callback con el id ya capturado, busca el producto,
   llama a agregarAlCarrito() y actualiza el stock en Firestore.
   Por que: Item no sabe que producto es ni donde va el dato.
   Solo informa cuanto quiere comprar. El callback une ambos mundos.

B) agregarAlCarrito(producto, cantidad) - CartProvider.jsx
   Recibe la funcion de actualizacion del estado: setCarrito(prev => ...).
   Usa la forma funcional de setCarrito para leer el estado mas actualizado.
   Si el producto ya existe en el carrito, suma la cantidad.
   Si es nuevo, lo agrega con spread operator: [...prev, { ...producto, cantidad }].
   Por que funcional: evita leer un estado desactualizado en actualizaciones rapidas.

C) onAuthStateChanged(auth, callback) - AuthProvider.jsx
   Firebase llama a este callback cada vez que el estado de autenticacion cambia,
   pero tambien lo llama al arrancar la app para informar el estado inicial.
   Esto significa que se ejecuta siempre: tanto si hay sesion activa como si no la hay.
   El callback recibe currentUser (objeto usuario) o null (sin sesion).
   Si hay usuario: busca su rol en Firestore y actualiza el estado global (user, rol).
   Si no hay usuario: limpia user y rol a null. En ambos casos la app se renderiza,
   porque en ambos casos setLoading(false) se ejecuta al final.
   Por que callback: Firebase maneja la autenticacion de forma asincrona.
   No sabemos cuando va a responder (puede ser rapido o lento segun la red).
   El callback es el mecanismo que Firebase usa para avisarnos cuando tiene la respuesta,
   sin que tengamos que preguntar repetidamente.
   La funcion unsubscribe que retorna useEffect elimina el observador al desmontar,
   evitando actualizaciones de estado en un componente que ya no existe.

D) onChange={(e) => setInputBono(e.target.value.toUpperCase())} - Carrito.jsx
   Callback de evento de React. Se ejecuta cada vez que el usuario tipea en el input.
   Recibe el evento e, extrae el valor del input y lo convierte a mayusculas.
   Actualiza el estado inputBono en tiempo real.
   Por que: React necesita controlar el valor del input (controlled component).

E) onClick={aplicarBono} - Carrito.jsx
   Callback de evento asignado al boton "Aplicar".
   Compara inputBono con codigoBono generado para esta sesion.
   Si coinciden: setBonoAplicado(true).
   Si no: setErrorBono con mensaje de error.

F) getClass({ isActive }) - Navbar.jsx
   Callback que NavLink ejecuta automaticamente.
   NavLink le pasa { isActive: true/false } segun si la ruta actual coincide con el "to".
   Devuelve una clase CSS o dos segun el valor de isActive.
   Por que: permite aplicar estilos visuales al link de la ruta activa.

---
3. IMPLEMENTACION DE MERCADO PAGO

ARQUITECTURA
  Frontend (React) <-> Backend Express (backEnd/) <-> API MercadoPago
  El token de acceso NUNCA sale al frontend. Solo vive en el backend via backEnd/.env.
  El backend esta separado en la carpeta backEnd/ con estructura en capas:
    app.js (entry point) -> routes -> controllers -> services -> config/environment.js
  Ya no es un archivo monolitico (server.js fue eliminado y dividido en capas).

FLUJO TECNICO

  3.1 - CREACION DE LA PREFERENCIA (backEnd/ - endpoint POST /pagos)
    El endpoint vive en backEnd/src/controllers/pagos.js y la logica en backEnd/src/services/mercadopago.service.js
    Recibe del frontend: items, totalFinal, orderId.
    Construye un unico item resumen:
      { title: "Compra tienda online", quantity: 1, unit_price: totalFinal, currency_id: "ARS" }
    Esto simplifica el manejo de descuentos: React calcula todo y envia solo el total final.
    Llama a preference.create() del SDK de MercadoPago con:
      - items: el item resumen

```
      - back_urls: URLs a donde redirigir segun el resultado del pago
        success -> /pago-exitoso?orderId=xxx
        failure -> /pago-fallido?orderId=xxx
        pending -> /pago-pendiente?orderId=xxx
    Devuelve al frontend: { id, init_point }

  3.2 - REDIRECCION AL CHECKOUT (Carrito.jsx)
    window.location.href = data.init_point
    init_point es la URL del checkout que MercadoPago devuelve por convencion.
    Es la forma recomendada por MercadoPago (no construir la URL manualmente).

  3.3 - RETORNO DEL PAGO (PagoExitoso / PagoFallido / PagoPendiente)
    MercadoPago incluye en la URL de retorno: orderId, payment_id, status.
    Cada componente lee esos parametros con useSearchParams().
    Llama a actualizarEstadoOrden() del service para actualizar Firestore.

  3.4 - QUE ES UNA PREFERENCIA
    El termino viene de la API de MercadoPago. Es la configuracion que le indicamos
    a MercadoPago ANTES de que ocurra el pago. Le estamos diciendo nuestras "preferencias"
    para esa transaccion: que se vende, cuanto cuesta, en que moneda, a donde redirigir.
    MercadoPago la almacena en sus servidores y le asigna un ID unico.
    Cuando el usuario llega al checkout con ese ID, MercadoPago sabe exactamente
    que mostrar y cuanto cobrar. La preferencia y el pago son dos objetos distintos:
    la preferencia es la configuracion previa, el pago es el evento que ocurre
    cuando el usuario confirma con su tarjeta.

  3.5 - AUTO_RETURN (actualmente comentado)
    auto_return: "approved" hace que MercadoPago redirija automaticamente
    a back_urls.success cuando el pago es aprobado, sin que el usuario tenga
    que hacer nada. Sin esta linea, MercadoPago muestra una pantalla de confirmacion
    pero el usuario debe volver manualmente al sitio.
    La razon por la que esta comentada durante el desarrollo es tecnica:
    MercadoPago valida que back_urls.success sea una URL publica y accesible
    desde sus servidores. localhost:5173 no es publica, solo existe en la maquina
    del desarrollador. Cuando MercadoPago intenta verificarla, falla y devuelve
    el error "auto_return invalid. back_url.success must be defined".
    En produccion (cuando el frontend este deployado en Vercel o similar),
    las URLs seran publicas del tipo https://mi-tienda.vercel.app/pago-exitoso
    y en ese momento se puede descomentar auto_return: "approved" para que
    la redireccion sea automatica al completar el pago.

  3.6 - MODO PRUEBA / PRODUCCION
    Token TEST-... -> usa cuentas de prueba (Seller + Buyer Test User)
    Token APP_USR-... -> produccion real
    sandbox_init_point -> checkout de pruebas
    init_point -> checkout productivo


------------------------------------------------------------
4. SEPARACION DE RESPONSABILIDADES (services)
------------------------------------------------------------

  cargaProductos.jsx   -> CRUD de productos con Firestore + fallback JSON
  ordenesService.js    -> crear y actualizar ordenes en Firestore
  migrarDatosIniciales.js -> seed inicial de la base de datos (comentado, historico)
  pruebaLectura.js     -> diagnostico al arrancar: lee y loguea productos en consola


------------------------------------------------------------
5. ESTADO GLOBAL vs ESTADO LOCAL
------------------------------------------------------------

  CartContext (CartProvider)  -> carrito, totalUnidades - compartido entre rutas
  AuthContext (AuthProvider)  -> user, rol, loading    - compartido en toda la app
  useState en Carrito.jsx     -> codigoBono, bonoAplicado, inputBono - local al componente
  useState en CatalogoProductos -> productos, loading, error - local al componente

  ESTADO DERIVADO:
  totalUnidades = carrito.reduce(...) -> se recalcula en cada render, no es un estado separado.
  totalBruto, descuento, totalFinal -> calculados en el render de Carrito, no almacenados.
  Esto evita la doble fuente de verdad (two sources of truth).

------------------------------------------------------------
6. INCONVENIENTES CON MERCADO PAGO Y COMO SE RESOLVIERON
------------------------------------------------------------

PROBLEMA 1 - Internal Server Error al crear la preferencia
```

```
  Error: "Error al generar el pago"
  Causa: El token de acceso en el .env tenia espacios alrededor del signo igual:
    MERCADOPAGO_ACCESS_TOKEN = APP_USR-...
  Node leia el token con espacios incluidos, lo que lo invalidaba ante la API de MercadoPago.
  Solucion: Eliminar los espacios:
    MERCADOPAGO_ACCESS_TOKEN=APP_USR-...
  Leccion: En archivos .env no se usan espacios alrededor del =.

PROBLEMA 2 - URL de redireccion a MercadoPago rota
  Error: el usuario no era redirigido al checkout despues de confirmar la compra.
  Causa: La URL tenia un bug de interpolacion. Faltaba el signo $ en el template literal:
    window.location.href = `https://mercadopago.com.ar{data.id}`;  // mal
  Solucion: Corregir la interpolacion:
    window.location.href = `https://www.mercadopago.com.ar/checkout/v1/redirect?pref_id=${data.id}`;
  Luego se simplfico usando el init_point que devuelve MercadoPago directamente:
    window.location.href = data.init_point;
  Leccion: data.init_point es la URL oficial que MercadoPago provee por convencion
  en la respuesta de la preferencia. Es la forma recomendada.

PROBLEMA 3 - auto_return invalid. back_url.success must be defined
  Error devuelto por MercadoPago al intentar crear la preferencia.
  Causa: Se habia activado auto_return: "approved" con back_urls apuntando a localhost.
  MercadoPago valida que las back_urls sean URLs publicas accesibles desde internet.
  localhost no es una URL publica, solo existe en la maquina del desarrollador.
  Solucion: Comentar auto_return: "approved" durante el desarrollo local.
  Las back_urls siguen definidas correctamente para cuando se despliegue en produccion.

PROBLEMA 4 - At least one policy returned UNAUTHORIZED
  Error 500 del servidor al intentar crear la preferencia.
  Causa: El token de acceso era invalido. Se estaba usando el nombre de usuario
  de la cuenta de prueba (TESTUSER3920171327548682112) en lugar del access token real.
  El nombre de usuario y el access token son cosas distintas:
    - nombre de usuario: identifica la cuenta (TESTUSER...)
    - access token: clave de autorizacion para usar la API (TEST-... o APP_USR-...)

  *** QUE ES EL ACCESS TOKEN ***
  El ACCESS TOKEN es la clave que autoriza al servidor a hacer llamadas a la API
  de MercadoPago en nombre de la cuenta del vendedor. Sin el, MercadoPago no sabe
  quien esta haciendo la solicitud y devuelve UNAUTHORIZED.
  Va en el archivo .env del servidor (NUNCA en el frontend):
    MERCADOPAGO_ACCESS_TOKEN=APP_USR-8395297306015993-...
  Y se usa en backEnd/src/config/environment.js para configurar el cliente oficial del SDK:
    const client = new MercadoPagoConfig({
      accessToken: process.env.MERCADOPAGO_ACCESS_TOKEN
    });

  *** CONFUSION QUE COSTO COMPRENDER - EL PREFIJO NO DETERMINA EL ENTORNO ***
  Durante el desarrollo se asumio que:
    TEST-...    = sandbox (pruebas)
    APP_USR-... = produccion (pagos reales)
  Esto es INCORRECTO y genero varios errores dificiles de diagnosticar.

  La realidad es que el prefijo del token (TEST- o APP_USR-) NO determina
  si se esta en modo prueba o en produccion.
  Lo que determina el entorno es DE QUE CUENTA proviene el token:
    - Si el token fue generado desde las CREDENCIALES DE PRUEBA de una cuenta
      (sea TEST- o APP_USR-), opera en sandbox. Las transacciones son simuladas.
    - Si el token fue generado desde las CREDENCIALES DE PRODUCCION,
      opera en produccion. Las transacciones son reales y mueven dinero real.

  En nuestro caso el token en uso es:
    APP_USR-8395297306015993-063014-...-3507432147
  El User ID 3507432147 corresponde al Seller Test User del panel de pruebas.
  Por lo tanto aunque empieza con APP_USR-, este token opera en SANDBOX.
  Las transacciones son simuladas y no mueven dinero real.

  RESUMEN: No mirar el prefijo del token. Verificar siempre desde que seccion
  del panel de MercadoPago fue obtenido: "Credenciales de prueba" o "Produccion".
  *** FIN CONFUSION ***

  *** FIN ACCESS TOKEN ***

  Solucion: Obtener el access token real desde el panel de developers de MercadoPago
  (Integraciones -> Credenciales de prueba -> ccess Token) y usarlo en el .env.

PROBLEMA 5 - Descuentos rechazados por MercadoPago
  Error al intentar enviar los items con precios individuales ya descontados.
```

```
  Causa: MercadoPago tiene validaciones internas sobre los precios y los descuentos
  proporcionales calculados por item podian generar inconsistencias o valores
  que no pasaban las validaciones de la API.
  Solucion: Cambiar la estrategia. En lugar de enviar cada producto con su precio
  descontado, se envia un unico item resumen con el total final ya calculado en React:
    { title: "Compra tienda online", quantity: 1, unit_price: totalFinal, currency_id: "ARS" }
  React calcula todo (bruto, descuento, total) y le pasa a MercadoPago solo el numero final.
  Leccion: Para un e-commerce con logica de descuentos propia, es mas simple y seguro
  delegar el calculo al frontend y enviar un unico monto a MercadoPago.

PROBLEMA 6 - "Una de las partes con la que intentas hacer el pago es de prueba"
  Error mostrado por MercadoPago en el checkout al intentar pagar.
  Causa: Se estaba mezclando una cuenta vendedora de prueba (token TEST-...)
  con una tarjeta real o una cuenta compradora real.
  MercadoPago en modo sandbox requiere que AMBAS partes sean de prueba:
    - Vendedor: cuenta con token TEST-... configurado en el servidor
    - Comprador: iniciar sesion en el checkout con el Buyer Test User
      (TESTUSER3168... con contrasena jo5dkclPpn) y usar tarjetas de prueba
  Solucion: Iniciar sesion en el checkout con el Buyer Test User y usar
  las tarjetas de prueba que provee MercadoPago en su panel de developers.

PROBLEMA 7 - La orden quedaba en estado "pendiente" aunque el pago fuera aprobado
  Causa: MercadoPago no redirige automaticamente siAn auto_return activado.
  El usuario completaba el pago pero quedaba en la pantalla de MercadoPago
  sin volver a la app. La pagina /pago-exitoso nunca se cargaba y por lo tanto
  Firestore nunca recibia la actualizacion del estado de la orden.
  Solucion: Se implementaron tres paginas de retorno (PagoExitoso, PagoFallido,
  PagoPendiente). Cada una lee el orderId de la URL y llama a actualizarEstadoOrden()
  del service para actualizar Firestore. En desarrollo se navega manualmente a esas
  URLs. En produccion con auto_return activado la redireccion sera automatica.
MercadoPago aprueba el pago ✓
MercadoPago intenta redirigir a http://localhost:5173/pago-exitoso?orderId=xxx
Como auto_return está comentado, no redirige automáticamente
PagoExitoso.jsx nunca se carga
La orden queda en "pendiente" en Firestore hasta que se despliegue en vercel en la segunda etapa del proyecto


------------------------------------------------------------
6. POR QUE SE IMPLEMENTO vercel.json
------------------------------------------------------------

React Router maneja las rutas del lado del cliente (SPA).
Cuando el usuario navega dentro de la app (de / a /carrito),
React Router intercepta el cambio y renderiza el componente
correspondiente sin recargar la pagina — funciona bien.

El problema ocurre cuando el usuario:
  - Recarga la pagina estando en /carrito, /login, /pago-exitoso, etc.
  - Entra directamente a una URL como /carrito desde el navegador
  - MercadoPago redirige a /pago-exitoso?orderId=xxx

En esos casos el navegador hace una peticion real al servidor de Vercel
pidiendo el archivo /carrito o /pago-exitoso. Vercel busca esos archivos
en el build y no los encuentra — porque no existen como archivos fisicos,
solo existen como rutas de React Router. Vercel devuelve un 404.

SOLUCION — vercel.json:
El "rewrite" le dice a Vercel: para CUALQUIER ruta que llegue,
servi siempre el archivo index.html. Una vez que index.html carga,
React Router lee la URL del navegador y renderiza el componente correcto.

{
  "rewrites": [
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}

Es la configuracion estandar para deployar SPAs (Single Page Applications)
en Vercel con React Router. Sin este archivo, todas las rutas excepto "/"
devuelven 404 al recargar o al volver de MercadoPago.

NOTA: JSON no admite comentarios, por eso la explicacion esta aqui
y no dentro del archivo vercel.json.
```

src/components/Administrar/admin.jsx JSX · 29 lineas

```jsx
import { useAuth } from "../../context/auth/AuthProvider";
import { Navigate } from "react-router-dom";
import GestionProductos from "./GestionProductos";

export default function AdminI() {
  const { rol, loading } = useAuth();

  if (loading) {
    return <p>Cargando...</p>;
  }

  if (rol !== "admin") {
    return <Navigate to="/" replace />;
  }

  return (
    <div>
      <h1>Panel de Administración</h1>
      <GestionProductos />
    </div>
  );
}
/**
 * loading = true  →  muestra "Cargando..." (guarda para el caso de re-renders)
rol !== "admin" →  Navigate to="/" (redirige, nunca muestra el panel)
rol === "admin" →  renderiza <GestionProductos />
 *  (el componente GestionProductos se encarga de mostrar un mensaje si no hay productos)
 *
 */
```

src/components/Administrar/FormularioProducto.jsx JSX · 159 lineas

```
import { useState } from "react";
import styles from "./FormularioProducto.module.css";

// Componente hijo presentacional — solo renderiza el formulario.
// No tiene logica ni estado propio. Recibe datos y callbacks del padre.
// Flujo de bajada: padre -> hijo via datosForm y titulo.
// Flujo de subida: hijo -> padre via manejarCambio y manejarEnvio.
// Reutilizable para Alta, Edicion y cualquier operacion CRUD via prop 'titulo'.
// El selector de archivo genera una preview visual local (no sube el archivo al servidor).
// El admin debe copiar el archivo manualmente a /public y escribir la ruta en el campo de texto.
// El formulario recibe desde el padre los datos (datosForm)
// y los callbacks (manejarCambio y manejarEnvio).
// Los inputs capturan lo que escribe el usuario y envían los eventos al padre.
// El padre actualiza el estado mediante setDatosForm y vuelve a pasar
// los datos actualizados al formulario.

function FormularioProducto({ datosForm, manejarCambio, manejarEnvio, titulo }) {
  // previewUrl: URL temporal generada por el browser para mostrar la imagen seleccionada.
  // Solo existe en memoria mientras el formulario está abierto, no se guarda en ningún lado.
  const [previewUrl, setPreviewUrl] = useState(null);

  // Cuando el usuario selecciona un archivo, genera una URL temporal para el preview.
  // No sube nada al servidor — solo muestra cómo se vería la imagen.
  const manejarSeleccionArchivo = (evento) => {
    const archivo = evento.target.files[0];
    if (!archivo) return;

    // Revoca la URL anterior para liberar memoria
    if (previewUrl) URL.revokeObjectURL(previewUrl);

    const urlTemporal = URL.createObjectURL(archivo);
    setPreviewUrl(urlTemporal);
  };

  return (
    <div className={styles.card}>
      <h3 className={styles.titulo}>{titulo || "Producto"}</h3>

      <form onSubmit={manejarEnvio}>

        <div className={styles.campo}>
          <label className={styles.label}>Nombre del Producto</label>
          <input
            className={styles.input}
            type="text"
            name="nombre"
            placeholder="Ej: Teclado Mecanico"
            value={datosForm.nombre}
            onChange={manejarCambio}
            /*En React, los inputs controladosLos eventos onChange de inputs,
          selects y textareas suelen exponer
target.name y target.value.*/




          />
        </div>

        <div className={styles.campo}>
          <label className={styles.label}>Stock</label>
          <input
            className={styles.input}
            type="number"
            name="stock"
            placeholder="Ej: 10"
            value={datosForm.stock}
            onChange={manejarCambio}
          />
        </div>

        <div className={styles.campo}>
          <label className={styles.label}>Precio ($)</label>
          <input
            className={styles.input}
```

```
    type="number"
    name="precio"
    placeholder="Ej: 95"
    value={datosForm.precio}
    onChange={manejarCambio}
  />
</div>

<div className={styles.campo}>
  <label className={styles.label}>Categoria</label>
  {/* El select usa el mismo manejarCambio que los inputs via name="categoria" */}
  <select
    className={styles.select}
    name="categoria"
    value={datosForm.categoria}
    onChange={manejarCambio}
  >
    <option value="">Seleccionar...</option>
    <option value="computacion">Computacion</option>
    <option value="accesorios">Accesorios</option>
    <option value="audio">Audio</option>
    <option value="moviles">Moviles</option>
    <option value="almacenamiento">Almacenamiento</option>
    <option value="componentes">Componentes</option>
    <option value="redes">Redes</option>
    <option value="muebles">Muebles</option>
    <option value="gaming">Gaming</option>
    <option value="streaming">Streaming</option>
    <option value="fotografia">Fotografia</option>
    <option value="drones">Drones</option>
    <option value="wearables">Wearables</option>
    <option value="video">Video</option>
    <option value="seguridad">Seguridad</option>
  </select>
</div>

<div className={styles.campo}>
  <label className={styles.label}>Imagen</label>

  {/* Selector de archivo — solo genera preview, no sube nada */}
  <input
    type="file"
    accept="image/*"
    onChange={manejarSeleccionArchivo}
    className={styles.fileInput}
  />

  {/* Preview de la imagen seleccionada */}
  {previewUrl && (
    <div className={styles.previewContainer}>
      <img
        src={previewUrl}
        alt="Vista previa"
        className={styles.previewImage}
      />
      <p className={styles.previewWarning}>
        ⚠ Vista previa local. Para que la imagen funcione en la tienda,
        copiá el archivo a la carpeta <strong>/public</strong> y escribí la ruta abajo.
      </p>
    </div>
  )}

  {/* Campo de texto para la ruta final que se guarda en Firestore */}
  <input
    className={styles.input}
    type="text"
    name="imagen"
    placeholder="Ej: /11.jpeg"
    value={datosForm.imagen}
    onChange={manejarCambio}
  />
</div>

<button type="submit" className={styles.boton}>
  Guardar Producto
</button>
```

```
      </form>
    </div>
  );
}

export default FormularioProducto;
```

src/components/Administrar/FormularioProducto.module.css CSS · 165 lineas

```css
/* ============================================================
   FORMULARIO PRODUCTO — estilos del formulario CRUD
   Usado por FormularioProducto.jsx en el panel de administración
   ============================================================ */

/* Página completa: centra el formulario vertical y horizontalmente */
.pagina {
  display: flex;
  flex-direction: column;
  align-items: center;
  padding: 2rem 1rem;
}

.volverLink {
  display: block;
  margin-bottom: 12px;
  text-decoration: underline;
  color: blue;
  font-weight: 500;
  text-align: left;
}

/* Tarjeta blanca que contiene el formulario */
.card {
  background-color: var(--white);
  border-radius: var(--radius);
  box-shadow: var(--shadow);
  padding: 2rem;
  width: 100%;
  max-width: 480px;
}

/* Título del formulario */
.titulo {
  font-size: 1.4rem;
  color: var(--green-forest);
  margin-bottom: 1.5rem;
  border-bottom: 2px solid var(--green-forest);
  padding-bottom: 0.5rem;
}

/* Cada campo: label arriba, input abajo */
.campo {
  display: flex;
  flex-direction: column;
  gap: 4px;
  margin-bottom: 1rem;
}

.label {
  font-size: 0.9rem;
  font-weight: bold;
  color: var(--gray-dark);
}

/* Estilo compartido para input, number y select */
.input,
.select {
  padding: 0.6rem 0.8rem;
  border: 1px solid var(--gray-light);
  border-radius: 8px;
  font-size: 1rem;
  color: var(--gray-dark);
  background-color: var(--white);
  transition: border-color var(--transition);
}

/* Resalta el campo activo con el verde principal */
.input:focus,
.select:focus {
  outline: none;
  border-color: var(--green-forest);
}

/* Botón de envío */
```

```
.boton {
  width: 100%;
  padding: 0.75rem;
  margin-top: 0.5rem;
  background-color: var(--green-forest);
  color: var(--white);
  border: none;
  border-radius: 8px;
  font-size: 1rem;
  font-weight: bold;
  cursor: pointer;
  transition: background-color var(--transition), transform var(--transition);
}

.boton:hover {
  background-color: var(--green-forest-dark);
  transform: translateY(-2px);
}

/* Preview de imagen */
.fileInput {
  margin-bottom: 0.5rem;
  display: block;
}

.previewContainer {
  margin-bottom: 0.5rem;
}

.previewImage {
  width: 120px;
  height: 120px;
  object-fit: cover;
  border-radius: 6px;
  border: 1px solid #ccc;
}

.previewWarning {
  font-size: 0.75rem;
  color: #e65100;
  margin-top: 0.3rem;
}

/* ============================================================
   RESPONSIVE
   ============================================================ */

@media (max-width: 768px) {
  .pagina {
    padding: 1.5rem 1rem;
  }

  .card {
    padding: 1.5rem;
    max-width: 100%;
  }

  .titulo {
    font-size: 1.2rem;
  }
}

@media (max-width: 480px) {
  .pagina {
    padding: 1rem 0.5rem;
  }

  .card {
    padding: 1.25rem;
  }

  .titulo {
    font-size: 1.1rem;
  }

  .campo {
    margin-bottom: 0.8rem;
```

```
  }

  .input,
  .select {
    font-size: 0.95rem;
  }

  .previewImage {
    width: 100px;
    height: 100px;
  }
}
```

src/components/Administrar/GestionProductos.jsx JSX · 218 lineas

```jsx
import { useState, useEffect, useRef } from "react";
import FormularioProducto from "./FormularioProducto";
import {
  obtenerProductos,
  crearProducto,
  actualizarProducto,
  eliminarProducto,
} from "../../services/cargaProductos";
import styles from "./GestionProductos.module.css";

// Vista de administración de productos.
// Lista todos los productos con acciones Agregar, Editar y Eliminar.
// Reutiliza FormularioProducto para alta y edición via prop 'titulo' y estado 'modo'.
function GestionProductos() {
  const [productos, setProductos] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  // modo: null = solo tabla | "alta" = formulario nuevo | "edicion" = formulario con datos
  const [modo, setModo] = useState(null);
  const [productoEditando, setProductoEditando] = useState(null);

  const datosVacios = { nombre: "", stock: "", precio: "", categoria: "", imagen: "" };
  const [datosForm, setDatosForm] = useState(datosVacios);
  const formularioRef = useRef(null);

  // Carga inicial de productos desde Firestore
  useEffect(() => {
    const cargar = async () => {
      try {
        setLoading(true);
        setError(null);
        const data = await obtenerProductos();
        setProductos(data);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    };
    cargar();
  }, []);

  // Scroll automático al formulario cuando cambia el modo
  useEffect(() => {
    if (!modo) return;
    formularioRef.current?.scrollIntoView({ behavior: "smooth" });
  }, [modo]);

  // Abre el formulario en modo alta con campos vacíos
  const iniciarAlta = () => {
    // el cambio lo identifica react al renderizar nuevamente por cambio de estado
    // Un cambio de estado provoca un re-render.
    // vuelve a ejecutar la función componente para obtener un nuevo JSX.
    setModo("alta");
    setProductoEditando(null);
    setDatosForm(datosVacios);
  };

  // Abre el formulario en modo edición con los datos del producto seleccionado
  const iniciarEdicion = (producto) => {
    // los hooks (useState, useRef, useReducer) guardan sus valores fuera de la función,
    //  dentro de React, y por eso sobreviven entre ejecuciones entre re-renderes por cambio de estado
    setModo("edicion");
    setProductoEditando(producto);
    setDatosForm({
      nombre:    producto.nombre,
      stock:     producto.stock,
      precio:    producto.precio,
      categoria: producto.categoria,
      imagen:    producto.imagen || "",
    });
  };

  // Cierra el formulario y limpia el estado
```

```
  const cancelar = () => {
    setModo(null);
    setProductoEditando(null);
    setDatosForm(datosVacios);
  };

  // Maneja los cambios en el formulario (compartido para alta y edición)
  // React crea el objeto evento cuando el usuario interactúa con el formulario.
//El hijo recibe ese evento y lo pasa al callback del padre manejarCambio,
//que actualiza el estado correspondiente.
  const manejarCambio = (evento) => {
    const { name, value } = evento.target;
    console.log("Campo:", name, "Valor:", value);
    setDatosForm({
      ...datosForm,
      /*: el navegador siempre devuelve el value como un texto (string),
      incluso si el input es de tipo type="number".
      si es verdadero lo transformamos en un numero real si no queda como string : value, */
      [name]: name === "stock" || name === "precio" ? Number(value) : value,
    });
  };

  // Envia el formulario — llama a crearProducto o actualizarProducto según el modo
  //El padre le habla al hijo mediante props.
//El hijo le responde al padre mediante eventos.
  const manejarEnvio = async (evento) => {
    // const manejarCambio = (algoQueMeVanAPasar que es el evento que se creo en el hijo) => {
    evento.preventDefault();
    try {
      if (modo === "alta") {
        const nuevo = await crearProducto(datosForm);
        setProductos((prev) => [...prev, nuevo]);
        alert("Producto agregado correctamente");
      } else if (modo === "edicion") {
        await actualizarProducto(productoEditando.id, datosForm);
        setProductos((prev) =>
          prev.map((p) =>
            p.id === productoEditando.id ? { ...p, ...datosForm } : p
          )
        );
        alert("Producto actualizado correctamente");
      }
      cancelar();
    } catch (error) {
      console.error("Error al guardar producto:", error);
      alert(`Error: ${error.message}`);
    }
  };

  // Elimina el producto de Firestore y lo quita de la lista local
  const manejarEliminar = async (id, nombre) => {
    if (!confirm(`¿Seguro que querés eliminar "${nombre}"?`)) return;
    try {
      await eliminarProducto(id);
      setProductos((prev) => prev.filter((p) => p.id !== id));
      alert("Producto eliminado correctamente");
    } catch (error) {
      console.error("Error al eliminar producto:", error);
      alert(`Error: ${error.message}`);
    }
  };

  if (loading) return <p>Cargando productos...</p>;
  if (error) return <p>Error: {error}</p>;

  return (
    <div className={styles.container}>
      {/* Encabezado con título y botón Agregar alineado a la derecha */}
      <div className={styles.header}>
        <h2>Gestión de Productos</h2>
        {!modo && (
          <button
            onClick={iniciarAlta}
            className={styles.addButton}
          >
            + Agregar Producto
          </button>
```

```
        )}
      </div>

      {/* Formulario compartido — aparece en modo alta o edición */}
      {modo && (
        <div className={styles.formularioContainer} ref={formularioRef}>
          <FormularioProducto
            datosForm={datosForm}
            manejarCambio={manejarCambio}
            manejarEnvio={manejarEnvio}
            titulo={modo === "alta" ? "Agregar Nuevo Producto" : `Editar: ${productoEditando.nombre}`}
          />
          <button
            onClick={cancelar}
            className={styles.cancelButton}
          >
            Cancelar
          </button>
        </div>
      )}

      {/* Listado de productos */}
      <div className={styles.tableContainer}>
      <table className={styles.table}>
        <thead className={styles.thead}>
          <tr>
            <th className={styles.th}>Nombre</th>
            <th className={styles.th}>Categoria</th>
            <th className={styles.th}>Precio</th>
            <th className={styles.th}>Stock</th>
            <th className={styles.th}>Imagen</th>
            <th className={styles.th}>Acciones</th>
          </tr>
        </thead>
        <tbody className={styles.tbody}>
          {productos.map((producto) => (
            <tr key={producto.id}>
              <td className={styles.td}>{producto.nombre}</td>
              <td className={styles.td}>{producto.categoria}</td>
              <td className={styles.td}>${producto.precio}</td>
              <td className={styles.td}>{producto.stock}</td>
              <td className={styles.td}>{producto.imagen}</td>
              <td className={styles.td}>
                <button
                  onClick={() => iniciarEdicion(producto)}
                  className={styles.editButton}
                >
                  Editar
                </button>
                <button
                  onClick={() => manejarEliminar(producto.id, producto.nombre)}
                  className={styles.deleteButton}
                >
                  Eliminar
                </button>
              </td>
            </tr>
          ))}
        </tbody>
      </table>
      </div>
    </div>
  );
}

export default GestionProductos;
```

src/components/Administrar/GestionProductos.module.css CSS · 172 lineas

```css
/* ============================================================
   GESTION PRODUCTOS
   ============================================================ */

.container {
  width: 100%;
}

.header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 1rem;
  gap: 1rem;
}

.header h2 {
  margin: 0;
  color: var(--gray-dark);
}

.addButton {
  background-color: #42a5f5;
  color: #fff;
  border: none;
  padding: 0.5rem 1.2rem;
  border-radius: 4px;
  cursor: pointer;
  font-weight: bold;
  transition: background-color 0.2s;
}

.addButton:hover {
  background-color: #1976d2;
}

.formularioContainer {
  margin-bottom: 1.5rem;
}

.cancelButton {
  margin-top: 0.5rem;
  background-color: #757575;
  color: #fff;
  border: none;
  padding: 0.4rem 1rem;
  border-radius: 4px;
  cursor: pointer;
  transition: background-color 0.2s;
}

.cancelButton:hover {
  background-color: #616161;
}

.tableContainer {
  overflow-x: auto;
}

.table {
  width: 100%;
  border-collapse: collapse;
  margin-top: 1rem;
  min-width: 560px;
}

.thead {
  background-color: #f0f0f0;
}

.th {
  padding: 0.6rem 1rem;
  text-align: left;
  font-weight: bold;
  border-bottom: 2px solid #ccc;
```

```
}

.tbody tr {
  border-bottom: 1px solid #ddd;
}

.td {
  padding: 0.5rem 1rem;
}

.editButton {
  margin-right: 0.5rem;
  background-color: #1976d2;
  color: #fff;
  border: none;
  padding: 0.3rem 0.8rem;
  border-radius: 4px;
  cursor: pointer;
  transition: background-color 0.2s;
}

.editButton:hover {
  background-color: #1565c0;
}

.deleteButton {
  background-color: #d32f2f;
  color: #fff;
  border: none;
  padding: 0.3rem 0.8rem;
  border-radius: 4px;
  cursor: pointer;
  transition: background-color 0.2s;
}

.deleteButton:hover {
  background-color: #c62828;
}

/* ========================================================
   RESPONSIVE
   ======================================================== */

@media (max-width: 768px) {
  .header {
    flex-direction: column;
    align-items: flex-start;
  }

  .addButton {
    width: 100%;
  }

  .table {
    min-width: 500px;
  }

  .th,
  .td {
    padding: 0.4rem 0.6rem;
    font-size: 0.9rem;
  }

  .editButton,
  .deleteButton {
    padding: 0.25rem 0.6rem;
    font-size: 0.85rem;
  }
}

@media (max-width: 480px) {
  .header h2 {
    font-size: 1.2rem;
  }

  .table {
    min-width: 400px;
```

```
    font-size: 0.85rem;
  }

  .th,
  .td {
    padding: 0.3rem 0.5rem;
  }

  .editButton,
  .deleteButton {
    padding: 0.2rem 0.5rem;
    font-size: 0.8rem;
    margin-right: 0.3rem;
  }

  .editButton {
    margin-right: 0.3rem;
  }
}
```

src/components/Administrar/PromoverAdmin.jsx JSX · 88 lineas

```jsx
import { useState } from "react";
import { collection, query, where, getDocs, updateDoc, doc } from "firebase/firestore";
import { db } from "../../firebase/config";
import styles from "./PromoverAdmin.module.css";

// Permite al administrador promover un usuario cliente a administrador.
// Busca el documento en la colección "usuarios" por email y cambia el rol a "admin".
// Solo accesible desde /privada, que ya está protegida por RutaPrivadaElemental.
function PromoverAdmin() {
  const [emailBuscado, setEmailBuscado] = useState("");
  const [mensaje, setMensaje] = useState(null);
  const [error, setError] = useState(null);
  const [cargando, setCargando] = useState(false);

  const handlePromover = async (e) => {
    e.preventDefault();
    setMensaje(null);
    setError(null);
    setCargando(true);

    try {
      // Busca en la colección "usuarios" el documento cuyo campo email coincida
      const usuariosRef = collection(db, "usuarios");
      const q = query(usuariosRef, where("email", "==", emailBuscado));
      const snapshot = await getDocs(q);

      if (snapshot.empty) {
        setError(`No se encontró ningún usuario con el email: ${emailBuscado}`);
        return;
      }

      // Toma el primer documento que coincida (el email es único)
      const usuarioDoc = snapshot.docs[0];

      if (usuarioDoc.data().rol === "admin") {
        setMensaje(`${emailBuscado} ya es administrador.`);
        return;
      }

      // Actualiza solo el campo rol en el documento encontrado
      await updateDoc(doc(db, "usuarios", usuarioDoc.id), { rol: "admin" });

      setMensaje(`✓ ${emailBuscado} ahora es administrador.`);
      setEmailBuscado("");

    } catch (err) {
      console.error("Error al promover usuario:", err.message);
      setError("Hubo un error al intentar promover el usuario. Revisá la consola.");
    } finally {
      setCargando(false);
    }
  };

  return (
    <div className={styles.container}>
      <div className={styles.header}>
        <h3>Promover usuario a administrador</h3>
      </div>
      <p className={styles.description}>
        Ingresá el email de un usuario registrado para darle rol de administrador.
      </p>

      <form onSubmit={handlePromover} className={styles.form}>
        <input
          type="email"
          placeholder="Email del usuario"
          value={emailBuscado}
          onChange={(e) => setEmailBuscado(e.target.value)}
          required
          className={styles.input}
        />
        <button
          type="submit"
          disabled={cargando}
          className={styles.submitButton}
```

```
        >
          {cargando ? "Procesando..." : "Promover a admin"}
        </button>
      </form>

      {mensaje && <p className={styles.successMessage}>{mensaje}</p>}
      {error   && <p className={styles.errorMessage}>{error}</p>}
    </div>
  );
}

export default PromoverAdmin;
```

src/components/Administrar/PromoverAdmin.module.css CSS · 111 lineas

```css
/* ============================================================
   PROMOVER ADMIN
   ============================================================ */

.container {
  margin-top: 2rem;
  padding: 1.5rem;
  border: 1px solid #ddd;
  border-radius: 8px;
  max-width: 480px;
}

.header h3 {
  margin-top: 0;
  color: var(--gray-dark);
}

.description {
  color: #555;
  font-size: 0.9rem;
}

.form {
  margin-top: 1rem;
}

.input {
  width: 100%;
  padding: 0.5rem;
  margin-bottom: 0.8rem;
  border-radius: 4px;
  border: 1px solid #ccc;
  box-sizing: border-box;
  font-size: 1rem;
}

.input:focus {
  outline: none;
  border-color: #1976d2;
}

.submitButton {
  background-color: #1976d2;
  color: #fff;
  border: none;
  padding: 0.6rem 1.2rem;
  border-radius: 4px;
  cursor: pointer;
  font-size: 1rem;
  transition: background-color 0.2s;
}

.submitButton:hover:not(:disabled) {
  background-color: #1565c0;
}

.submitButton:disabled {
  opacity: 0.6;
  cursor: not-allowed;
}

.successMessage {
  color: #2e7d32;
  margin-top: 1rem;
}

.errorMessage {
  color: #c62828;
  margin-top: 1rem;
}

/* ============================================================
   RESPONSIVE
   ============================================================ */
```

```
@media (max-width: 768px) {
  .container {
    margin-top: 1.5rem;
    padding: 1.2rem;
    max-width: 100%;
  }

  .header h3 {
    font-size: 1.2rem;
  }

  .description {
    font-size: 0.85rem;
  }
}

@media (max-width: 480px) {
  .container {
    margin-top: 1rem;
    padding: 1rem;
  }

  .header h3 {
    font-size: 1.1rem;
  }

  .input {
    font-size: 0.95rem;
  }

  .submitButton {
    width: 100%;
    padding: 0.5rem 1rem;
  }
}
```

src/components/Administrar/TablaOrdenes.jsx JSX · 156 lineas

```jsx
import { useState, useEffect } from "react";
import { collection, getDocs, query, orderBy, deleteDoc, doc } from "firebase/firestore";
import { db } from "../../firebase/config";
import styles from "./TablaOrdenes.module.css";

// Muestra todas las órdenes generadas, ordenadas por fecha descendente.
// Solo accesible desde /privada (protegida por RutaPrivadaElemental).
function TablaOrdenes() {
  const [ordenes, setOrdenes] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  // id de la orden expandida para ver el detalle de items
  const [expandida, setExpandida] = useState(null);

  useEffect(() => {
    const cargar = async () => {
      try {
        const q = query(
          collection(db, "ordenes"),
          orderBy("fecha", "desc")   // más reciente primero
        );
        const snapshot = await getDocs(q);
        const data = snapshot.docs.map((doc) => ({
          id: doc.id,
          ...doc.data(),
          // Firestore Timestamp → Date de JS para formatear
          fechaJS: doc.data().fecha?.toDate() ?? null,
        }));
        setOrdenes(data);
      } catch (err) {
        console.error("Error al cargar órdenes:", err.message);
        setError("No se pudieron cargar las órdenes. Revisá la consola.");
      } finally {
        setLoading(false);
      }
    };
    cargar();
  }, []);

  const formatearFecha = (fecha) => {
    if (!fecha) return "Sin fecha";
    return fecha.toLocaleString("es-AR", {
      day:    "2-digit",
      month:  "2-digit",
      year:   "numeric",
      hour:   "2-digit",
      minute: "2-digit",
    });
  };

  const toggleExpandir = (id) => {
    setExpandida((prev) => (prev === id ? null : id));
  };

  const eliminarOrden = async (id) => {
    if (!window.confirm("¿Seguro que querés eliminar esta orden?")) return;
    try {
      await deleteDoc(doc(db, "ordenes", id));
      setOrdenes((prev) => prev.filter((o) => o.id !== id));
    } catch (err) {
      console.error("Error al eliminar la orden:", err.message);
      alert("No se pudo eliminar la orden. Revisá la consola.");
    }
  };

  if (loading) return <p>Cargando órdenes...</p>;
  if (error)   return <p style={{ color: "red" }}>{error}</p>;

  return (
    <div className={styles.container}>
      <div className={styles.header}>
        <h3>Órdenes generadas</h3>
      </div>

      {ordenes.length === 0 ? (
```

```
        <p className={styles.emptyMessage}>No hay órdenes registradas todavía.</p>
      ) : (
        <div className={styles.tableContainer}>
        <table className={styles.table}>
          <thead className={styles.thead}>
            <tr>
              <th className={styles.th}>Fecha</th>
              <th className={styles.th}>Usuario</th>
              <th className={styles.th}>Total</th>
              <th className={styles.th}>Estado</th>
              <th className={styles.th}>Items</th>
              <th className={styles.th}>Eliminar</th>
            </tr>
          </thead>
          {ordenes.map((orden) => (
              <tbody key={orden.id} className={styles.tbody}>
                <tr>
                  <td className={styles.td}>{formatearFecha(orden.fechaJS)}</td>
                  <td className={styles.td}>{orden.usuarioEmail}</td>
                  <td className={styles.td}>${orden.total}</td>
                  <td className={styles.td}>
                    <span className={styles.estadoBadge}>
                      {orden.estado}
                    </span>
                  </td>
                  <td className={styles.td}>
                    <button
                      onClick={() => toggleExpandir(orden.id)}
                      className={styles.actionButton}
                    >
                      {expandida === orden.id ? "Ocultar" : `Ver (${orden.items?.length ?? 0})`}
                    </button>
                  </td>
                  <td className={styles.td}>
                    <button
                      onClick={() => eliminarOrden(orden.id)}
                      className={`${styles.actionButton} ${styles.deleteButton}`}
                    >
                      Eliminar
                    </button>
                  </td>
                </tr>

                {/* Fila de detalle de items — visible solo cuando está expandida */}
                {expandida === orden.id && (
                  <tr className={styles.expandedRow}>
                    <td colSpan={5} className={styles.expandedCell}>
                      <table className={styles.subTable}>
                        <thead className={styles.subThead}>
                          <tr>
                            <th className={styles.subTh}>Producto</th>
                            <th className={styles.subTh}>Precio unitario</th>
                            <th className={styles.subTh}>Cantidad</th>
                            <th className={styles.subTh}>Subtotal</th>
                          </tr>
                        </thead>
                        <tbody className={styles.subTbody}>
                          {orden.items?.map((item, i) => (
                            <tr key={i}>
                              <td className={styles.subTd}>{item.nombre}</td>
                              <td className={styles.subTd}>${item.precio}</td>
                              <td className={styles.subTd}>{item.cantidad}</td>
                              <td className={styles.subTd}>${item.precio * item.cantidad}</td>
                            </tr>
                          ))}
                        </tbody>
                      </table>
                    </td>
                  </tr>
                )}
              </tbody>
            ))}
        </table>
        </div>
      )}
    </div>
  );
```

```
}

export default TablaOrdenes;
```

src/components/Administrar/TablaOrdenes.module.css CSS · 182 lineas

```css
/* ==========================================================
   TABLA ORDENES
   ========================================================== */

.container {
  margin-top: 2rem;
}

.header {
  margin-bottom: 1rem;
}

.header h3 {
  margin: 0;
  color: var(--gray-dark);
}

.emptyMessage {
  color: var(--gray-mid);
}

.tableContainer {
  overflow-x: auto;
}

.table {
  width: 100%;
  border-collapse: collapse;
  min-width: 600px;
}

.thead {
  background-color: #f0f0f0;
}

.th {
  padding: 0.6rem 1rem;
  text-align: left;
  font-weight: bold;
  border-bottom: 2px solid #ccc;
}

.tbody tr {
  border-bottom: 1px solid #ddd;
}

.td {
  padding: 0.5rem 1rem;
}

.estadoBadge {
  background-color: #e8f5e9;
  color: #2e7d32;
  padding: 0.2rem 0.6rem;
  border-radius: 12px;
  font-size: 0.85rem;
}

.actionButton {
  background: none;
  border: 1px solid #1976d2;
  color: #1976d2;
  padding: 0.2rem 0.6rem;
  border-radius: 4px;
  cursor: pointer;
  font-size: 0.85rem;
  transition: all 0.2s;
}

.actionButton:hover {
  background-color: #1976d2;
  color: white;
}

.deleteButton {
```

```
  border-color: #c62828;
  color: #c62828;
}

.deleteButton:hover {
  background-color: #c62828;
  color: white;
}

/* Tabla de items expandida */
.expandedRow {
  background-color: #fafafa;
}

.expandedCell {
  padding: 0.8rem 1.5rem;
}

.subTable {
  width: 100%;
  border-collapse: collapse;
}

.subThead {
  background-color: #eeeeee;
}

.subTh {
  padding: 0.4rem 0.8rem;
  text-align: left;
  font-weight: bold;
  color: #555;
}

.subTbody tr {
  border-bottom: 1px solid #e0e0e0;
}

.subTd {
  padding: 0.4rem 0.8rem;
}

/* ============================================================
   RESPONSIVE
   ============================================================ */

@media (max-width: 768px) {
  .container {
    margin-top: 1.5rem;
  }

  .header h3 {
    font-size: 1.2rem;
  }

  .table {
    min-width: 500px;
  }

  .th,
  .td {
    padding: 0.4rem 0.6rem;
    font-size: 0.9rem;
  }

  .actionButton {
    padding: 0.15rem 0.5rem;
    font-size: 0.8rem;
  }
}

@media (max-width: 480px) {
  .container {
    margin-top: 1rem;
  }

  .header h3 {
```

```
    font-size: 1rem;
  }

  .table {
    min-width: 400px;
    font-size: 0.85rem;
  }

  .th,
  .td {
    padding: 0.3rem 0.5rem;
  }

  .estadoBadge {
    font-size: 0.75rem;
    padding: 0.15rem 0.4rem;
  }

  .actionButton {
    padding: 0.1rem 0.4rem;
    font-size: 0.75rem;
  }

  .subTh,
  .subTd {
    padding: 0.3rem 0.5rem;
    font-size: 0.8rem;
  }
}
```

src/components/Auth/Login.jsx JSX · 158 lineas

```jsx
import { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { useAuth } from '../../context/auth/AuthProvider';
import styles from './Login.module.css';

function Login() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  // modoReset alterna entre el formulario de login y el de recuperación
  const [modoReset, setModoReset] = useState(false);
  const [emailReset, setEmailReset] = useState('');
  const [mensajeReset, setMensajeReset] = useState(null);
  const [errorReset, setErrorReset] = useState(null);

  const navigate = useNavigate();

  // user y rol vienen del contexto global — persisten aunque Login se desmonte
  // y remonte.
  // Si el usuario ya está autenticado y vuelve a /login, user != null y se muestra
  // la pantalla de bienvenida en lugar del formulario.
  const { login, resetPassword, rol, logout, user } = useAuth();

  const handleSubmit = async (e) => {
    e.preventDefault();
    try {
      await login(email, password);
      // No hace falta setear nada: user y rol del contexto ya se actualizan
      // solos via onAuthStateChanged. El re-render mostrará la bienvenida.
      setEmail("");
      setPassword("");
    } catch (error) {
      console.error("Error al iniciar sesión:", error.message);
    }
  };

  // Llama a Firebase Auth para enviar el email de restablecimiento.
  // Firebase maneja el envío del link — no toca Firestore.
  const handleReset = async (e) => {
    e.preventDefault();
    setMensajeReset(null);
    setErrorReset(null);
    try {
      await resetPassword(emailReset);
      setMensajeReset("Te enviamos un email con el link para restablecer tu contraseña. Revisá también la carpeta de
spam.");
      setEmailReset("");
    } catch (error) {
      // auth/user-not-found: el email no existe en Firebase Auth
      // auth/invalid-email: formato de email incorrecto
      setErrorReset("No encontramos una cuenta con ese email. Verificá que sea correcto.");
      console.error("Error al enviar reset:", error.message);
    }
  };

  // Vista de recuperación de contraseña
  if (modoReset) {
    return (
      <div className={styles.contenedor}>
        <h2>Recuperar contraseña</h2>
        <form className={styles.formularioVertical} onSubmit={handleReset}>
          <div className={styles.grupoCampo}>
            <input
              type="email"
              placeholder="Tu email registrado"
              value={emailReset}
              onChange={(e) => setEmailReset(e.target.value)}
              required
            />
          </div>
          <div className={styles.grupoAcciones}>
            <button type="submit" className={styles.btnPrincipal}>
              Enviar email de recuperación
            </button>
          </div>
        </form>
```

```
        {mensajeReset && <p className={styles.mensajeOk}>{mensajeReset}</p>}
        {errorReset   && <p className={styles.mensajeError}>{errorReset}</p>}
        <button
          className={styles.linkRecuperacion}
          onClick={() => { setModoReset(false); setMensajeReset(null); setErrorReset(null); }}
        >
          Volver al login
        </button>
      </div>
    );
  }

  // Si hay sesión activa (user del contexto), muestra bienvenida con accesos según rol.
  if (user) {
    return (
      <div className={styles.contenedor}>
        <h2 style={{ color: "#2e7d32" }}>¡Sesión iniciada!</h2>

        {rol === "admin" ? (
          <div>
            <p>Bienvenido, administrador. Tenés disponibles los siguientes accesos:</p>
            <div style={{ display: "flex", flexDirection: "column", marginTop: "1rem" }}>
              <button className={`${styles.btnNav} ${styles.btnAdmin}`}    onClick={() => navigate("/admin")}>Panel de
Administración — gestionar productos</button>
              <button className={`${styles.btnNav} ${styles.btnOrdenes}`}  onClick={() =>
navigate("/privada")}>Órdenes</button>
              <button className={`${styles.btnNav} ${styles.btnCatalogo}`} onClick={() => navigate("/")}>Ir al
catálogo</button>
              <button className={`${styles.btnNav} ${styles.btnSalir}`}    onClick={() => logout()}>Cerrar sesión</button>
            </div>
          </div>
        ) : (
          <div>
            <p>Bienvenido. Podés explorar el catálogo y realizar compras.</p>
            <div style={{ display: "flex", flexDirection: "column", marginTop: "1rem" }}>
              <button className={`${styles.btnNav} ${styles.btnCatalogo}`} onClick={() => navigate("/")}>Ir al
catálogo</button>
              <button className={`${styles.btnNav} ${styles.btnSalir}`}    onClick={() => logout()}>Cerrar sesión</button>
            </div>
          </div>
        )}
      </div>
    );
  }

  // Formulario de login — se muestra cuando no hay sesión activa
  return (
    <div className={styles.contenedor}>
      <form className={styles.formularioVertical} onSubmit={handleSubmit}>
        <h2>Iniciar Sesión:</h2>

        <div className={styles.grupoCampo}>
          <input
            type="email"
            placeholder="Email"
            value={email}
            onChange={(e) => setEmail(e.target.value)}
          />
        </div>

        <div className={styles.grupoCampo}>
          <input
            type="password"
            placeholder="Contraseña"
            value={password}
            onChange={(e) => setPassword(e.target.value)}
          />
        </div>

        <div className={styles.grupoAcciones}>
          <button type="submit" className={styles.btnPrincipal}>Ingresar</button>
          <p>
            <button
              type="button"
              className={styles.linkRecuperacion}
              onClick={() => setModoReset(true)}
            >
```

```
              ¿Olvidaste tu contraseña?
            </button>
          </p>
        </div>
      </form>
    </div>
  );
}

export default Login;
```

src/components/Auth/Login.module.css CSS · 131 lineas

```css
/* Contenedor centrado de la pantalla de login */
.contenedor {
  max-width: 420px;
  margin: 3rem auto;
  padding: 2rem;
  background: #fff;
  border-radius: 8px;
  box-shadow: 0 2px 12px rgba(0, 0, 0, 0.1);
}

.contenedor h2 {
  margin-bottom: 1.5rem;
  font-size: 1.4rem;
  color: #212121;
}

/* Formulario vertical */
.formularioVertical {
  display: flex;
  flex-direction: column;
  gap: 1rem;
}

/* Cada campo del formulario */
.grupoCampo {
  display: flex;
  flex-direction: column;
  gap: 0.3rem;
}

.grupoCampo input {
  padding: 0.55rem 0.8rem;
  border: 1px solid #bdbdbd;
  border-radius: 4px;
  font-size: 1rem;
  transition: border-color 0.2s;
}

.grupoCampo input:focus {
  outline: none;
  border-color: #1976d2;
}

/* Botón y link de recuperación */
.grupoAcciones {
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
  margin-top: 0.5rem;
}

.btnPrincipal {
  padding: 0.6rem 1.2rem;
  background-color: #1976d2;
  color: #fff;
  border: none;
  border-radius: 4px;
  font-size: 1rem;
  cursor: pointer;
  transition: background-color 0.2s;
}

.btnPrincipal:hover {
  background-color: #1565c0;
}

.grupoAcciones p {
  margin: 0;
}

.linkRecuperacion {
  background: none;
  border: none;
  color: #1976d2;
  cursor: pointer;
```

```
  text-decoration: underline;
  padding: 0;
  font-size: 0.9rem;
}

/* Botones de navegación post-login */
.btnNav {
  width: 100%;
  border: none;
  padding: 0.6rem 1.2rem;
  border-radius: 4px;
  cursor: pointer;
  text-align: left;
  font-size: 0.95rem;
  color: #fff;
}

.btnAdmin    { background-color: #1976d2; }
.btnOrdenes  { background-color: #6a1b9a; }
.btnCatalogo { background-color: #455a64; }
.btnSalir    { background-color: #b71c1c; }

.btnNav + .btnNav {
  margin-top: 0.5rem;
}

/* Mensajes de feedback */
.mensajeOk    { color: #2e7d32; margin-top: 0.5rem; }
.mensajeError { color: #c62828; margin-top: 0.5rem; }

/* ============================================================
   RESPONSIVE
   ============================================================ */

@media (max-width: 768px) {
  .contenedor {
    margin: 2rem 1rem;
    padding: 1.5rem;
  }

  .contenedor h2 {
    font-size: 1.2rem;
  }
}

@media (max-width: 480px) {
  .contenedor {
    margin: 1rem 0.5rem;
    padding: 1rem;
  }

  .btnPrincipal {
    width: 100%;
  }
}
```

src/components/Auth/Registro.jsx JSX · 103 lineas

```jsx
import { useState } from "react";
import { useAuth } from "../../context/auth/AuthProvider";
import { useNavigate } from "react-router-dom";
import styles from "./Login.module.css";

function Registro() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState(null);
  const [exito, setExito] = useState(false);
  const { register, user } = useAuth();
  const navigate = useNavigate();

  // Si el usuario ya está logueado, mostrar mensaje en lugar del formulario
  if (user) {
    return (
      <div className={styles.contenedor} style={{ textAlign: "center" }}>
        <h2>Ya estás registrado e iniciaste sesión</h2>
        <p>Estás logueado como <strong>{user.email}</strong>.</p>
        <div style={{ display: "flex", flexDirection: "column", marginTop: "1rem" }}>
          <button
            className={`${styles.btnNav} ${styles.btnCatalogo}`}
            onClick={() => navigate("/")}
          >
            Ir a la tienda
          </button>
        </div>
      </div>
    );
  }

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError(null);
    setExito(false);
    try {
      // El provider se encarga de:
      // 1. Crear usuario en Authentication
      // 2. Crear documento en Firestore
      // 3. Guardar rol = "cliente"
      // 4. Devolver la credencial
      const credencial = await register(email, password);
      const usuarioCreado = credencial.user;
      console.log("usuario registrado", usuarioCreado);
      setExito(true);
      setEmail("");
      setPassword("");
    } catch (err) {
      console.error(err.code);
      console.error(err.message);
      setError("No se pudo crear la cuenta. Verificá que el email sea válido y la contraseña tenga al menos 6
caracteres.");
    }
  };

  return (
    <div className={styles.contenedor}>
      <form className={styles.formularioVertical} onSubmit={handleSubmit}>
        <h2>Crear cuenta:</h2>

        <div className={styles.grupoCampo}>
          <input
            type="email"
            placeholder="Correo electrónico"
            value={email}
            onChange={(e) => setEmail(e.target.value)}
            required
          />
        </div>

        <div className={styles.grupoCampo}>
          <input
            type="password"
            placeholder="Contraseña (mínimo 6 caracteres)"
            value={password}
```

```
            onChange={(e) => setPassword(e.target.value)}
            required
          />
        </div>

        <div className={styles.grupoAcciones}>
          <button type="submit" className={styles.btnPrincipal}>
            Registrarse
          </button>
          <p>
            <button
              type="button"
              className={styles.linkRecuperacion}
              onClick={() => navigate("/login")}
            >
              ¿Ya tenés cuenta? Iniciá sesión
            </button>
          </p>
        </div>
      </form>

      {exito && <p className={styles.mensajeOk}>✓ Cuenta creada correctamente. Ya podés iniciar sesión.</p>}
      {error && <p className={styles.mensajeError}>{error}</p>}
    </div>
  );
}

export default Registro;
```

src/components/Carrito/Carrito.jsx JSX · 236 lineas

```jsx
import { useState, useContext } from "react";
import { useNavigate } from "react-router-dom";
import { CartContext } from "../../context/cart/CartContext";
import { useAuth } from "../../context/auth/AuthProvider";
import CarritoItem from "./CarritoItem";
import { crearOrden, validarStockDisponible } from "../../services/ordenesService";
import styles from "./Carrito.module.css";

// Umbral a partir del cual se aplica el bono de descuento
const UMBRAL_BONO   = 1_000_000;
const PORCENTAJE_BONO = 0.30;

// Genera un código de bono único con prefijo BONO- y 8 caracteres aleatorios
const generarCodigoBono = () =>
  "BONO-" + Math.random().toString(36).toUpperCase().slice(2, 10);

export function Carrito() {
  const { carrito, vaciarCarrito } = useContext(CartContext);
  const { user } = useAuth();
  const navigate  = useNavigate();

  // codigoBono: null = no generado aún, string = bono activo
  const [codigoBono,    setCodigoBono]    = useState(null);
  // inputBono: lo que el usuario tipea para canjear
  const [inputBono,     setInputBono]     = useState("");
  const [bonoAplicado,  setBonoAplicado]  = useState(false);
  const [errorBono,     setErrorBono]     = useState(null);

  // Total bruto sin descuento
  const totalBruto = carrito.reduce(
    (acc, item) => acc + item.precio * item.cantidad, 0
  );

  // Si el total supera el umbral y aún no se generó el bono, lo generamos
  // La generación ocurre una sola vez: cuando el estado pasa de null a string
  const bonoDisponible = totalBruto >= UMBRAL_BONO;

  if (bonoDisponible && !codigoBono) {
    setCodigoBono(generarCodigoBono());
  }

  const descuento   = bonoAplicado ? Math.round(totalBruto * PORCENTAJE_BONO) : 0;
  const totalFinal  = totalBruto - descuento;

  // Valida el código ingresado contra el generado para esta sesión
  const aplicarBono = () => {
    setErrorBono(null);
    if (inputBono.trim() === codigoBono) {
      setBonoAplicado(true);
    } else {
      setErrorBono("Código incorrecto. Verificá el código que aparece arriba.");
    }
  };

  const continuarComprando = () => navigate("/");

  const finalizarCompra = async () => {
    if (carrito.length === 0) return;
    // PUERTA DE COMPRA — control en el carrito:
    // Verifica que user exista antes de procesar la compra.
    // user es el objeto de Firebase Auth — si es null, no hay sesión activa.
    // A diferencia de RutaPrivadaElemental que controla el rol,
    // esta puerta solo verifica autenticación (cualquier usuario logueado puede comprar).
    if (!user) {
      alert("Debés iniciar sesión para finalizar tu compra.");
      navigate("/login");
      return;
    }

    const nuevaOrden = {
      usuarioId:     user.uid,
      usuarioEmail:  user.email,
      items: carrito.map((item) => ({
        id:       item.id,
        nombre:   item.nombre,
```

```
    precio:   item.precio,
    cantidad: item.cantidad,
  })),
  totalBruto,
  descuento,
  total:     totalFinal,
  bonoUsado: bonoAplicado ? codigoBono : null,
};

try {
  // ── VALIDACIÓN DE STOCK ──────────────────────────────────────
  // Verificamos contra Firestore que el stock real sea suficiente ANTES de
  // crear la orden. La compra aún no está pagada en este punto — es solo
  // una red de seguridad contra overselling (dos usuarios comprando lo mismo
  // al mismo tiempo o stock desactualizado en pantalla).
  const itemsParaValidar = carrito.map((item) => ({
    id:       item.id,
    nombre:   item.nombre,
    cantidad: item.cantidad,
  }));
  const { valido, faltantes } = await validarStockDisponible(itemsParaValidar);

  if (!valido) {
    const detalle = faltantes
      .map((f) => `• ${f.nombre}: pedís ${f.pedido}, hay ${f.disponible}`)
      .join("\n");
    alert(`Stock insuficiente para los siguientes productos:\n\n${detalle}\n\nPor favor actualizá tu carrito.`);
    return;
  }
  // ─────────────────────────────────────────────────────────────
  // Para reactivar el flujo directo a Firestore: descomentar estas 4 líneas
  // y comentar el bloque de Mercado Pago de abajo
  //
  // const orderId = await crearOrden(nuevaOrden);
  // console.log("Orden guardada en Firestore. ID:", orderId);
  // alert(`¡Compra exitosa!\nNúmero de orden: ${orderId}`);
  // vaciarCarrito();
  // navigate("/");
  // ─────────────────────────────────────────────────────────────

  // ── MERCADO PAGO ─────────────────────────────────────────────

  // 1. Guardar orden en Firestore con estado "pendiente"
  const orderId = await crearOrden(nuevaOrden);
  console.log("Orden pendiente creada en Firestore. ID:", orderId);

  // 2. Crear preferencia de pago en MercadoPago pasando el orderId
  const BASE_URL = import.meta.env.VITE_API_URL || "http://localhost:3001";
  const response = await fetch(`${BASE_URL}/pagos`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      items: nuevaOrden.items,
      totalBruto: Number(totalBruto),
      descuento: Number(descuento),
      totalFinal: Number(totalFinal),
      orderId,
    }),
  });

  const data = await response.json();
  console.log("Respuesta del backend:", data);

  if (!response.ok || !data.id) {
    alert(`No se pudo iniciar el pago: ${data.error || "Error desconocido"}`);
    return;
  }

  // 3. Vaciar carrito y redirigir a MercadoPago
  vaciarCarrito();
 // window.location.href = `https://www.mercadopago.com.ar/checkout/v1/redirect?pref_id=${data.id}`;
 // window.location.href = `https://www.mercadopago.com.ar/checkout/v1/redirect?pref_id=${data.id}`;
 window.location.href = data.init_point;
 // ─────────────────────────────────────────────────────────────

} catch (error) {
  console.error("Error al procesar la compra:", error);
```

```
      alert("Hubo un problema al procesar la compra. Intentá nuevamente.");
    }
  };

  return (
    <div className={styles.container}>
      <div className={styles.header}>
        <h2>Mi carrito</h2>
        <p className={styles.infoText}>
          ℹ️ Para probar el pago con tarjetas de prueba consultá el archivo <strong>INSTRUCCIONES_TARJETAS.txt</strong> en
la raíz del proyecto.
        </p>
      </div>

      {carrito.length === 0 ? (
        <p className={styles.emptyMessage}>El carrito está vacío.</p>
      ) : (
        carrito.map((item) => (
          <CarritoItem key={item.id} item={item} />
        ))
      )}

      {/* Resumen de precios */}
      <div className={styles.priceSummary}>
        <p className={styles.priceRow}>Subtotal: ${totalBruto.toLocaleString("es-AR")}</p>

        {bonoAplicado && (
          <p className={styles.discountRow}>
            Descuento 30%: -${descuento.toLocaleString("es-AR")}
          </p>
        )}

        <h3 className={styles.totalRow}>
          Total a pagar: ${totalFinal.toLocaleString("es-AR")}
        </h3>
      </div>

      {/* Sección de bono — aparece solo si el total supera el umbral */}
      {bonoDisponible && (
        <div className={styles.bonoSection}>
          <p className={styles.bonoTitle}>
            🎉 ¡Comprá superó $1.000.000! Tenés un bono del 30%
          </p>
          <p className={styles.bonoCode}>
            Tu código de bono: <strong>{codigoBono}</strong>
          </p>

          {!bonoAplicado ? (
            <div className={styles.bonoInputContainer}>
              <input
                type="text"
                placeholder="Ingresá el código"
                value={inputBono}
                onChange={(e) => setInputBono(e.target.value.toUpperCase())}
                className={styles.bonoInput}
              />
              <button
                onClick={aplicarBono}
                className={styles.bonoButton}
              >
                Aplicar
              </button>
            </div>
          ) : (
            <p className={styles.bonoApplied}>✓ Bono aplicado correctamente</p>
          )}

          {errorBono && <p className={styles.bonoError}>{errorBono}</p>}
        </div>
      )}

      <div className={styles.actionButtons}>
        <button onClick={continuarComprando} className={`${styles.actionButton} ${styles.continueButton}`}>
          Continuar comprando
        </button>
        <button onClick={finalizarCompra} className={`${styles.actionButton} ${styles.checkoutButton}`}>
          Finalizar compra
```

```
        </button>
      </div>
    </div>
  );
}

export default Carrito;
```

src/components/Carrito/Carrito.module.css CSS · 201 lineas

```css
/* ============================================================
   CARRITO
   ============================================================ */

.container {
  max-width: 800px;
  margin: 0 auto;
  padding: 20px;
}

.header {
  margin-bottom: 1rem;
}

.infoText {
  font-size: 0.85rem;
  color: #666;
  margin-top: 0.3rem;
}

.emptyMessage {
  text-align: center;
  padding: 2rem;
  color: #666;
}

/* ============================================================
   RESUMEN DE PRECIOS
   ============================================================ */

.priceSummary {
  margin-top: 1rem;
  border-top: 1px solid #ddd;
  padding-top: 1rem;
}

.priceRow {
  margin: 0.3rem 0;
  font-size: 1rem;
}

.discountRow {
  margin: 0.3rem 0;
  color: #2e7d32;
}

.totalRow {
  margin: 0.5rem 0;
  font-size: 1.2rem;
  font-weight: bold;
}

/* ============================================================
   SECCIÓN DE BONO
   ============================================================ */

.bonoSection {
  margin-top: 1.2rem;
  padding: 1rem;
  background-color: #fff8e1;
  border: 1px solid #ffe082;
  border-radius: 8px;
  max-width: 420px;
}

.bonoTitle {
  margin: 0 0 0.5rem;
  font-weight: bold;
  color: #f57f17;
}

.bonoCode {
  margin: 0 0 0.8rem;
  font-size: 0.9rem;
}
```

```
.bonoCode strong {
  letter-spacing: 0.1rem;
}

.bonoInputContainer {
  display: flex;
  gap: 0.5rem;
  align-items: center;
}

.bonoInput {
  padding: 0.4rem 0.6rem;
  border-radius: 4px;
  border: 1px solid #ccc;
  flex-grow: 1;
}

.bonoButton {
  background-color: #f57f17;
  color: #fff;
  border: none;
  padding: 0.4rem 0.8rem;
  border-radius: 4px;
  cursor: pointer;
  transition: background-color 0.2s;
}

.bonoButton:hover {
  background-color: #e65100;
}

.bonoApplied {
  color: #2e7d32;
  margin: 0;
}

.bonoError {
  color: #c62828;
  margin: 0.5rem 0 0;
  font-size: 0.85rem;
}

/* ========================================================
   BOTONES DE ACCIÓN
   ======================================================== */

.actionButtons {
  display: flex;
  gap: 1rem;
  margin-top: 1.5rem;
  flex-wrap: wrap;
}

.actionButton {
  padding: 0.6rem 1.2rem;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  font-size: 1rem;
  transition: background-color 0.2s;
}

.continueButton {
  background-color: #1976d2;
  color: #fff;
}

.continueButton:hover {
  background-color: #1565c0;
}

.checkoutButton {
  background-color: #4caf50;
  color: #fff;
}
```

```
.checkoutButton:hover {
  background-color: #388e3c;
}

/* ============================================================
   RESPONSIVE
   ============================================================ */

@media (max-width: 768px) {
  .container {
    padding: 15px;
  }

  .bonoSection {
    max-width: 100%;
  }

  .actionButtons {
    flex-direction: column;
  }

  .actionButton {
    width: 100%;
  }
}

@media (max-width: 480px) {
  .container {
    padding: 10px;
  }

  .header h2 {
    font-size: 1.5rem;
  }

  .bonoInputContainer {
    flex-direction: column;
    align-items: stretch;
  }

  .bonoInput {
    width: 100%;
  }

  .bonoButton {
    width: 100%;
  }
}
```

src/components/Carrito/CarritoItem.jsx JSX · 48 lineas

```jsx
import styles from "./CarritoItem.module.css";
import { useContext } from "react";
import { CartContext } from "../../context/cart/CartContext";

/**
 * CarritoItem

Se ocupa de:

renderizar UN producto
mostrar subtotal individual
estilos visuales
*/

function CarritoItem({ item }) {
    const { eliminarDelCarrito } = useContext(CartContext);
    return (
        <div className={`${styles.item} ${item.cantidad > 3 ? styles.highQuantity : ""}`}>
            <h3>{item.nombre}</h3>

            <p>Cantidad: {item.cantidad}</p>

            <p>Precio unitario: ${item.precio}</p>

            <p className={styles.subtotal}>
                Subtotal: ${item.precio * item.cantidad}
            </p>

            <button
                onClick={() => eliminarDelCarrito(item.id)}
                style={{
                    backgroundColor: "#dc3545",
                    color: "white",
                    border: "none",
                    padding: "8px 14px",
                    borderRadius: "4px",
                    cursor: "pointer",
                    marginTop: "10px"
                }}
            >
                Eliminar
            </button>
        </div>
    );
}

export default CarritoItem;
```

src/components/Carrito/CarritoItem.module.css CSS · 80 lineas

```css
/* ============================================================
   CARRITO ITEM
   ============================================================ */

.item {
  border: 1px solid #ccc;
  border-radius: 10px;
  padding: 1rem;
  margin-bottom: 1rem;
  background-color: #f9f9f9;
  transition: box-shadow 0.2s;
}

.item:hover {
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
}

.item h3 {
  margin: 0 0 0.5rem 0;
  color: var(--gray-dark);
  font-size: 1.1rem;
}

.item p {
  margin: 0.3rem 0;
  color: var(--gray-mid);
  font-size: 0.95rem;
}

.subtotal {
  font-weight: bold;
  color: var(--green-forest);
  font-size: 1rem;
}

/* Color verde para compras mayores a 3 unidades */
.highQuantity {
  color: #2e7d32;
}

/* ============================================================
   RESPONSIVE
   ============================================================ */

@media (max-width: 768px) {
  .item {
    padding: 0.8rem;
    margin-bottom: 0.8rem;
  }

  .item h3 {
    font-size: 1rem;
  }

  .item p {
    font-size: 0.9rem;
  }
}

@media (max-width: 480px) {
  .item {
    padding: 0.6rem;
    margin-bottom: 0.6rem;
    border-radius: 8px;
  }

  .item h3 {
    font-size: 0.95rem;
  }

  .item p {
    font-size: 0.85rem;
    margin: 0.25rem 0;
  }
```

```
  .subtotal {
    font-size: 0.95rem;
  }
}
```

src/components/CatalogoProductos/CatalogoProductos.jsx JSX · 196 lineas

```jsx
import { useState, useEffect, useContext } from "react";
import { Link } from "react-router-dom";
import { FiShoppingBag } from "react-icons/fi";
import { Item } from "../Item/Item";
import styles from "./CatalogoProductos.module.css";
import { CartContext } from "../../context/cart/CartContext";
import { obtenerProductos, actualizarProducto } from "../../services/cargaProductos";
/*
Orquestador de la tienda. Carga el catalogo y renderiza la grilla de productos.
 Delega la carga de datos a cargaProductos — no sabe de fetch ni URLs.

Comunicacion con Carrito: ambos componentes estan en rutas distintas y nunca
 se hablan directamente. CatalogoProductos escribe en CartContext via
 agregarAlCarrito(), Carrito lee del mismo contexto. CartProvider (en App.jsx)
 mantiene el estado vivo mientras el usuario navega entre paginas.
   CatalogoProductos  →  escribe en el contexto via agregarAlCarrito()
  Carrito            →  lee del contexto via carrito[]
  El intermediario es CartProvider (en App.jsx), que vive por encima de ambas
 rutas y mantiene el estado vivo aunque el usuario navegue entre páginas lo use por ejemplo
 con el contador de unidades compradas
 crearCallbackCompra(productoId)  ← CatalogoProductos crea un callback por producto,
 capturando el id
  devuelve (cantidad) => { ... } ← esa funcion se pasa a Item como prop onCompra

Item opera con su estado local cantidadSeleccionada (+ / - / Comprar)
  al hacer click en Comprar llama onCompra(cantidadSeleccionada)
  Item no sabe el id ni donde va el dato, solo informa cuanto quiere el usuario

El callback recibe la cantidad, busca el producto por el id que ya tenia capturado
y ejecuta agregarAlCarrito + descuenta stock

*/

export function CatalogoProductos({ mensaje }) {
  const [productos, setProductos]       = useState([]);
  const [loading, setLoading]           = useState(true);
  const [error, setError]               = useState(null);

  const { agregarAlCarrito, totalUnidades } = useContext(CartContext);

  // Carga inicial — se ejecuta una sola vez al montar el componente.
  useEffect(() => {
    const cargar = async () => {
      try {
        setLoading(true);
        setError(null);
        const productosCargados = await obtenerProductos();
        console.log("productosCargados",productosCargados)

        // la actualizacion del estado rovoca que el componente se re-renderice
        setProductos(productosCargados);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    };

    cargar();
  }, []);

  // Patron closure: crearCallbackCompra(id) devuelve una funcion personalizada
  // para ese producto. El hijo (Item) la recibe como prop onCompra y la llama
  // con la cantidad cuando el usuario hace clic en "Comprar".
  //
  // Hace dos cosas:
  //   1. Agrega el producto al carrito global (CartContext)
  //   2. Descuenta el stock en el estado local para actualizar la UI
  //
  // El parametro "cantidad" que llega aqui es el valor final confirmado por el usuario.
  // En Item.jsx ese mismo valor se llama "cantidadSeleccionada" mientras el usuario
  // usa + / - para elegir. Al hacer click en Comprar, cantidadSeleccionada se pasa
  // como argumento a onCompra() y llega aqui como "cantidad" — ya es el valor definitivo.
  const crearCallbackCompra = (productoId) => {
    return async (cantidad) => {
```

```
    const productoComprado = productos.find((p) => p.id === productoId);
    console.log("Producto comprado:", productoComprado);
    if (productoComprado) {
      // 1. Agrega el producto al carrito global en memoria (Frontend)
      agregarAlCarrito(productoComprado, cantidad);

      // Calcular el nuevo stock restante
      const nuevoStock = productoComprado.stock - cantidad;

      try {
        // 2. IMPACTO EN FIRESTORE: Actualiza el stock definitivo en la base de datos
        // Pasamos el ID del producto y solo el campo que queremos modificar
        await actualizarProducto(productoId, { stock: nuevoStock });

        // 3. Si la base de datos se actualizó con éxito, actualizamos la UI localmente
        setProductos((productosActuales) =>
          productosActuales.map((p) =>
            p.id === productoId ? { ...p, stock: nuevoStock } : p
          )
        );
      } catch (errorFirebase) {
        console.error("No se pudo actualizar el stock en Firestore:", errorFirebase);
        alert("Hubo un problema de conexión al procesar el stock en el servidor.");
      }
    }
  };
};

if (loading) {
  return (
    <div className={styles.container}>
      <div className={styles.loading}>
        <div className={styles.spinner}></div>
        <h2>Cargando productos...</h2>
        <p>Por favor, espera un momento mientras cargamos el catalogo.</p>
      </div>
    </div>
  );
}

if (error) {
  return (
    <div className={styles.container}>
      <div className={styles.error}>
        <div className={styles.errorIcon}>aviso</div>
        <h2>Error al cargar los productos</h2>
        <p className={styles.errorMessage}>{error}</p>
        <div className={styles.errorActions}>
          <button onClick={() => window.location.reload()} className={styles.retryButton}>
            Reintentar
          </button>
          <button onClick={() => window.location.reload()} className={styles.reloadButton}>
            Recargar pagina
          </button>
        </div>
        <div className={styles.errorHelp}>
          <h3>Posibles soluciones:</h3>
          <ul>
            <li>Verifica tu conexion a internet</li>
            <li>Comprueba que la URL de firestore sea correcta</li>
            <li>Revisa la consola para mas detalles del error</li>
          </ul>
        </div>
      </div>
    </div>
  );
}

return (
  <div className={styles.container}>

    <header className={styles.header}>
      <h1>{mensaje || "Tienda Virtual"}</h1>

      <div className={styles.stats}>
        <span className={styles.stat}>{productos.length} Productos</span>
       <span className={styles.stat}>{totalUnidades} Unidades Compradas Hoy </span>
```

```
        </div>

        <div>
          <Link to="/carrito" className={styles.linkCarrito}>
            <h1 className={styles.textoEnlace}>
              <FiShoppingBag className={styles.iconoEnlace} /> Ver Compras realizadas
            </h1>
          </Link>
        </div>
      </header>

      {productos.length === 0 ? (
        <div className={styles.empty}>
          <h2>No hay productos disponibles</h2>
          <p>El catalogo esta vacio en este momento.</p>
        </div>
      ) : (
        <main className={styles.catalogo}>
          <div className={styles.grid}>
            {productos.map((producto) => (
              <Item
                key={producto.id}
                id={producto.id}
                categoria={producto.categoria}
                nombre={producto.nombre}
                precio={producto.precio}
                stock={producto.stock}
                imagen={producto.imagen}
                onCompra={crearCallbackCompra(producto.id)}
              />
            ))}
          </div>
        </main>
      )}



    </div>
  );
}

export default CatalogoProductos;
```

src/components/CatalogoProductos/CatalogoProductos.module.css CSS · 332 lineas

```css
/* ============================================================
   CONTENEDOR PRINCIPAL DE LA TIENDA
   ============================================================ */

.container {
  max-width: 1200px;
  margin: 0 auto;
  padding: 20px;
}

/* ============================================================
   HEADER DE LA TIENDA
   Degradado violeta/púrpura con el título y estadísticas.
   ============================================================ */

.header {
  text-align: center;
  margin-bottom: 40px;
  padding: 20px;
  /* degradado de azul-violeta a púrpura */
  background: linear-gradient(135deg, #60c67b 0%, #19a027 100%);
  color: white;
  border-radius: var(--radius);
  box-shadow: var(--shadow);
}

.header h1 {
  margin: 0 0 15px 0;
  font-size: 2.5em;
  font-weight: 300;
}
/* en el módulo CSS */
/* carrito*/
.linkCarrito {
  color: yellow;
  font-size: 1.2rem;
}

/* Fila de estadísticas: cantidad de productos y compras */
.stats {
  display: flex;
  justify-content: center;
  gap: 30px;
  flex-wrap: wrap;
}

/* Cada estadística en una píldora semitransparente */
.stat {
  background: rgba(255, 255, 255, 0.2);
  padding: 8px 16px;
  border-radius: 20px;
  font-size: 0.9em;
  backdrop-filter: blur(10px);
}

/* ============================================================
   ESTADO DE CARGA
   Se muestra mientras se hace el fetch de productos.json
   ============================================================ */

.loading {
  text-align: center;
  padding: 60px 20px;
  background: white;
  border-radius: var(--radius);
  box-shadow: var(--shadow);
}

/* Círculo giratorio azul-violeta */
.spinner {
  width: 50px;
  height: 50px;
  border: 4px solid #f3f3f3;           /* gris muy claro de base */
  border-top: 4px solid #667eea;       /* azul-violeta que gira */
  border-radius: 50%;
```

```
  animation: spin 1s linear infinite;
  margin: 0 auto 20px;
}

@keyframes spin {
  0%   { transform: rotate(0deg); }
  100% { transform: rotate(360deg); }
}

/* ========================================================
   ESTADO DE ERROR
   Se muestra si el fetch falla o el JSON tiene mal formato.
   ======================================================== */

.error {
  text-align: center;
  padding: 40px 20px;
  background: rgba(228, 230, 171, 0);
  border-radius: var(--radius);
  box-shadow: var(--shadow);
  max-width: 600px;
  margin: 0 auto;
}

.errorIcon {
  font-size: 4em;
  margin-bottom: 20px;
}

.error h2 {
  color: var(--red-alert);             /* rojo para el título de error */
  margin-bottom: 15px;
}

/* Caja rosada con el mensaje de error */
.errorMessage {
  color: var(--gray-mid);
  font-size: 1.1em;
  margin-bottom: 25px;
  background: #d73a7b;                 /* rosa claro */
  padding: 15px;
  border-radius: 8px;
  border: 1px solid #f5c6cb;          /* borde rosa */
}

.errorActions {
  display: flex;
  gap: 15px;
  justify-content: center;
  margin-bottom: 25px;
  flex-wrap: wrap;
}

.retryButton,
.reloadButton {
  padding: 12px 24px;
  border: none;
  border-radius: 8px;
  font-size: 1em;
  cursor: pointer;
  transition: all var(--transition);
}

/* Botón principal: verde confirmación */
.retryButton {
  background: var(--green-ok);
  color: white;
}

.retryButton:hover {
  background: #218838;
  transform: translateY(-2px);
}

/* Botón secundario: gris neutro */
.reloadButton {
  background: #6c757d;
```

```
  color: white;
}

.reloadButton:hover {
  background: #5a6268;
  transform: translateY(-2px);
}

/* Caja de ayuda con fondo gris muy claro */
.errorHelp {
  text-align: left;
  background: #f8f9fa;
  padding: 20px;
  border-radius: 8px;
  border: 1px solid #dee2e6;
}

.errorHelp h3 {
  color: #495057;                    /* gris oscuro medio */
  margin-bottom: 10px;
}

.errorHelp ul {
  padding-left: 20px;
  color: #6c757d;                    /* gris medio */
}

.errorHelp li {
  margin-bottom: 8px;
}

/* ============================================================
   ESTADO VACÍO
   Cuando el JSON existe pero no tiene productos.
   ============================================================ */

.empty {
  text-align: center;
  padding: 60px 20px;
  background: white;
  border-radius: var(--radius);
  box-shadow: var(--shadow);
}

/* ============================================================
   CATÁLOGO Y GRILLA DE PRODUCTOS
   ============================================================ */

.catalogo {
  margin-bottom: 40px;
}

/* Grilla responsiva: columnas de mínimo 300px */
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(300px, 1fr));
  gap: 25px;
  padding: 20px 0;
}

/* ============================================================
   HISTORIAL DE COMPRAS
   ============================================================ */

.historial {
  background: white;
  padding: 30px;
  border-radius: var(--radius);
  box-shadow: var(--shadow);
  margin-bottom: 40px;
}

/* Título con línea inferior azul-violeta */
.historial h2 {
  color: var(--gray-dark);
  margin-bottom: 20px;
  border-bottom: 2px solid #667eea;
```

```
  padding-bottom: 10px;
}

.historialList {
  display: flex;
  flex-direction: column;
  gap: 15px;
}

/* Cada fila del historial: info a la izquierda, total a la derecha */
.historialItem {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 15px;
  background: #f8f9fa;                /* gris muy claro de fondo */
  border-radius: 8px;
  border: 1px solid #dee2e6;
  transition: all var(--transition);
}

.historialItem:hover {
  transform: translateY(-2px);
  box-shadow: var(--shadow);
}

.historialInfo strong {
  display: block;
  color: var(--gray-dark);
  margin-bottom: 5px;
}

.historialInfo span {
  color: var(--gray-mid);
  font-size: 0.9em;
}

.historialTotal {
  text-align: right;
}

/* Total en verde confirmación */
.historialTotal strong {
  color: var(--green-ok);
  font-size: 1.1em;
  display: block;
  margin-bottom: 5px;
}

/* Fecha/hora en gris muy claro */
.historialTotal small {
  color: #999;
  font-size: 0.8em;
}

/* ============================================================
   RESPONSIVE
   ============================================================ */

@media (max-width: 768px) {
  .container {
    padding: 10px;
  }

  .header h1 {
    font-size: 2em;
  }

  /* Columnas más angostas en tablet */
  .grid {
    grid-template-columns: repeat(auto-fill, minmax(250px, 1fr));
    gap: 15px;
  }

  /* El historial se apila verticalmente */
  .historialItem {
    flex-direction: column;
```

```
    align-items: flex-start;
    gap: 10px;
  }

  .historialTotal {
    text-align: left;
  }
}

@media (max-width: 480px) {
  /* Una sola columna en móvil */
  .grid {
    grid-template-columns: 1fr;
  }

  .errorActions {
    flex-direction: column;
    align-items: center;
  }

  .retryButton,
  .reloadButton {
    width: 200px;
  }
}
```

src/components/CatalogoProductos/constans.jsx JSX · 12 lineas

```
// URL del mock REST — YA NO SE USA. Reemplazado por Firestore via cargaProductos.jsx
// export const URL_API =
//   "https://6a025e4b0d92f63dd2539b82.mockapi.io/api/v1/productos";
/*
React
↓
MockAPI (DEPRECADO)
fetch(URL_API) iba directo a la mock
↓
Ahora se usa: obtenerProductos() / crearProducto() / actualizarProducto() / eliminarProducto()
en src/services/cargaProductos.jsx conectados directamente a Firestore
*/
```

src/components/ChatIA/ChatIA.jsx JSX · 208 lineas

```
import { useState } from "react";
import styles from "./ChatIA.module.css";
import { Link } from "react-router-dom";

export const ChatIA = () => {
  const [mensaje, setMensaje]               = useState("");
  const [chat, setChat]                     = useState([]);
  const [cargando, setCargando]             = useState(false);
  const [nombre, setNombre]                 = useState("");
  const [telefono, setTelefono]             = useState("");
  const [mostrarFormulario, setMostrarFormulario] = useState(false);
  const [servidorOff, setServidorOff]       = useState(false);

  const detectarIntencion = (texto) => {
    const t = texto.toLowerCase();
    return (
      t.includes("comprar") ||
      t.includes("precio") ||
      t.includes("cuanto") ||
      t.includes("cuanto") ||
      t.includes("quiero") ||
      t.includes("interesa")
    );
  };

  const manejarEnvio = async (e) => {
    e.preventDefault();
    if (!mensaje.trim()) return;

    const textoUsuario = mensaje.toLowerCase();
    const respuestasVacias = ["si", "si", "ok", "dale", "bueno"];

    if (respuestasVacias.includes(textoUsuario)) {
      setChat((prev) => [
        ...prev,
        { tipo: "usuario", texto: mensaje },
        { tipo: "ia", texto: "Para que la vas a usar? (trabajo, estudio, gaming o diseno)" },
      ]);
      setMensaje("");
      return;
    }

    setChat((prev) => [...prev, { tipo: "usuario", texto: mensaje }]);

    if (detectarIntencion(mensaje)) {
      setMostrarFormulario(true);
      setChat((prev) => [
        ...prev,
        { tipo: "ia", texto: "Perfecto, dejame tus datos y te contactamos." },
      ]);
      setMensaje("");
      return;
    }

    setCargando(true);

    const instrucciones = `Eres vendedor de Laptop Store.

Si el usuario responde sin contexto (ej: "si", "ok"):
-> pedir aclaracion especifica

Nunca asumir intencion.
Nunca responder generico.

Catalogo disponible:
- HP 250 G8 - Intel i5, 8GB RAM, 256GB SSD - Oficina - $800
- Lenovo IdeaPad 3 - Ryzen 5, 16GB RAM, 512GB SSD - Uso general - $950
- ASUS TUF Gaming - Ryzen 7, 16GB RAM, RTX 3050 - Gaming - $1400

Reglas:
1. Recomendar 1 o 2 modelos.
2. Explicar por que.
3. Maximo 2 parrafos.
`;
```

```
  try {
    // VITE_API_URL apunta a localhost en desarrollo y al backend de Render en produccion.
    const BASE_URL = import.meta.env.VITE_API_URL || "http://localhost:3001";

    const response = await fetch(`${BASE_URL}/chat`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ mensaje: instrucciones + "\nUsuario: " + mensaje }),
    });

    const data = await response.json();
    let texto = data.texto || "No se obtuvo respuesta";

    if (
      texto.toLowerCase().includes("nombre") ||
      texto.toLowerCase().includes("telefono")
    ) {
      setMostrarFormulario(true);
    }

    setChat((prev) => [...prev, { tipo: "ia", texto }]);
    setMensaje("");
  } catch (err) {
    if (err.message && err.message.includes("Failed to fetch")) {
      setServidorOff(true);
    }
    setChat((prev) => [
      ...prev,
      { tipo: "ia", texto: "Error al conectar con el servidor" },
    ]);
  }

  setCargando(false);
};

const enviarDatos = () => {
  if (!nombre || !telefono) {
    alert("Completa nombre y telefono");
    return;
  }

  setChat((prev) => [
    ...prev,
    { tipo: "ia", texto: `Gracias ${nombre}. Te contactaremos al ${telefono}.` },
  ]);

  setNombre("");
  setTelefono("");
  setMostrarFormulario(false);
};

return (
  <div className={styles.wrapper}>
    <div className={styles.topBar}>
      <h4 className={styles.title}>Asistente Virtual</h4>
      <Link to="/" className={styles.volverLink}>
        Volver al Catalogo
      </Link>
    </div>

    {/* Aviso cuando el servidor no esta activo */}
    {servidorOff && (
      <div className={styles.servidorOff}>
        El servidor de IA no esta activo. Ejecuta en la terminal:
        <code> npm run server</code>
      </div>
    )}

    <div className={styles.chatBox}>
      {chat.map((msg, i) => (
        <div
          key={i}
          className={`${styles.message} ${msg.tipo === "usuario" ? styles.messageUser : styles.messageIA}`}
        >
          <span
            className={`${styles.bubble} ${msg.tipo === "usuario" ? styles.bubbleUser : styles.bubbleIA}`}
          >
```

```
              {msg.texto}
            </span>
          </div>
        ))}
      </div>

      <form onSubmit={manejarEnvio} className={styles.form}>
        <input
          type="text"
          value={mensaje}
          onChange={(e) => setMensaje(e.target.value)}
          placeholder="Hazme una pregunta..."
          className={styles.input}
        />
        <button type="submit" disabled={cargando} className={styles.sendButton}>
          {cargando ? "..." : "Enviar"}
        </button>
      </form>

      {!mostrarFormulario && (
        <button
          onClick={() => setMostrarFormulario(true)}
          className={styles.contactButton}
        >
          Quiero que me contacten
        </button>
      )}

      {mostrarFormulario && (
        <div className={styles.formContact}>
          <h5>Dejar datos para compra</h5>
          <div className={styles.formContactFields}>
            <input
              type="text"
              placeholder="Nombre"
              value={nombre}
              onChange={(e) => setNombre(e.target.value)}
              className={styles.input}
            />
            <input
              type="text"
              placeholder="Telefono"
              value={telefono}
              onChange={(e) => setTelefono(e.target.value)}
              className={styles.input}
            />
            <button onClick={enviarDatos} className={styles.sendButton}>
              Enviar datos
            </button>
          </div>
        </div>
      )}
    </div>
  );
};
```

src/components/ChatIA/ChatIA.module.css CSS · 248 lineas

```css
/* ============================================================
   BARRA SUPERIOR
   Título izquierda + volver derecha
   ============================================================ */

.topBar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 12px;
}

.volverLink {
  text-decoration: underline;
  color: blue;
  font-weight: 500;
}


/* ============================================================
   CHAT IA - CONTENEDOR PRINCIPAL
   Panel gris claro que envuelve todo el chat.
   ============================================================ */

.wrapper {
  padding: 20px;
  background: #f4f4f4;                /* gris muy claro de fondo */
  border-radius: var(--radius);
  margin: 20px 0;
}

/* Título del panel */
.title {
  margin: 0;
  color: var(--gray-dark);
}
/* ============================================================
   CAJA DE MENSAJES
   Área scrolleable donde aparecen los mensajes del chat.
   ============================================================ */

.chatBox {
  background: var(--white);
  height: 250px;
  overflow-y: auto;                   /* scroll vertical cuando se llena */
  padding: 10px;
  margin-bottom: 10px;
  border-radius: 6px;
}

.message {
  margin: 5px 0;
}

/* Mensajes del usuario alineados a la derecha */
.messageUser {
  text-align: right;
}

/* Mensajes de la IA alineados a la izquierda */
.messageIA {
  text-align: left;
}

/* Burbuja base: tamaño máximo del 80% del ancho */
.bubble {
  display: inline-block;
  padding: 8px 12px;
  border-radius: 10px;
  max-width: 80%;
}

/* Burbuja del usuario: azul con texto blanco */
.bubbleUser {
  background: #007bff;                /* azul estándar */
```

```
  color: var(--white);
}

/* Burbuja de la IA: gris claro con texto oscuro */
.bubbleIA {
  background: #e4e6eb;                /* gris claro */
  color: var(--gray-dark);
}

/* ============================================================
   FORMULARIO DE ENVÍO
   Input de texto + botón Enviar en una fila.
   ============================================================ */

.form {
  display: flex;
  gap: 8px;
  margin-bottom: 10px;
}

/* Input ocupa todo el espacio disponible */
.input {
  flex: 1;
  padding: 10px;
  border: 1px solid var(--gray-light); /* borde gris claro */
  border-radius: 6px;
}

/* Botón verde bosque */
.sendButton {
  padding: 10px 16px;
  background: var(--green-forest);
  color: var(--white);
  border: none;
  border-radius: 6px;
  cursor: pointer;
  transition: background var(--transition);
}

/* Hover: verde más oscuro */
.sendButton:hover:not(:disabled) {
  background: var(--green-forest-dark);
}

/* Deshabilitado mientras espera respuesta */
.sendButton:disabled {
  opacity: 0.6;
  cursor: not-allowed;
}

/* ============================================================
   BOTÓN "QUIERO QUE ME CONTACTEN"
   Naranja fuego para llamar la atención.
   ============================================================ */

.contactButton {
  margin-top: 10px;
  padding: 8px 16px;
  background: var(--orange-fire);      /* naranja fuego */
  color: var(--white);
  border: none;
  border-radius: 6px;
  cursor: pointer;
  transition: background var(--transition);
}

/* Hover: naranja más oscuro */
.contactButton:hover {
  background: #e65100;
}

/* ============================================================
   FORMULARIO DE CONTACTO
   Aparece cuando el usuario quiere dejar sus datos.
   ============================================================ */

.formContact {
```

```
  margin-top: 15px;
}

.formContact h5 {
  margin-bottom: 10px;
  color: var(--gray-dark);
}

/* Campos en fila, se apilan en pantallas chicas */
.formContactFields {
  display: flex;
  gap: 8px;
  flex-wrap: wrap;
}

/* ============================================================
   BANNER: servidor apagado
   Aparece cuando el fetch falla por ERR_CONNECTION_REFUSED
   ============================================================ */

.servidorOff {
  background-color: #fff3cd;       /* amarillo suave de advertencia */
  color: #856404;                  /* marrón oscuro legible */
  border: 1px solid #ffc107;
  border-radius: 8px;
  padding: 10px 14px;
  margin-bottom: 10px;
  font-size: 0.9rem;
}

/* El comando se muestra en monospace para que parezca terminal */
.servidorOff code {
  background-color: #f8f0d0;
  padding: 2px 6px;
  border-radius: 4px;
  font-family: monospace;
  font-weight: bold;
}

/* ============================================================
   RESPONSIVE
   ============================================================ */

@media (max-width: 768px) {
  .wrapper {
    padding: 15px;
    margin: 15px 0;
  }

  .chatBox {
    height: 200px;
  }

  .bubble {
    max-width: 90%;
  }

  .formContactFields {
    flex-direction: column;
  }

  .formContactFields input {
    width: 100%;
  }
}

@media (max-width: 480px) {
  .topBar {
    flex-direction: column;
    align-items: flex-start;
    gap: 8px;
  }

  .wrapper {
    padding: 10px;
    margin: 10px 0;
  }
```

```
  .chatBox {
    height: 180px;
    padding: 8px;
  }

  .form {
    flex-direction: column;
  }

  .sendButton {
    width: 100%;
  }

  .contactButton {
    width: 100%;
  }
}
```

src/components/Item/Item.jsx JSX · 105 lineas

```jsx
import { useState } from "react";
import styles from "./Item.module.css";
import { Link } from "react-router-dom";

export function Item({ id,nombre, precio, stock, imagen, onCompra }) {
  console.log("imagen  *****", imagen);
  // cantidadSeleccionada: estado local de este componente.
  // Representa cuantas unidades el usuario eligio con + / - antes de confirmar.
  // No es la cantidad que va al carrito todavia — es el selector previo al click en Comprar.
  // Distinto de "cantidad" en CartProvider, que es la cantidad ya confirmada dentro del carrito.
  const [cantidadSeleccionada, setCantidadSeleccionada] = useState(0);

  const sumar = () => {
    if (cantidadSeleccionada < stock) setCantidadSeleccionada((c) => c + 1);
  };

  const restar = () => {
    if (cantidadSeleccionada > 0) setCantidadSeleccionada((c) => c - 1);
  };

  const comprar = () => {
    if (cantidadSeleccionada === 0)
      return;
    // onCompra es el callback creado por crearCallbackCompra(productoId)
    // en CatalogoProductos.
    // Item no sabe el id del producto ni a donde va el dato.
    // Solo informa cuantas unidades quiere el usuario — el padre
    //  ya tiene el id capturado en el closure.
    onCompra(cantidadSeleccionada);
    setCantidadSeleccionada(0);
  };

  return (
    <article className={styles.card}>
      {id && <Link to={`/producto/${id}`}>Ver detalle</Link>}

      <img
        src={imagen || "/carrito1.png"}
        alt={nombre}
        className={styles.image}
        onError={(e) => {
           e.target.onerror = null;
          const imagenesRespaldo = [
            "/carrito1.png",
            "/carrito2.png",
            "/carrito3.png",
            "/carrito4.png",
            "/carrito5.png",
            "/carrito6.png",
            "/carrito7.png",
            "/carrito8.png",
            "/carrito9.png",
            "/carrito10.png",
            "/carrito11.png",
            "/carrito12.png",
            "/carrito13.png",
            "/carrito14.png",
            "/carrito15.png",
          ];

          // elige una imagen aleatoria
          const random =
            imagenesRespaldo[
              Math.floor(Math.random() * imagenesRespaldo.length)
            ];

          e.target.src = random;
        }}
      />

      <h3>{nombre}</h3>
      <p>${precio}</p>
      <p translate="no">Stock: {stock}</p>

      {stock <= 2 && <p className={styles.alerta}>⚠ Stock bajo - Reponer</p>}
```

```
      <div className={styles.quantityControl}>
        <button
          className={styles.quantityButton}
          onClick={restar}
          disabled={cantidadSeleccionada === 0}
        >
          -
        </button>
        <span className={styles.quantity}>{cantidadSeleccionada}</span>
        <button
          className={styles.quantityButton}
          onClick={sumar}
          disabled={cantidadSeleccionada >= stock}
        >
          +
        </button>
      </div>

      <button
        onClick={comprar}
        disabled={cantidadSeleccionada === 0}
        className={cantidadSeleccionada === 0 ? styles.buttonDisabled : styles.buttonActive}
      >
        Comprar
      </button>
    </article>
  );
}
```

src/components/Item/Item.module.css CSS · 205 lineas

```css
/* ============================================================
   CARD DE PRODUCTO
   Cada producto se muestra en una tarjeta blanca con sombra.
   ============================================================ */

.card {
  background-color: var(--white);
  border: 1px solid var(--gray-light);  /* borde gris claro */
  border-radius: var(--radius);
  overflow: hidden;
  box-shadow: var(--shadow);
  padding: 12px;
}

/* La imagen ocupa todo el ancho y tiene altura fija */
.image {
/* ocupa todo el ancho del contenedor padre (.card) */
   width: 100%;
   /* fuerza a que la imagen mantenga una proporción 16:9 (ancho/alto)
   aunque las imágenes originales tengan tamaños distintos */
  aspect-ratio: 16 / 9;

  object-fit: contain;  /* muestra completa, deja espacios */
object-position: top;
/* convierte la imagen en elemento de bloque
   evitando espacios raros típicos de elementos inline */
  display: block;
  /* redondea las esquinas de la imagen */
  border-radius: 8px;
  /* agrega separación debajo de la imagen */

  margin-bottom: 8px;
}

/* ============================================================
   ALERTA DE STOCK BAJO
   Aparece cuando quedan 2 o menos unidades.
   ============================================================ */

.alerta {
  color: var(--red-alert);              /* texto rojo */
  font-weight: bold;
  background-color: #f8d7da;           /* fondo rosa claro */
  padding: 4px 8px;
  border-radius: 4px;
  border: 1px solid var(--red-alert);
  font-size: 0.85em;
  margin: 4px 0;
  text-align: center;
}

/* ============================================================
   CONTROL DE CANTIDAD
   Botones circulares + / - con el número en el medio.
   ============================================================ */

.quantityControl {
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 15px;
  margin: 15px 0;
}

.quantityButton {
  width: 35px;
  height: 35px;
  border-radius: 50%;
  border: 2px solid var(--green-forest);
  background-color: var(--white);
  color: var(--green-forest);           /* verde sobre blanco */
  font-size: 1.2em;
  font-weight: bold;
  cursor: pointer;
  transition: all var(--transition);
```

```
  display: flex;
  align-items: center;
  justify-content: center;
}

/* Al hacer hover: se invierte — fondo verde, texto blanco */
.quantityButton:hover:not(:disabled) {
  background-color: var(--green-forest);
  color: var(--white);
  transform: scale(1.1);
}

/* Deshabilitado: semitransparente, sin cursor de mano */
.quantityButton:disabled {
  opacity: 0.5;
  cursor: not-allowed;
}

/* Número de unidades seleccionadas */
.quantity {
  font-size: 1.2em;
  font-weight: bold;
  min-width: 40px;
  text-align: center;
  color: var(--gray-dark);
}

/* ============================================================
   BOTÓN DE COMPRA
   Verde cuando hay unidades seleccionadas, gris cuando no.
   ============================================================ */

/* Estado activo: verde confirmación con sombra */
.buttonActive {
  width: 100%;
  padding: 8px 12px;
  border: 0;
  border-radius: 6px;
  background-color: var(--green-ok);   /* verde confirmación */
  color: var(--white);
  cursor: pointer;
  transition: all var(--transition);
  transform: translateY(-2px);
  box-shadow: 0 4px 12px rgba(40, 167, 69, 0.3);
}

.buttonActive:hover {
  background-color: #218838;           /* verde un poco más oscuro */
  transform: translateY(-3px);
}

/* Estado deshabilitado: gris, sin interacción */
.buttonDisabled {
  width: 100%;
  padding: 8px 12px;
  border: 0;
  border-radius: 6px;
  background-color: #6c757d;           /* gris neutro */
  color: var(--white);
  cursor: not-allowed;
  opacity: 0.7;
}

/* ============================================================
   ANIMACIONES
   ============================================================ */

/* Pulso suave para badges de stock nuevo */
@keyframes pulse {
  0%   { opacity: 1; }
  50%  { opacity: 0.7; }
  100% { opacity: 1; }
}

/* ============================================================
   RESPONSIVE
   ============================================================ */
```

```
@media (max-width: 768px) {
  .card {
    padding: 10px;
  }

  .image {
    aspect-ratio: 4 / 3;
  }

  .quantityButton {
    width: 30px;
    height: 30px;
    font-size: 1em;
  }

  .quantity {
    font-size: 1em;
    min-width: 35px;
  }

  .buttonActive,
  .buttonDisabled {
    padding: 6px 10px;
    font-size: 0.9rem;
  }
}

@media (max-width: 480px) {
  .card {
    padding: 8px;
  }

  .image {
    aspect-ratio: 3 / 2;
  }

  .quantityControl {
    gap: 10px;
    margin: 10px 0;
  }

  .quantityButton {
    width: 28px;
    height: 28px;
  }

  .alerta {
    font-size: 0.8em;
    padding: 3px 6px;
  }
}
```

src/components/Layout/Footer.jsx JSX · 43 lineas

```jsx
import styles from "./Footer.module.css";
//import staffImage from "./NuestroStaf.jpg";

function Footer() {
  return (
    <footer className={styles.footer}>
      <div className={styles.footerContent}>
        <div className={styles.footerSection}>
          <h3>Sobre Nosotros</h3>
          <p>Nos dedicamos al software en Groelandia</p>
          <p>Experiencia en todo lo conocido</p>
        </div>

        <div className={styles.footerSection}>
          <h3>Nuestro Staff</h3>
          <ul>
            <li>Lio Musquito</li>
            <li>Larry Dirty</li>
            <li>Mark Bazuca</li>
          </ul>
        </div>

        <div className={styles.footerSection}>
          <h3>Contacto</h3>
          <p>Dirección: Calle Principal 123, Groelandia</p>
          <p>Teléfono: +555 123-4567</p>
          <p>Email: lio.musquito@groelandia.com</p>
        </div>

        <div className={styles.footerSection}>
          <img src="/NuestroStaf.jpg" alt="Nuestro Staff" className={styles.staffImage} />
        </div>
      </div>

      <div className={styles.footerBottom}>
        <p>&copy; 2026 - Mi Tienda React - Groelandia Software</p>
      </div>
    </footer>
  );
}

export default Footer;
```

src/components/Layout/Footer.module.css CSS · 60 lineas

```css
/* ============================================================
   FOOTER
   Barra inferior en turquesa claro con texto negro.
   ============================================================ */

.footer {
  background-color: #8de2d6;  /* turquesa claro — color fijo, no variable */
  color: black;
  margin-top: 2rem;
}

.footerContent {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  gap: 2rem;
  padding: 2rem;
  max-width: 1200px;
  margin: 0 auto;
}

.footerSection {
  text-align: center;
}

.footerSection h3 {
  margin-bottom: 1rem;
  font-size: 1.1rem;
  font-weight: bold;
}

.footerSection p {
  margin: 0.5rem 0;
  font-size: 0.9rem;
}

.footerSection ul {
  list-style: none;
  padding: 0;
  margin: 0;
}

.footerSection li {
  margin: 0.3rem 0;
  font-size: 0.9rem;
}

.staffImage {
  max-width: 200px;
  height: auto;
  border-radius: 8px;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
}

.footerBottom {
  text-align: center;
  padding: 1rem;
  border-top: 1px solid rgba(0, 0, 0, 0.1);
  font-size: 0.85rem;
}
```

src/components/Layout/Header.jsx JSX · 22 lineas

```jsx
import styles from "./Header.module.css";
import logo from "../../assets/react.svg";
import Navbar from "./Navbar";

function Header() {
  return (
    <header className={styles.header}>
      <div className={styles.brand}>
        <img
          src={logo}
          alt="Logo React"
          className={styles.logo}
        />
        <h1 className={styles.title}>Mi Tienda React</h1>
      </div>
      <Navbar />
    </header>
  );
}

export default Header;
```

src/components/Layout/Header.module.css CSS · 52 lineas

```css
/* ============================================================
   HEADER PRINCIPAL
   Barra superior verde que contiene logo, título y navbar.
   ============================================================ */

.header {
  background-color: var(--green-forest);  /* verde oscuro de fondo */
  color: var(--white);
  padding: 1rem 2rem;
  display: flex;
  justify-content: space-between;  /* logo a la izquierda, nav a la derecha */
  align-items: center;
  gap: 1rem;
  flex-wrap: wrap;  /* en móvil se apilan verticalmente */
}

/* ============================================================
   BLOQUE IZQUIERDO: logo + título
   ============================================================ */

.brand {
  display: flex;
  align-items: center;
  gap: 1rem;
}

.logo {
  width: 42px;
  height: 42px;
}

.title {
  font-size: 1.5rem;
  color: var(--white);  /* blanco sobre el fondo verde */
}

/* ============================================================
   RESPONSIVE
   En pantallas chicas todo se centra y apila
   ============================================================ */

@media (max-width: 768px) {
  .header {
    flex-direction: column;
    text-align: center;
  }

  .brand {
    justify-content: center;
  }
}
```

src/components/Layout/Layout.jsx JSX · 37 lineas

```jsx
import Header from "./Header";
import Footer from "./Footer";
import styles from "./Layout.module.css";
import { Outlet } from "react-router-dom";
// <Outlet /> es el lugar donde React Router renderiza
// a la ruta hija que esté activa en ese momento.
function Layout() {
  return (
    <div className={styles.layout}>
      <Header />

      <main className={styles.main}>
        <Outlet />
      </main>

      <Footer />
    </div>
  );
}

export default Layout;

/*
export function Layout({ children }) {
  return (
    <div className={styles.layout}>
      <Header />

      <main className={styles.main}>
        {children}
      </main>

      <Footer />
    </div>
  );
}
*/
```

src/components/Layout/Layout.module.css CSS · 21 lineas

```css
/* ============================================================
   LAYOUT
   Envuelve toda la página: header arriba, footer abajo,
   y el contenido del medio crece para ocupar el espacio.
   ============================================================ */

.layout {
  min-height: 100vh;
  display: flex;
  flex-direction: column;  /* apila header → main → footer */
}

/* flex: 1 hace que el main crezca y empuje el footer hacia abajo,
   aunque la página tenga poco contenido */
.main {
  flex: 1;
  width: min(var(--container-width), 100%);
  margin-inline: auto;  /* centra horizontalmente */
  padding: 2rem 1rem;
}
```

src/components/Layout/Navbar.jsx JSX · 98 lineas

```jsx
import { useState } from "react";
import styles from "./Navbar.module.css";
import { NavLink } from "react-router-dom";
import { useAuth } from "../../context/auth/AuthProvider";
import { useNavigate } from "react-router-dom";

const Navbar = () => {
  const navigate = useNavigate();
  const { user, logout, rol } = useAuth();
  const [menuAbierto, setMenuAbierto] = useState(false);

  const handleLogout = async () => {
    await logout();
    navigate("/login");
  };

  const cerrarMenu = () => setMenuAbierto(false);

  const getClass = ({ isActive }) =>
    isActive
      ? `${styles.link} ${styles.active}`
      : styles.link;

  return (
    <nav className={styles.navbar}>
      {/* Botón hamburguesa — solo visible en mobile */}
      <button
        className={styles.hamburger}
        onClick={() => setMenuAbierto((prev) => !prev)}
        aria-label="Abrir menú"
      >
        ☰
      </button>

      {/* Links — en desktop siempre visibles, en mobile se muestran/ocultan */}
      <div className={`${styles.links} ${menuAbierto ? styles.linksAbierto : ""}`}>
        <NavLink to="/" className={getClass} end onClick={cerrarMenu}>Inicio</NavLink>
        <NavLink to="/chat" className={getClass} onClick={cerrarMenu}>Chat IA</NavLink>
        <NavLink to="/carrito" className={getClass} onClick={cerrarMenu}>Ver carrito</NavLink>

        {!user && (
          <NavLink to="/registro" className={getClass} end onClick={cerrarMenu}>Registrarse</NavLink>
        )}

        {/* PUERTA VISUAL — control en el Navbar:
            Muestra el link "Administración" solo si el rol es "admin".
            Misma lógica que RutaPrivadaElemental pero aplicada a visibilidad,
            no a acceso. Un cliente no ve la opción, un admin sí. */}
        {rol === "admin" && (
          <NavLink to="/admin" className={getClass} onClick={cerrarMenu}>Administración</NavLink>
        )}

        {user ? (
          <>
            <NavLink to="/privada" className={getClass} onClick={cerrarMenu}>Ordenes</NavLink>
            <button onClick={() => { handleLogout(); cerrarMenu(); }} className={styles.logoutButton}>
              Salir
            </button>
          </>
        ) : (
          <NavLink to="/login" className={getClass} onClick={cerrarMenu}>Login</NavLink>
        )}
      </div>
    </nav>
  );
};

export default Navbar;
// NavLink es igual a Link pero agrega la clase "active" automaticamente
// cuando la ruta actual coincide con el "to". Usamos la prop className
// como funcion para combinar el estilo base con el activo.
/**
 *  mantener el efecto visual (fondo naranja cuando está activo),
 * entonces necesito getClass y .activ
 *  envía el objeto { isActive }?
```

```
NavLink lo envía automáticamente
Si la ruta actual coincide con el to, envía { isActive: true }
Si no coincide, envía { isActive: false }
mecanismo de React Router para poder aplicar estilos condicionales al link activo.

const Navbar = () => {
  const getClass = ({ isActive }) =>
    isActive ? `${styles.link} ${styles.active}` : styles.link;

  return (
    <nav className={styles.navbar}>
      <NavLink to="/" className={getClass} end>Inicio</NavLink>
      <NavLink to="/chat" className={getClass}>Chat IA</NavLink>
      <NavLink to="/alta" className={getClass}>Alta Producto</NavLink>
      <NavLink to="/privada" className={getClass}>Área Privada</NavLink>
      <NavLink to="/carrito" className={getClass}>Ver carrito</NavLink>
    </nav>
  );
};

export default Navbar;*/
```

src/components/Layout/Navbar.module.css CSS · 92 lineas

```css
/* ============================================================
   NAVBAR
   ============================================================ */

.navbar {
  display: flex;
  align-items: center;
  gap: 1rem;
  flex-wrap: wrap;
  position: relative;
}

/* Botón hamburguesa — oculto en desktop */
.hamburger {
  display: none;
  background: none;
  border: none;
  font-size: 1.5rem;
  cursor: pointer;
  color: var(--orange-fire);
  padding: 0.3rem 0.6rem;
}

/* Contenedor de links */
.links {
  display: flex;
  align-items: center;
  gap: 1rem;
  flex-wrap: wrap;
}

/* En mobile: oculto por defecto, se muestra al abrir el menú */
@media (max-width: 768px) {
  .hamburger {
    display: block;
  }

  .links {
    display: none;
    flex-direction: column;
    align-items: flex-start;
    width: 100%;
    padding: 0.5rem 0;
    gap: 0.4rem;
  }

  .linksAbierto {
    display: flex;
  }
}

.logoutButton {
  background: rgb(232, 65, 19);
  color: #9dff00;
  border: none;
  border-radius: 8px;
  padding: 10px 15px;
  cursor: pointer;
}

.logoutButton:hover {
  opacity: 0.9;
}

/* ============================================================
   LINKS
   ============================================================ */

.link {
  background-color: var(--white);
  color: var(--orange-fire);
  padding: 0.6rem 1rem;
  border-radius: 8px;
  font-weight: bold;
  transition:
```

```
    background-color var(--transition),
    color var(--transition),
    transform var(--transition);
}

.link:hover {
  background-color: var(--orange-fire);
  color: var(--white);
  transform: translateY(-2px);
}

.active {
  background-color: var(--orange-fire);
  color: var(--white);
  transform: translateY(-2px);
}
```

src/components/Pago/PagoExitoso.jsx JSX · 71 lineas

```jsx
import { useEffect, useState } from "react";
import { useSearchParams, useNavigate } from "react-router-dom";
import { actualizarEstadoOrden } from "../../services/ordenesService";
import styles from "./PagoExitoso.module.css";

export function PagoExitoso() {
  const [searchParams] = useSearchParams();
  const navigate = useNavigate();
  const [estado, setEstado] = useState("procesando");

  useEffect(() => {
    const orderId = searchParams.get("orderId");

    if (!orderId) {
      setEstado("error");
      return;
    }

    const actualizar = async () => {
      try {
        await actualizarEstadoOrden(orderId, "pagada", {
          payment_id: searchParams.get("payment_id") || null,
          payment_status: searchParams.get("status") || "approved",
        });
        setEstado("ok");
      } catch (error) {
        console.error("Error actualizando orden:", error);
        setEstado("error");
      }
    };

    actualizar();
  }, [searchParams]);

  return (
    <div className={styles.container}>
      {estado === "procesando" && (
        <p className={styles.processing}>Procesando tu pago...</p>
      )}

      {estado === "ok" && (
        <>
          <h2 className={styles.successTitle}>✅ ¡Pago realizado con éxito!</h2>
          <p className={styles.successMessage}>Tu orden fue confirmada y guardada correctamente.</p>
          <button
            onClick={() => navigate("/")}
            className={styles.actionButton}
          >
            Volver a la tienda
          </button>
        </>
      )}

      {estado === "error" && (
        <>
          <h2 className={styles.errorTitle}>⚠️ No pudimos confirmar tu pago</h2>
          <p className={styles.errorMessage}>Si el pago se realizó correctamente, contactanos para verificarlo.</p>
          <button
            onClick={() => navigate("/")}
            className={`${styles.actionButton} ${styles.errorButton}`}
          >
            Volver a la tienda
          </button>
        </>
      )}
    </div>
  );
}

export default PagoExitoso;
```

src/components/Pago/PagoExitoso.module.css CSS · 88 lineas

```css
/* ============================================================
   PAGO EXITOSO
   ============================================================ */

.container {
  text-align: center;
  padding: 3rem;
}

.processing {
  color: var(--gray-mid);
}

.successTitle {
  color: #2e7d32;
  margin-bottom: 1rem;
}

.successMessage {
  color: var(--gray-dark);
  margin-bottom: 1.5rem;
}

.errorTitle {
  color: #c62828;
  margin-bottom: 1rem;
}

.errorMessage {
  color: var(--gray-dark);
  margin-bottom: 1.5rem;
}

.actionButton {
  margin-top: 1.5rem;
  background-color: #1976d2;
  color: #fff;
  border: none;
  padding: 0.6rem 1.4rem;
  border-radius: 4px;
  cursor: pointer;
  font-size: 1rem;
  transition: background-color 0.2s;
}

.actionButton:hover {
  background-color: #1565c0;
}

.errorButton {
  background-color: #c62828;
}

.errorButton:hover {
  background-color: #b71c1c;
}

/* ============================================================
   RESPONSIVE
   ============================================================ */

@media (max-width: 768px) {
  .container {
    padding: 2rem 1.5rem;
  }

  .successTitle,
  .errorTitle {
    font-size: 1.5rem;
  }
}

@media (max-width: 480px) {
  .container {
    padding: 1.5rem 1rem;
```

```
  }

  .successTitle,
  .errorTitle {
    font-size: 1.3rem;
  }

  .actionButton {
    width: 100%;
    padding: 0.5rem 1rem;
  }
}
```

src/components/Pago/PagoFallido.jsx JSX · 51 lineas

```jsx
import { useEffect } from "react";
import { useSearchParams, useNavigate } from "react-router-dom";
import { actualizarEstadoOrden } from "../../services/ordenesService";
import styles from "./PagoFallido.module.css";

export function PagoFallido() {
  const [searchParams] = useSearchParams();
  const navigate = useNavigate();

  useEffect(() => {
    const orderId = searchParams.get("orderId");
    if (!orderId) return;

    const actualizar = async () => {
      try {
        await actualizarEstadoOrden(orderId, "fallida", {
          payment_id: searchParams.get("payment_id") || null,
          payment_status: searchParams.get("status") || "rejected",
        });
      } catch (error) {
        console.error("Error actualizando orden:", error);
      }
    };

    actualizar();
  }, [searchParams]);

  return (
    <div className={styles.container}>
      <h2 className={styles.title}>❌ El pago no se pudo completar</h2>
      <p className={styles.message}>Tu orden no fue procesada. Podés intentarlo nuevamente.</p>
      <div className={styles.actions}>
        <button
          onClick={() => navigate("/carrito")}
          className={styles.actionButton}
        >
          Volver al carrito
        </button>
        <button
          onClick={() => navigate("/")}
          className={`${styles.actionButton} ${styles.secondaryButton}`}
        >
          Ir a la tienda
        </button>
      </div>
    </div>
  );
}

export default PagoFallido;
```

src/components/Pago/PagoFallido.module.css CSS · 86 lineas

```css
/* ============================================================
   PAGO FALLIDO
   ============================================================ */

.container {
  text-align: center;
  padding: 3rem;
}

.title {
  color: #c62828;
  margin-bottom: 1rem;
}

.message {
  color: var(--gray-dark);
  margin-bottom: 1.5rem;
}

.actions {
  display: flex;
  gap: 1rem;
  justify-content: center;
  margin-top: 1.5rem;
}

.actionButton {
  background-color: #4caf50;
  color: #fff;
  border: none;
  padding: 0.6rem 1.4rem;
  border-radius: 4px;
  cursor: pointer;
  font-size: 1rem;
  transition: background-color 0.2s;
}

.actionButton:hover {
  background-color: #388e3c;
}

.secondaryButton {
  background-color: #1976d2;
}

.secondaryButton:hover {
  background-color: #1565c0;
}

/* ============================================================
   RESPONSIVE
   ============================================================ */

@media (max-width: 768px) {
  .container {
    padding: 2rem 1.5rem;
  }

  .title {
    font-size: 1.5rem;
  }

  .actions {
    flex-direction: column;
    gap: 0.8rem;
  }

  .actionButton {
    width: 100%;
  }
}

@media (max-width: 480px) {
  .container {
    padding: 1.5rem 1rem;
```

```
  }

  .title {
    font-size: 1.3rem;
  }

  .actionButton {
    padding: 0.5rem 1rem;
  }
}
```

src/components/Pago/PagoPendiente.jsx JSX · 46 lineas

```jsx
import { useEffect } from "react";
import { useSearchParams, useNavigate } from "react-router-dom";
import { actualizarEstadoOrden } from "../../services/ordenesService";
import styles from "./PagoPendiente.module.css";

export function PagoPendiente() {
  const [searchParams] = useSearchParams();
  const navigate = useNavigate();

  useEffect(() => {
    const orderId = searchParams.get("orderId");
    if (!orderId) return;

    const actualizar = async () => {
      try {
        await actualizarEstadoOrden(orderId, "pendiente", {
          payment_id: searchParams.get("payment_id") || null,
          payment_status: searchParams.get("status") || "pending",
        });
      } catch (error) {
        console.error("Error actualizando orden:", error);
      }
    };

    actualizar();
  }, [searchParams]);

  return (
    <div className={styles.container}>
      <h2 className={styles.title}>⏳ Tu pago está pendiente de acreditación</h2>
      <p className={styles.message}>Una vez acreditado, tu orden será confirmada automáticamente.</p>
      <p className={styles.infoText}>
        Podés revisar el estado de tu orden en tu historial de compras.
      </p>
      <button
        onClick={() => navigate("/")}
        className={styles.actionButton}
      >
        Ir a la tienda
      </button>
    </div>
  );
}

export default PagoPendiente;
```

src/components/Pago/PagoPendiente.module.css CSS · 70 lineas

```css
/* ============================================================
   PAGO PENDIENTE
   ============================================================ */

.container {
  text-align: center;
  padding: 3rem;
}

.title {
  color: #f57f17;
  margin-bottom: 1rem;
}

.message {
  color: var(--gray-dark);
  margin-bottom: 0.5rem;
}

.infoText {
  font-size: 0.9rem;
  color: #666;
  margin-bottom: 1.5rem;
}

.actionButton {
  margin-top: 1.5rem;
  background-color: #1976d2;
  color: #fff;
  border: none;
  padding: 0.6rem 1.4rem;
  border-radius: 4px;
  cursor: pointer;
  font-size: 1rem;
  transition: background-color 0.2s;
}

.actionButton:hover {
  background-color: #1565c0;
}

/* ============================================================
   RESPONSIVE
   ============================================================ */

@media (max-width: 768px) {
  .container {
    padding: 2rem 1.5rem;
  }

  .title {
    font-size: 1.5rem;
  }
}

@media (max-width: 480px) {
  .container {
    padding: 1.5rem 1rem;
  }

  .title {
    font-size: 1.3rem;
  }

  .actionButton {
    width: 100%;
    padding: 0.5rem 1rem;
  }
}
```

src/components/ProductoDetalle/ProductoDetalle.jsx JSX · 100 lineas

```jsx
import { doc, getDoc } from "firebase/firestore";
import { useParams } from "react-router-dom";
import { useEffect, useState } from "react";
import { Link } from "react-router-dom";
import { db } from "../../firebase/config";
import styles from "./ProductoDetalle.module.css";

function ProductoDetalle() {

  const [productoDetalle, setProductoDetalle] = useState(null);
  const { id } = useParams();

  useEffect(() => {
    const obtenerProducto = async () => {

      //doc apunta al documento específico en la colección "productos" a diferencia
      //de collection que apunta a la colección completa

      const productoRef = doc(db, "productos", id);

      const producto = await getDoc(productoRef);

       if(!producto.exists()){
        console.log("Producto no encontrado");
        return;
       }
      setProductoDetalle({
        id: producto.id,
        ...producto.data()
      });

      console.log("producto :", producto, "data :", producto.data());

      // data() es un método del snapshot
      //  que extrae el contenido del documento


      console.log("imagen *****", producto.data().imagen);
    };
    obtenerProducto();
  }, [id]);

  // correctamente modelada la dependencia
  //el array de dependencias [id] asegura que el hook se
  //vuelva a ejecutar si el ID en la URL cambia

  if (!productoDetalle) {
    return <h2>Cargando producto...</h2>;
  }


  return (
    <div className={styles.container}>
      <div className={styles.header}>
        <h2>Detalle del Producto</h2>
        <p className={styles.productId}>ID: {id}</p>
      </div>

      <div className={styles.content}>
        <div className={styles.imageContainer}>
          <img
            src={productoDetalle.imagen}
            alt={productoDetalle.nombre}
            className={styles.image}
          />
        </div>

        <div className={styles.details}>
          <div className={styles.detailRow}>
            <span className={styles.detailLabel}>Nombre</span>
            <p className={styles.detailValue}>{productoDetalle.nombre}</p>
          </div>

          <div className={styles.detailRow}>
            <span className={styles.detailLabel}>Precio</span>
```

```
            <p className={styles.price}>${productoDetalle.precio}</p>
          </div>

          <div className={styles.detailRow}>
            <span className={styles.detailLabel}>Stock</span>
            <p className={`${styles.detailValue} ${styles.stock}`}>
              {productoDetalle.stock} unidades
            </p>
          </div>

          <div className={styles.detailRow}>
            <span className={styles.detailLabel}>Categoría</span>
            <span className={styles.category}>{productoDetalle.categoria}</span>
          </div>
        </div>
      </div>

      <Link to="/" className={styles.backLink}>
        Volver al Catálogo
      </Link>
    </div>
  );
}

export default ProductoDetalle;
```

src/components/ProductoDetalle/ProductoDetalle.module.css CSS · 174 lineas

```css
/* ============================================================
   PRODUCTO DETALLE
   ============================================================ */

.container {
  max-width: 800px;
  margin: 0 auto;
  padding: 20px;
}

.header {
  margin-bottom: 2rem;
  padding-bottom: 1rem;
  border-bottom: 2px solid var(--green-forest);
}

.header h2 {
  color: var(--green-forest);
  font-size: 2rem;
  margin: 0 0 0.5rem 0;
}

.productId {
  color: var(--gray-mid);
  font-size: 0.9rem;
}

.content {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 2rem;
  align-items: start;
}

.imageContainer {
  background: white;
  padding: 1rem;
  border-radius: var(--radius);
  box-shadow: var(--shadow);
}

.image {
  width: 100%;
  height: auto;
  border-radius: 8px;
  object-fit: contain;
}

.details {
  background: white;
  padding: 2rem;
  border-radius: var(--radius);
  box-shadow: var(--shadow);
}

.detailRow {
  margin-bottom: 1.5rem;
}

.detailLabel {
  font-weight: bold;
  color: var(--gray-dark);
  font-size: 0.9rem;
  text-transform: uppercase;
  margin-bottom: 0.3rem;
  display: block;
}

.detailValue {
  font-size: 1.2rem;
  color: var(--gray-dark);
}

.price {
  font-size: 2rem;
```

```
  color: var(--green-forest);
  font-weight: bold;
}

.stock {
  color: var(--gray-mid);
}

.category {
  display: inline-block;
  background: var(--green-forest);
  color: white;
  padding: 0.4rem 1rem;
  border-radius: 20px;
  font-size: 0.9rem;
}

.backLink {
  display: inline-block;
  margin-top: 2rem;
  padding: 0.6rem 1.2rem;
  background: var(--gray-dark);
  color: white;
  text-decoration: none;
  border-radius: 6px;
  transition: background var(--transition);
}

.backLink:hover {
  background: var(--gray-mid);
}

/* ============================================================
   RESPONSIVE
   ============================================================ */

@media (max-width: 768px) {
  .container {
    padding: 15px;
  }

  .header h2 {
    font-size: 1.5rem;
  }

  .content {
    grid-template-columns: 1fr;
    gap: 1.5rem;
  }

  .details {
    padding: 1.5rem;
  }

  .price {
    font-size: 1.5rem;
  }
}

@media (max-width: 480px) {
  .container {
    padding: 10px;
  }

  .header {
    margin-bottom: 1.5rem;
  }

  .header h2 {
    font-size: 1.3rem;
  }

  .imageContainer {
    padding: 0.5rem;
  }

  .details {
```

```
    padding: 1rem;
  }

  .detailRow {
    margin-bottom: 1rem;
  }

  .detailValue {
    font-size: 1rem;
  }

  .price {
    font-size: 1.3rem;
  }

  .backLink {
    width: 100%;
    text-align: center;
    margin-top: 1.5rem;
  }
}
```

src/components/RutaPrivadaElementales.jsx JSX · 20 lineas

```jsx
import { useEffect } from "react";
import { Navigate } from "react-router-dom";
import { useAuth } from "../context/auth/AuthProvider";

// PUERTA DE ACCESO — control en la ruta /privada:
// Lee el rol del contexto (el "documento de identidad" emitido por AuthProvider).
// Si loading = true: espera — Firebase todavía no confirmó el estado de autenticación.
// Si rol !== "admin": redirige al catálogo — el usuario no tiene permiso.
// Si rol === "admin": renderiza children — el administrador puede ver el contenido.
export function RutaPrivadaElemental({ children }) {

     const { rol, loading } = useAuth();

     if (loading) return null

     if (rol !="admin") return <Navigate to ="/" />
// admin renderiza lo que le pasamos y lo puede ver el admin
     return children

}
```

src/context/auth/AuthContext.jsx JSX · 14 lineas

```jsx
import { createContext } from "react";

export const AuthContext = createContext();
/*
creamos al contenedor vacio que luego contendra a
user
rol
login()
logout()
register()
loading


*/
```

src/context/auth/AuthProvider.jsx JSX · 230 lineas

```jsx
import { createContext, useContext, useState, useEffect } from 'react';
import { doc, setDoc, getDoc } from "firebase/firestore";
import { db } from "../../firebase/config";
import {
  onAuthStateChanged,
  signInWithEmailAndPassword,
  createUserWithEmailAndPassword,
  signOut,
  sendPasswordResetEmail
} from 'firebase/auth';
import { auth } from '../../firebase/config';

// ============================================================
// AUTENTICACIÓN vs AUTORIZACIÓN — dos conceptos distintos
// ============================================================
//
// AUTENTICACIÓN: verificar QUIÉN es el usuario.
//   Firebase Auth lo maneja: valida credenciales y emite un Token JWT.
//
// AUTORIZACIÓN: determinar QUÉ puede hacer ese usuario.
//   Firestore + el campo "rol" lo manejan: el token JWT viaja en cada
//   petición a Firestore, que lo verifica y aplica las reglas de seguridad.
//
// FLUJO COMPLETO:
//
// PASO 1 — Login y activación de onAuthStateChanged:
//   El usuario ingresa email y contraseña.
//   Firebase Auth valida los datos y devuelve un Token JWT al navegador.
//   El SDK guarda ese token en LocalStorage / IndexedDB automáticamente.
//   onAuthStateChanged detecta el cambio y avisa: "usuario XYZ_123 autenticado".
//
// PASO 2 — setDoc asigna el rol (autorización inicial):
//   Con el usuario ya autenticado, se ejecuta:
//     setDoc(doc(db, "usuarios", "XYZ_123"), { rol: "cliente" })
//
// PASO 3 — Qué viaja por la red al ejecutar setDoc:
//   El SDK NO envía texto plano. Envía una petición HTTP/gRPC a Google con:
//     {
//       "Authorization": "Bearer eyJhbGciOiJSUzI1NiIs...",  ← Token JWT adjuntado automáticamente
//       "Payload": { "rol": { "stringValue": "cliente" } }   ← los datos a guardar
//     }
//   El SDK adjunta el token automáticamente — no hace falta escribirlo.
//
// PASO 4 — Firestore recibe y desempaqueta la petición:
//   Antes de escribir, Firestore verifica el token JWT.
//   Si es válido, extrae la identidad y la mapea en request.auth.uid = "XYZ_123".
//
// PASO 5 — Las Reglas de Seguridad deciden:
//   Firestore evalúa las reglas configuradas. Ejemplo:
//     allow write: if request.auth != null && request.auth.uid == usuarioId;
//   Si se cumple → escribe el dato. Si no → rechaza la petición.
//
// El Token JWT actúa como puente invisible entre lo que detecta Auth
// y lo que valida Firestore en cada operación de lectura/escritura.
// ============================================================

// Contexto que distribuye user, rol, loading y las funciones de auth
// a todos los componentes que lo consuman via useAuth()
const AuthContext = createContext();

// REGISTRO — primer paso del ciclo de autenticación:
// 1. Crea el usuario en Firebase Auth (autenticación).
// 2. Crea su documento en Firestore con rol = "cliente" (autorización inicial).
//    Todo usuario nuevo nace como "cliente" — solo un admin puede promoverlo.
// El SDK adjunta el Token JWT automáticamente en la petición a Firestore (Paso 3).
const register = async (email, password) => {
  console.log("========== AUTH ==========");
  console.log(auth);
  console.log("Proyecto asociado:", auth.app);
  console.log("Usuario actual antes del registro:", auth.currentUser);
  const credencial = await createUserWithEmailAndPassword(auth, email, password);
  console.log("========== CREDENCIAL ==========");
  console.log(credencial);

  console.log("UID:", credencial.user.uid);
```

```
  console.log("EMAIL:", credencial.user.email);
  console.log("EMAIL VERIFICADO:", credencial.user.emailVerified);

  await setDoc(
    doc(db, "usuarios", credencial.user.uid),
    {
      email,
      rol: "cliente"
    }
  );

  return credencial;
};

export function AuthProvider({ children }) {

  // DOCUMENTO DE IDENTIDAD del usuario (autorización):
  // rol = null     → Firebase todavía no respondió
  // rol = "cliente"→ usuario sin privilegios especiales
  // rol = "admin"  → acceso a rutas y funciones protegidas
  // Las "puertas" del sistema (RutaPrivadaElemental, Navbar, Carrito)
  // leen este valor para decidir qué mostrar o permitir.
  const [rol, setRol] = useState(null);

  // Objeto del usuario autenticado de Firebase Auth (o null si no hay sesión).
  // Contiene uid, email, etc. Dice QUIÉN es el usuario — el rol dice QUÉ puede hacer.
  const [user, setUser] = useState(null);

  // Semáforo que bloquea el renderizado hasta que Firebase confirme el estado.
  // true  → Firebase no respondió aún → la app no muestra nada ({!loading && children})
  // false → Firebase respondió → la app renderiza con user y rol correctos
  // Sin loading, ambos estados serían null un instante al arrancar y las puertas
  // tomarían decisiones incorrectas (ej: redirigir a un admin al login).
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    // useEffect registra el callback — Firebase es quien lo ejecuta más tarde.
    // onAuthStateChanged se dispara tanto si hay sesión como si no hay —
    // en ambos casos llega a setLoading(false) y la app se renderiza.
    console.log("Registro de observador de autenticación");

    const unsubscribe = onAuthStateChanged(auth, async (currentUser) => {
      console.log("onAuthStateChanged:", currentUser);
      console.log("Estado de carga:", loading);
      console.log("Rol:", rol);
      console.log("Usuario:", currentUser);

// onAuthStateChanged(auth, callback)
// registra el callback dentro del servicio auth.
// Firebase Auth es quien "escuchara"  a partir de aca a
// los cambios de estado de autenticación.
// auth:
// Subject (Observable) del patrón Observer.
// Mantiene el estado de autenticación (currentUser, tokens, etc.).
//
// onAuthStateChanged(auth, callback):
// Registra un callback para ser notificado cuando cambie
// el estado de autenticación.
//
// El Observer es interno al SDK de Firebase.
// Detecta los cambios del Subject (auth) y ejecuta
// el callback registrado.

      setUser(currentUser); // actualiza la autenticación en el estado global

      if (currentUser) {
        // Hay usuario autenticado → busca su rol en Firestore (autorización).
        // El SDK adjunta el Token JWT automáticamente en esta petición (Paso 3).
        // Firestore verifica el token y aplica las reglas de seguridad (Pasos 4 y 5).
        console.log("usuario conectado con UID:", currentUser.uid);
        console.log("Usuario autenticado:", currentUser);
        const token = await currentUser.getIdToken();

        console.log("JWT:", token);
        console.log("Usuario autenticado, obteniendo rol desde Firestore...");
        const docRef = doc(db, "usuarios", currentUser.uid);
        const docSnap = await getDoc(docRef);
```

```
        if (docSnap.exists()) {
          // EMISIÓN DEL DOCUMENTO DE IDENTIDAD:
          // setRol guarda el rol leído de Firestore en el contexto global.
          // A partir de acá ese valor viaja con el usuario durante toda la sesión
          // y cada "puerta" lo consulta para autorizar o denegar el acceso.
          setRol(docSnap.data().rol);
        }
      } else {
        // CIERRE DE SESIÓN o sesión inexistente:
        // Se limpia tanto la autenticación (user) como la autorización (rol).
        // Todas las puertas quedan cerradas hasta una nueva autenticación.
        setUser(null);
        setRol(null);
        console.log("No hay usuario autenticado");
      }

      setLoading(false); // Firebase ya respondió, ahora sí podés mostrar la app
    });

    return () => unsubscribe(); // limpieza al desmontar: cancela el observador
  }, [auth]);

  // Funciones de autenticación que interactúan con Firebase Auth
  const login = (email, password) => {
    return signInWithEmailAndPassword(auth, email, password);
  };

  const logout = () => {
    return signOut(auth);
  };

  // Envía un email de restablecimiento de contraseña.
  // Firebase maneja el envío del link — no impacta en Firestore ni en el rol.
  const resetPassword = (email) => {
    return sendPasswordResetEmail(auth, email);
  };

  // Distribuye a toda la app:
  //   user     → QUIÉN es el usuario (autenticación)
  //   rol      → QUÉ puede hacer (autorización)
  //   loading  → semáforo hasta que Firebase responda
  //   login / logout / register / resetPassword → funciones de auth
  return (
    // La app permanece "congelada" mientras Firebase no confirme el estado.
    // {!loading && children} garantiza que user y rol ya tienen valores reales
    // cuando los componentes se renderizan por primera vez.
    <AuthContext.Provider value={{ user, login, logout, loading, register, rol, resetPassword }}>
      {!loading && children}
    </AuthContext.Provider>
  );
}

// Hook personalizado para consumir el contexto desde cualquier componente
export const useAuth = () => useContext(AuthContext);
/*


auth
│
├── currentUser  ← usuario autenticado
│
├── app          ← aplicación Firebase
│     │
│     ├── options
│     │     ├── apiKey
│     │     ├── projectId
│     │     ├── authDomain
│     │     └── ...
│     │
│     └── config
│
├── configuración del servicio Auth
│
├── observadores (onAuthStateChanged)
│
```

```
└── estado interno del SDK
*/
```

src/context/cart/CartContext.jsx JSX · 7 lineas

```jsx
import { createContext } from "react";

// Crea el canal de comunicacion global del carrito.
// No tiene estado ni logica — solo define el contexto.
// CartProvider es quien inyecta los datos.
export const CartContext = createContext();
```

src/context/cart/CartProvider.jsx JSX · 76 lineas

```jsx
import { useState } from "react";
import { CartContext } from "./CartContext";

// CartProvider administra el estado global del carrito y lo comparte
// con todos los componentes hijos via CartContext.Provider.
export function CartProvider({ children }) {

  const eliminarDelCarrito = (id) => {
    setCarrito(prev => prev.filter(item => item.id !== id));
};
  // El useState provoca el render de los componentes que lo usan.
  const [carrito, setCarrito] = useState([]);
/**
 *
 evitar la dependencia de dos estados (lo que en programación se conoce
  como Single Source of Truth o Fuente Única de Verdad). Al tener un solo
  estado (carrito), eliminamos
  la posibilidad de que un estado diga una cosa y el otro diga algo diferente.
 */
  //const [totalUnidades, setTotalUnidades] = useState(0);
  // no es necesario porque se calcula automaticamentede
  // acuerdo al estado del carrito
  // ademas produciria una dependencia de dos estados

  // Agrega un producto al carrito. Si ya existe, suma la cantidad.
  const agregarAlCarrito = (producto, cantidad) => {
    setCarrito((prev) => {
      console.log("Estado anterior del carrito:", prev);
      // uso el estado más actualizado que React entrega.
      const existe = prev.find((item) => item.id === producto.id);

      if (existe) {
        return prev.map((item) =>
          item.id === producto.id
            ? { ...item, cantidad: item.cantidad + cantidad }
            : item
        );
      }
      // ...prev (Spread Operator en Arrays): Copia todos los productos que ```8


      // { ...producto, cantidad }: Crea un nuevo objeto
      //  con todos los campos del producto original, pero sobrescribe
      // la cantidad con el valor que se quiere agregar.
//devolvemos el carrito con el nuevo producto
// como en este caso carrito estaba !!! vacio se agrega el producto y su cantidad al array vacio
      return [...prev, { ...producto, cantidad }];
    });
    // actualizamos el contadr de unidades que luego consumira CatalogoProductos
   // setTotalUnidades((prev) => prev + cantidad)
  };

  // Resetea el carrito a vacio. Se llama al finalizar la compra.
  const vaciarCarrito = () => {
    setCarrito([]);
  };
 // ESTADO DERIVADO: Se calcula automáticamente en cada renderizado
  const totalUnidades = carrito.reduce((acc, item) => acc + item.cantidad, 0);

  return (


    <CartContext.Provider value={{ carrito, agregarAlCarrito, vaciarCarrito, totalUnidades,eliminarDelCarrito  }}>
      {children}
    </CartContext.Provider>
  );
}
/**
 Usuario hace clic: Se ejecuta agregarAlCarrito.El estado cambia: Llama a setCarrito(...).
 Esto le avisa a React que el estado carrito tiene nuevos datos.React re-renderiza:
 Al cambiar el estado, React vuelve a ejecutar todo el código dentro de CartProvider
  de arriba a abajo.Cálculo instantáneo: Al ejecutarse el código otra vez,
  la línea del .reduce() se vuelve a leer, pero ahora usa el carrito actualizado.Componentes al día: El Provider recibe el
nuevo valor de
  totalUnidades y lo distribuye a todos los componentes (como tu catálogo) al instante.
```

```
 *
 */
```

src/firebase/config.js JS · 324 lineas

```
import { initializeApp } from "firebase/app";
import { getFirestore } from "firebase/firestore";
import { getStorage } from "firebase/storage";
import { getAuth } from "firebase/auth";
const firebaseConfig = {
  apiKey: import.meta.env.VITE_FIREBASE_API_KEY,
  authDomain: import.meta.env.VITE_FIREBASE_AUTH_DOMAIN,
  projectId: import.meta.env.VITE_FIREBASE_PROJECT_ID,
  storageBucket: import.meta.env.VITE_FIREBASE_STORAGE_BUCKET,
  messagingSenderId: import.meta.env.VITE_FIREBASE_MESSAGING_SENDER_ID,
  appId: import.meta.env.VITE_FIREBASE_APP_ID,
};
// cre la conexion con firebase
 const app = initializeApp(firebaseConfig);

export const db = getFirestore(app);
export const auth = getAuth(app);
export const storage = getStorage(app);
/*

// ============================================================
// ¿Qué significa "auth" según el contexto?
// ============================================================
//
// 1) "firebase/auth"
//    Es el MÓDULO DE AUTENTICACIÓN del SDK de Firebase.
//    Contiene todas las funciones relacionadas con la autenticación,
//    por ejemplo:
//      - getAuth()
//      - signInWithEmailAndPassword()
//      - createUserWithEmailAndPassword()
//      - signOut()
//      - onAuthStateChanged()
//
//    Ejemplo:
//      import { getAuth } from "firebase/auth";
//
//
// 2) getAuth(app)
//    Es una FUNCIÓN del módulo "firebase/auth".
//    Recibe el objeto "app" (la aplicación Firebase inicializada)
//    y crea o devuelve el servicio de autenticación asociado a
//    esa aplicación.
// • Si la aplicación aún no tiene un servicio de autenticación,
//   lo crea y lo asocia a esa aplicación.
//
// • Si el servicio ya existe, devuelve esa misma instancia.
//
// De esta manera, cada aplicación Firebase utiliza una única
// instancia del servicio de autenticación (Auth).
//    Ejemplo:
//      const auth = getAuth(app);
//
//
// 3) auth
//    Es una VARIABLE creada por nosotros.
//    Guarda el objeto devuelto por getAuth(app).
//    Ese objeto representa el SERVICIO DE AUTENTICACIÓN de nuestro
//    proyecto Firebase.
//
//    El nombre "auth" es solo una convención.
//    Podría llamarse perfectamente:
//
//      const servicioAuth = getAuth(app);
//      const authService = getAuth(app);
//      const autenticacion = getAuth(app);
//
//    Ese objeto contiene, entre otras cosas:
//
//      - auth.currentUser   → usuario autenticado
//      - auth.app           → aplicación Firebase asociada
//      - auth.config        → configuración interna
//      - métodos y estado internos del servicio
//
//    Luego se exporta para reutilizarlo en toda la aplicación:
```

```
//
//      export const auth = getAuth(app);
//
//    y en cualquier componente se importa:
//
//      import { auth } from "../../firebase/config";
//
// ============================================================
app
│
├── apiKey
├── projectId
├── authDomain
├── storageBucket
└── ...


========================
FIRESTORE - RESUMEN
========================

doc(db, coleccion, id)

Qué hace:
- Crea una referencia a un documento.

Parámetros:
- db: instancia de Firestore.
- coleccion: nombre de la colección.
- id: identificador del documento.

Devuelve:
- DocumentReference.

Efecto:
- Ninguno. No lee ni escribe datos.


collection(db, coleccion)

Qué hace:
- Crea una referencia a una colección.

Parámetros:
- db: instancia de Firestore.
- coleccion: nombre de la colección.

Devuelve:
- CollectionReference.

Efecto:
- Ninguno.


setDoc(ref, datos)

Qué hace:
- Crea o reemplaza completamente un documento.

Parámetros:
- ref: referencia del documento.
- datos: objeto a guardar.

Devuelve:
- Promise<void>

Efecto:
- Escribe datos en Firestore.


addDoc(collectionRef, datos)

Qué hace:
- Crea un documento con id automático.

Parámetros:
- collectionRef: referencia de colección.
```

- datos: objeto a guardar.

Devuelve:
- DocumentReference del documento creado.

Efecto:
- Inserta un nuevo documento.

getDoc(ref)

Qué hace:
- Lee un documento.

Parámetros:
- ref: referencia del documento.

Devuelve:
- DocumentSnapshot.

Efecto:
- Lectura de Firestore.

getDocs(queryOCollection)

Qué hace:
- Lee múltiples documentos.

Parámetros:
- queryOCollection: consulta o colección.

Devuelve:
- QuerySnapshot.

Efecto:
- Lectura de Firestore.

updateDoc(ref, datos)

Qué hace:
- Actualiza campos específicos.

Parámetros:
- ref: referencia del documento.
- datos: campos a modificar.

Devuelve:
- Promise<void>

Efecto:
- Modifica solo los campos indicados.

deleteDoc(ref)

Qué hace:
- Elimina un documento.

Parámetros:
- ref: referencia del documento.

Devuelve:
- Promise<void>

Efecto:
- Borra el documento.

query(...)

Qué hace:
- Construye una consulta.

Parámetros:
- colección y filtros.

Devuelve:
- Query.

Efecto:
- Ninguno. Solo prepara la consulta.

where(campo, operador, valor)

Qué hace:
- Agrega un filtro a una consulta.

Ejemplo:
- where("categoria", "==", "Notebook")

Devuelve:
- QueryConstraint.

Efecto:
- Ninguno por sí solo.

orderBy(campo, direccion)

Qué hace:
- Ordena resultados.

Ejemplo:
- orderBy("precio", "asc")

Devuelve:
- QueryConstraint.

Efecto:
- Ninguno por sí solo.

limit(cantidad)

Qué hace:
- Limita cantidad de resultados.

Ejemplo:
- limit(10)

Devuelve:
- QueryConstraint.

Efecto:
- Ninguno por sí solo.

serverTimestamp()

Qué hace:
- Inserta fecha y hora del servidor Firebase.

Devuelve:
- Timestamp especial.

Efecto:
- Guarda la fecha del servidor.

========================
SECUENCIA MÁS COMÚN
========================

Crear referencia:
const ref = doc(db, "productos", "1");

Guardar:
await setDoc(ref, datos);

Leer:
const snap = await getDoc(ref);

```
Actualizar:
await updateDoc(ref, datos);

Eliminar:
await deleteDoc(ref);

Consultar varios:
const q = query(
  collection(db, "productos"),
  where("categoria", "==", "Notebook"),
  orderBy("precio", "asc")
);

const snapshot = await getDocs(q);

*/
```

src/main.jsx JSX · 11 lineas

```jsx
import { createRoot } from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import "./styles/index.css";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);
```

src/services/cargaProductos.jsx JSX · 100 lineas

```jsx
import { collection, getDocs, addDoc, doc, updateDoc, deleteDoc } from "firebase/firestore";
import { db } from "../firebase/config";

// 1. Declaramos la referencia global para que la usen todas las funciones del archivo
const productosRef = collection(db, "productos");

// ========================================
// R - READ (función original intacta)
// ========================================
export async function obtenerProductos() {
  try {
    // ========================
    // FIRESTORE
    // ========================
    // peticion a la base de datos en forma asincronica usando la referencia global
    const snapshot = await getDocs(productosRef);

    const productos = snapshot.docs.map((doc) => ({
      ...doc.data(),
      // id del documento (string) va al final para que no sea pisado
      // por el campo numérico "id" que viene dentro de doc.data()
      id: doc.id,
    }));

    return productos;

  } catch (errorFirebase) {

    console.warn(
      "Firestore fallo, cargando JSON local...",
      errorFirebase.message
    );

    // ========================
    // FALLBACK LOCAL
    // ========================

    try {

      const localResponse = await fetch("/data/productos.json");

      if (!localResponse.ok) {
        throw new Error("No se pudo cargar el JSON local");
      }

      const localData = await localResponse.json();

      return localData;

    } catch (errorLocal) {

      throw new Error(
        "No se pudieron cargar productos desde ninguna fuente."
      );
    }
  }
}

// ========================================
// C - CREATE (Nueva función para el Alta)
// ========================================
export async function crearProducto(productoData) {
  try {
    // Usamos la misma referencia global 'productosRef'
    const docRef = await addDoc(productosRef, productoData);
    return { id: docRef.id, ...productoData };
  } catch (error) {
    console.error("Error al guardar en Firestore:", error.message);
    throw error;
  }
}

// ========================================
// U - UPDATE (Nueva función para Modificar)
// ========================================
```

```
export async function actualizarProducto(id, datosActualizados) {
  try {
    const productoDocRef = doc(db, "productos", id);
    await updateDoc(productoDocRef, datosActualizados);
    return { id, ...datosActualizados };
  } catch (error) {
    console.error("Error al actualizar en Firestore:", error.message);
    throw error;
  }
}

// ========================================
// D - DELETE (Nueva función para Eliminar)
// ========================================
export async function eliminarProducto(id) {
  try {
    const productoDocRef = doc(db, "productos", id);
    await deleteDoc(productoDocRef);
    return id;
  } catch (error) {
    console.error("Error al eliminar de Firestore:", error.message);
    throw error;
  }
}
```

src/services/migrarDatosIniciales.js JS · 71 lineas

```
// MIGRACIÓN INICIAL DESACTIVADA
// Este componente fue usado para cargar los datos iniciales a Firestore
// por primera vez. Una vez que los datos existen en la base, ya no es necesario.
// Se mantiene comentado como referencia histórica del proceso de seed.
//
// Para reactivar: descomentar el código y montar <CargaInicial /> en App.jsx

/*
import { useEffect } from "react";
import { db } from "../firebase/config.js";
import productosNew from "../data/productosNew.json";
import { doc, setDoc, collection, getDocs, limit, query  } from "firebase/firestore";

export default function CargaInicial() {
  useEffect(() => {
    async function cargarDatosIniciales() {
      try {
        await setDoc(doc(db, "test", "123"), { nombre: "prueba" });

        const productosRef = collection(db, "productos");
        const q = query(productosRef, limit(1));
        const querySnapshot = await getDocs(q);

        if (!querySnapshot.empty) {
          console.log("Firebase ya cuenta con datos iniciales.");
          return;
        }

        console.log("Firestore detectado como vacío. Cargando productos por primera vez...");
       // doc() Su propósito es:

//Crear una referencia a un documento de Firestore.
// db es la conexión a Firestore.

        for (const producto of productosNew) {
          const ref = doc(db, "productos", producto.id);
          // setDoc() guarda datos en la referencia indicada ref indica dónde guardar.
          // pára el primero sera
          // productos
          // └── 1
          console.log("Migrando producto:", producto.id);


          await setDoc(ref, {
            id: Number(producto.id),
            nombre: producto.nombre,
            precio: producto.precio,
            stock: producto.stock,
            categoria: producto.categoria,
            imagen: producto.imagen,
          });
          console.log(`Producto ${producto.id} migrado correctamente.`);
        }

        console.log("¡Todos los datos iniciales se inyectaron con éxito!");

      } catch (error) {
        console.error("Error crítico en el proceso de carga inicial:", error);
      }
    }
    cargarDatosIniciales();
  }, []);
  return null;
}
*/

// Exportamos null para que App.jsx no rompa si aún lo importa
export default function CargaInicial() {
  return null;
}
```

src/services/ordenesService.js JS · 93 lineas

```
import { collection, addDoc, doc, getDoc, updateDoc, serverTimestamp } from "firebase/firestore";
import { db } from "../firebase/config";
/**
 * Descripción de lo que hace la función
 * @param {tipo} nombre - Descripción del parámetro
 * @returns {tipo} Descripción de lo que devuelve
 * Utilidad:
 * l formato JSDoc. Es un estándar de documentación para
 *  JavaScript que los editores
 * como VS Code leen y muestran como ayuda contextual.
 * crearOrden( en cualquier parte del proyecto, VS Cod
e sugiere los parámetros y muestra la descripción
 * de la función y sus parámetros.
 * La función recibe un objeto con los datos de la orden y
 * devuelve el ID del documento creado en Firestore.
 *  El tipado ayuda a prevenir errores y mejora la experiencia
 * de desarrollo al detectar errores en tiempo de escritura.
 *  El tipado también permite autocompletar propiedades de objetos
 *  y detectar errores si se accede a una propiedad inexistente.
 *  La anotación @returns indica el tipo de dato que devuelve la función.
 *  El tipado de parámetros evita errores al llamar a la función
 *  con argumentos incorrectos.
 *  El tipado de retorno evita errores al asignar el resultado
 *  de la función a una variable de tipo incorrecto.
 */


/**
 * Valida que el stock disponible en Firestore sea suficiente para cada item del carrito.
 * Se llama ANTES de crear la orden — es una red de seguridad contra overselling.
 *
 * IMPORTANTE: el stock ya fue descontado visualmente al agregar al carrito (comportamiento
 * actual del catálogo). Esta validación protege el caso en que dos usuarios compren
 * el mismo producto simultáneamente o el stock haya cambiado desde que se cargó el catálogo.
 * La compra aún NO está pagada en este punto — solo se verifica que haya stock real.
 *
 * @param {Array} items - Items del carrito [{ id, nombre, cantidad }]
 * @returns {{ valido: boolean, faltantes: Array }} valido = true si todo tiene stock,
 *          faltantes = lista de productos con stock insuficiente
 */
export const validarStockDisponible = async (items) => {
  const faltantes = [];

  for (const item of items) {
    const productoRef = doc(db, "productos", item.id);
    const productoSnap = await getDoc(productoRef);

    if (!productoSnap.exists()) {
      faltantes.push({ nombre: item.nombre, disponible: 0, pedido: item.cantidad });
      continue;
    }

    const stockActual = productoSnap.data().stock ?? 0;
    if (stockActual < item.cantidad) {
      faltantes.push({
        nombre: item.nombre,
        disponible: stockActual,
        pedido: item.cantidad,
      });
    }
  }

  return { valido: faltantes.length === 0, faltantes };
};

/**
 * Crea una nueva orden en Firestore con estado "pendiente".
 * @param {object} ordenData - Datos de la orden (usuarioId, items, totales, etc.)
 * @returns {string} ID del documento creado
 */
export const crearOrden = async (ordenData) => {
  const docRef = await addDoc(collection(db, "ordenes"), {
    ...ordenData,
    fecha: serverTimestamp(),
    estado: "pendiente",
```

```
  });
  return docRef.id;
};

/**
 * Actualiza el estado de una orden existente en Firestore.
 * @param {string} orderId - ID del documento en Firestore
 * @param {string} estado - Nuevo estado: "pagada" | "fallida" | "pendiente"
 * @param {object} datosPago - Campos adicionales del pago (payment_id, payment_status)
 */
export const actualizarEstadoOrden = async (orderId, estado, datosPago = {}) => {
  const ordenRef = doc(db, "ordenes", orderId);
  await updateDoc(ordenRef, {
    estado,
    ...datosPago,
  });
};
```

src/services/pruebaLectura.js JS · 49 lineas

```
import { useEffect } from "react";
import { collection, getDocs ,query,orderBy} from "firebase/firestore";
import { db } from "../firebase/config.js";

export default function PruebaLectura() {
  useEffect(() => {
    async function probarObtenerProductos() {
      console.log("--- INICIANDO PRUEBA DE LECTURA DESDE FIRESTORE ---");
      try {
        const productosRef = collection(db, "productos");
        //Creamos la consulta ordenada numéricamente por el id interno
        const q = query(productosRef, orderBy("id", "asc"));

        const snapshot = await getDocs(q);

        const productos = snapshot.docs.map((doc) => ({
          id: doc.id,
          ...doc.data(),
        }));

        console.log("¡Éxito! Productos recuperados de Firestore:", productos);
        console.table(productos); // Te lo muestra ordenado en forma de tabla en la consola

      } catch (errorFirebase) {
        console.warn(
          "Firestore falló en la prueba, intentando fallback JSON local...",
          errorFirebase.message
        );

        try {
          const localResponse = await fetch("/data/productos.json"); // Ajustá a tu ruta de fallback
          if (!localResponse.ok) throw new Error("No se pudo cargar el JSON local");

          const localData = await localResponse.json();
          console.log("Productos recuperados desde Fallback Local:", localData);

        } catch (errorLocal) {
          console.error("Error total: No se pudo leer de ninguna fuente.", errorLocal.message);
        }
      }
      console.log("--- FIN DE LA PRUEBA ---");
    }

    probarObtenerProductos();
  }, []);

  return null; // Invisible, no rompe  renderizado actual
}
```

src/styles/globals.css CSS · 36 lineas

```css
/* ============================================================
   CLASES REUTILIZABLES GLOBALES
   Estas clases se pueden usar en cualquier componente
   sin necesidad de crear un módulo CSS nuevo.
   ============================================================ */

/* Centra el contenido y limita su ancho máximo */
.container {
  width: min(var(--container-width), 100%);
  margin-inline: auto;
  padding-inline: 1rem;
}

/* Texto secundario en gris medio */
.text-light {
  color: var(--gray-mid);
}

/* Bloque de "no hay nada para mostrar" */
.empty-state {
  text-align: center;
  padding: 3rem 1rem;
  color: var(--gray-mid);
}

/* ============================================================
   RESPONSIVE GLOBAL
   En pantallas chicas reducimos un poco el tamaño base
   ============================================================ */

@media (max-width: 768px) {
  body {
    font-size: 15px;
  }
}
```

src/styles/index.css CSS · 130 lineas

```css
/* ============================================================
   RESET GLOBAL
   Borramos los márgenes y paddings que el navegador
   agrega por defecto. box-sizing: border-box hace que
   el padding y el borde queden DENTRO del ancho declarado,
   no lo agranden.
   ============================================================ */

* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

/* ============================================================
   VARIABLES GLOBALES
   Todos los colores están en formato rgb(R, G, B).
   R = rojo, G = verde, B = azul. Cada canal va de 0 a 255.
   Ejemplos: rgb(255,0,0) rojo puro · rgb(0,255,0) verde puro
             rgb(0,0,0) negro · rgb(255,255,255) blanco
   Cambiás un valor acá y se actualiza en toda la app.
   ============================================================ */

:root {
  /* Amarillo muy claro, casi blanco cálido */
  --yellow-min: rgb(249, 249, 221);

  /* Verde extra suave, casi blanco frío */
  --green-extra-min: rgb(240, 251, 247);

  /* Fondo general: verde menta muy suave, casi blanco */
  --green-mint: rgb(217, 250, 230);

  /* Verde principal: el verde oscuro de los botones y el header */
  --green-forest: rgb(72, 175, 78);

  /* Verde más oscuro: se usa en hover para dar profundidad */
  --green-forest-dark: rgb(124, 170, 127);

  /* Naranja fuego: color de acento para links y botones secundarios */
  --orange-fire: rgb(255, 111, 0);

  /* Blanco puro: fondos de cards, textos sobre fondos oscuros */
  --white: rgb(255, 255, 255);

  /* Gris oscuro: color principal del texto */
  --gray-dark: rgb(51, 51, 51);

  /* Gris medio: texto secundario, subtítulos, placeholders */
  --gray-mid: rgb(102, 102, 102);

  /* Rojo alerta: errores, stock bajo, mensajes de peligro */
  --red-alert: rgb(220, 53, 69);

  /* Verde confirmación: compras exitosas, stock disponible */
  --green-ok: rgb(40, 167, 69);

  /* Gris claro: bordes de cards e inputs */
  --gray-light: rgb(217, 217, 217);

  /* Sombra suave: da elevación a cards y paneles */
  --shadow: 0 4px 12px rgba(0, 0, 0, 0.08);

  /* Radio de borde: cuánto se redondean las esquinas */
  --radius: 12px;

  /* Velocidad de las animaciones hover */
  --transition: 0.3s ease;

  /* Ancho máximo del contenido central */
  --container-width: 1200px;
}

/* ============================================================
   BASE DEL DOCUMENTO
```

```
   ========================================================== */

html {
  font-size: 16px;
  /* El scroll animado evita saltos bruscos al navegar */
  scroll-behavior: smooth;
}

body {
  background-color: var(--green-mint);
  color: var(--gray-dark);
  font-family: Arial, Helvetica, sans-serif;
  line-height: 1.5;
  min-height: 100vh;
}

/* ==========================================================
   ELEMENTOS BASE
   ========================================================== */

/* Las imágenes nunca rompen su contenedor */
img {
  max-width: 100%;
  display: block;
}

/* Botones, inputs y textareas heredan la tipografía del body */
button,
input,
textarea {
  font: inherit;
}

/* Los links no tienen subrayado por defecto */
a {
  text-decoration: none;
}

/* Las listas no tienen viñetas por defecto */
ul {
  list-style: none;
}

/* ==========================================================
   TÍTULOS
   ========================================================== */

/* Todos los títulos tienen interlineado compacto */
h1,
h2,
h3,
h4 {
  line-height: 1.2;
}
```

test.js JS · 12 lineas

```
fetch("https://mercadopago-carrito-backend.onrender.com/pagos", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    items: [{ title: "test", quantity: 1, unit_price: 1000 }],
    totalFinal: 1000,
    orderId: "123"
  })
})
.then(r => r.json())
.then(console.log);
// ejectar node test.js
```

vercel.json JSON · 9 lineas

```
{
  "rewrites": [
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ]
}
```

vite.config.js JS · 8 lineas

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
})
```